



TRABALLO FIN DE GRAO  
GRAO EN ENXEÑARÍA INFORMÁTICA  
MENCIÓN EN COMPUTACIÓN



# **Desenvolvemento dunha plataforma multilingüe de xeración automática de vídeos de Karaoke con Intelixencia Artificial**

**Estudante:** Gael Fernández Quintela

**Dirección:** Carlos Vázquez Regueiro

A Coruña, setembro de 2025.

*A miña familia e amigos*

### **Agradecementos**

Quero expresar o meu máis sincero agradecemento ao meu director de proxecto, Carlos Vázquez Regueiro, polo seu continuo apoio e por darme a oportunidade de realizar este traballo. Grazas pola confianza depositada en min e sobre todo pola paciencia durante todo o proceso. Tamén quero agradecer á miña familia pola constante motivación e aos meus amigos, tanto aos novos que coñecín como aos de toda a vida.

A todos, moitas grazas.

## Resumo

O presente traballo de fin de grao consiste no desenvolvemento dunha plataforma web que xera vídeos de karaoke de forma automática a partir de cancións obtidas a partir de ligazóns de YouTube ou arquivos [MP4](#). O sistema está optimizado para funcionar en múltiples idiomas, incluíndo linguas minoritarias como o galego. Para dito fin empréganse ferramentas de intelixencia artificial como modelos de recoñecemento de voz, para transcribir as cancións, sincronizar as letras coa música e crear vídeos con subtítulos animados.

En primeiro lugar, sepáranse as pistas de voz e as instrumentais empregando [Demucs](#) con aceleración [GPU](#). Posteriormente transcríbese a voz con WhisperX e derivados, obtendo marcas de tempo a nivel de palabra e incluso de sílaba para garantir unha sincronización precisa, ou ben se realiza *forced alignment* cunha letra proporcionada polo propio usuario, opción particularmente valiosa para os idiomas minoritarios coma o galego onde os modelos de aprendizaxe profundo actuais aínda non están suficientemente entrenados. Despois un módulo descrito en Python e sustentado por MoviePy, FFmpeg e outras ferramentas xera o vídeo e os subtítulos aplicando un resaltado progresivo silábico segundo vai avanzando o/a cantante. Todo isto baixo unha arquitectura de microservizos Docker.

A interface web con Flask e o procesamento asíncrono con Celery permiten unha experiencia de usuario que inclúe biblioteca de cancións, reproductor avanzado con sincronización de múltiples pistas audio e acceso público mediante ngrok. O proxecto vai moito máis alá da simple integración de ferramentas, o sistema conta con multitude de módulos e algoritmos complexos que fan que todo funcione correctamente.

## Abstract

This final degree project consists of the development of a Web platform that automatically generates karaoke videos from songs obtained through YouTube links or [MP4](#) files. The system is optimized to work in multiple languages, including minority languages such as Galician. For this purpose, artificial intelligence tools are employed such as speech recognition models, to transcribe songs, synchronize lyrics with music, and create videos with animated subtitles.

First, vocal and instrumental tracks are separated using [Demucs](#) with [GPU](#) acceleration. Subsequently, the vocals are transcribed with WhisperX and derivatives, obtaining timestamps at word and even syllable level to ensure precise synchronization, or alternatively, forced alignment is performed with lyrics provided by the user themselves, an option particularly valuable for minority languages like Galician where current deep learning models are



not yet sufficiently trained. Then, a module described in Python and supported by MoviePy, FFmpeg, and other tools generates the video and subtitles applying progressive syllabic highlighting as the singer advances. All of this under a Docker microservices architecture.

The web interface with Flask and asynchronous processing with Celery enable a user experience that includes a song library, an advanced player with multi-track audio synchronization, and public access through ngrok. The project goes far beyond simple tool integration, as the system incorporates multiple complex modules and algorithms that ensure everything works correctly.

**Palabras chave:**

- Karaoke automático
- Separación de fontes
- WhisperX
- Diarización de falantes
- Microservizos
- Aliñación forzada
- Composición de vídeo
- Flask
- Renderizado silábico
- Sincronización temporal

**Keywords:**

- Automatic karaoke
- Source separation
- WhisperX
- Speaker diarization
- Microservices
- Forced alignment
- Video composition
- Flask
- Syllabic rendering
- Temporal synchronization

# Índice Xeral

---

|          |                                                      |           |
|----------|------------------------------------------------------|-----------|
| <b>1</b> | <b>Introdución</b>                                   | <b>1</b>  |
| 1.1      | Motivación . . . . .                                 | 1         |
| 1.2      | Obxectivos . . . . .                                 | 2         |
| 1.3      | Proposta . . . . .                                   | 3         |
| 1.4      | Estrutura do documento . . . . .                     | 4         |
| <b>2</b> | <b>Fundamentos e estado da arte</b>                  | <b>5</b>  |
| 2.1      | Aprendizaxe profunda aplicada ao audio . . . . .     | 5         |
| 2.2      | Separación de fontes musicais . . . . .              | 7         |
| 2.3      | Recoñecemento e aliñamento da fala . . . . .         | 9         |
| 2.3.1    | Recoñecemento automático da fala . . . . .           | 9         |
| 2.3.2    | Aliñamento temporal . . . . .                        | 10        |
| 2.4      | Renderizado e contido multimedia . . . . .           | 12        |
| 2.5      | Estado da arte en sistemas de karaoke . . . . .      | 15        |
| 2.5.1    | Sistemas comerciais . . . . .                        | 15        |
| 2.5.2    | Proxectos de código aberto e investigación . . . . . | 16        |
| 2.5.3    | Limitacións dos enfoques existentes . . . . .        | 16        |
| 2.6      | Ferramentas e tecnoloxías . . . . .                  | 17        |
| 2.6.1    | Librerías de intelixencia artificial . . . . .       | 17        |
| 2.6.2    | Librerías de Python . . . . .                        | 18        |
| 2.6.3    | Desenvolvemento web . . . . .                        | 20        |
| 2.6.4    | Ferramentas de desenvolvemento . . . . .             | 21        |
| 2.6.5    | Documentación . . . . .                              | 22        |
| 2.6.6    | Outras ferramentas . . . . .                         | 22        |
| <b>3</b> | <b>Planificación e xestión do proxecto</b>           | <b>24</b> |
| 3.1      | Metodoloxía de desenvolvemento . . . . .             | 24        |
| 3.2      | Análise de requisitos . . . . .                      | 25        |

|          |                                                          |           |
|----------|----------------------------------------------------------|-----------|
| 3.2.1    | Requisitos funcionais . . . . .                          | 25        |
| 3.2.2    | Requisitos non funcionais . . . . .                      | 26        |
| 3.3      | Casos de uso . . . . .                                   | 26        |
| 3.4      | Planificación e seguimento . . . . .                     | 27        |
| 3.4.1    | Planificación do proxecto . . . . .                      | 27        |
| 3.4.2    | Seguimento . . . . .                                     | 29        |
| 3.4.3    | Diagrama de Gantt . . . . .                              | 30        |
| 3.5      | Análise de riscos e custos . . . . .                     | 30        |
| 3.5.1    | Análise de riscos . . . . .                              | 31        |
| 3.5.2    | Recursos utilizados . . . . .                            | 32        |
| 3.5.3    | Análise de custos . . . . .                              | 32        |
| <b>4</b> | <b>Arquitectura do sistema</b>                           | <b>34</b> |
| 4.1      | Visión xeral da arquitectura . . . . .                   | 34        |
| 4.1.1    | Principios de deseño adoptados e xustificación . . . . . | 34        |
| 4.2      | Arquitectura de contedores . . . . .                     | 36        |
| 4.2.1    | Xestión de recursos e hardware . . . . .                 | 36        |
| 4.2.2    | Redes e comunicación entre contedores . . . . .          | 36        |
| 4.3      | Compoñentes principais do sistema . . . . .              | 38        |
| 4.3.1    | Capa de presentación . . . . .                           | 38        |
| 4.3.2    | Capa de procesamento asíncrono . . . . .                 | 38        |
| 4.3.3    | Capa de servizos de IA . . . . .                         | 39        |
| 4.3.4    | Capa de renderizado . . . . .                            | 39        |
| 4.3.5    | Capa de persistencia . . . . .                           | 40        |
| 4.4      | Modelo de datos . . . . .                                | 40        |
| 4.4.1    | Datos internos compartidos . . . . .                     | 40        |
| 4.4.2    | Formatos de intercambio . . . . .                        | 42        |
| 4.4.3    | Protocolos de comunicación . . . . .                     | 44        |
| 4.5      | Fluxo de compoñentes principais . . . . .                | 45        |
| 4.5.1    | <i>Pipeline</i> de procesamento automático . . . . .     | 45        |
| 4.5.2    | <i>Pipeline</i> de procesamento manual . . . . .         | 47        |
| 4.5.3    | <i>Pipeline</i> de só instrumental . . . . .             | 48        |
| 4.6      | Patróns de deseño implementados . . . . .                | 48        |
| 4.6.1    | Patrón <i>Command</i> . . . . .                          | 48        |
| 4.6.2    | Patrón <i>Strategy</i> . . . . .                         | 48        |
| 4.6.3    | Patrón <i>Template Method</i> . . . . .                  | 49        |

|          |                                                                                                |           |
|----------|------------------------------------------------------------------------------------------------|-----------|
| <b>5</b> | <b>Detalles de implementación</b>                                                              | <b>50</b> |
| 5.1      | Implementación de contedores . . . . .                                                         | 50        |
| 5.1.1    | Configuración do ecosistema Docker . . . . .                                                   | 50        |
| 5.1.2    | Contedor Flask principal (demucs_container) . . . . .                                          | 50        |
| 5.1.3    | Contedor <i>worker</i> de Celery (celery_worker_container) . . . . .                           | 51        |
| 5.1.4    | Contedor WhisperX (whisperx_container) . . . . .                                               | 51        |
| 5.1.5    | Contedor Redis (redis_container) . . . . .                                                     | 52        |
| 5.1.6    | Contedor Ngrok (ngrok_container) . . . . .                                                     | 52        |
| 5.2      | Pipeline de procesamento de audio . . . . .                                                    | 52        |
| 5.2.1    | Normalización de vídeo . . . . .                                                               | 52        |
| 5.2.2    | Extracción e conversión de audio . . . . .                                                     | 53        |
| 5.2.3    | Separación de fontes con Demucs . . . . .                                                      | 53        |
| 5.3      | Sistema de transcripción e aliñamento . . . . .                                                | 54        |
| 5.3.1    | Transcripción automática: Integración Whisper + WhisperX . . . . .                             | 54        |
| 5.3.2    | <i>Forced alignment</i> : Algoritmos de aliñamento temporal . . . . .                          | 54        |
| 5.3.3    | Procesamento de texto: Normalización de letras manuais . . . . .                               | 56        |
| 5.3.4    | <i>Speaker diarization</i> : Implementación con pyannote.audio . . . . .                       | 57        |
| 5.4      | Motor de renderizado de karaoke . . . . .                                                      | 57        |
| 5.4.1    | Renderizado silábico: Algoritmo pyphen + resaltado progresivo . . . . .                        | 58        |
| 5.4.2    | Xeración de clips de texto: create_karaoke_text_clip() . . . . .                               | 59        |
| 5.4.3    | Composición multicapa: MoviePy CompositeVideoClip . . . . .                                    | 59        |
| 5.5      | Interface web e xestión de sesións . . . . .                                                   | 60        |
| 5.5.1    | Arquitectura Flask: Rutas, <i>templates</i> , formularios . . . . .                            | 60        |
| 5.5.2    | Procesamento asíncrono: Integración Celery + tracking de progreso . . . . .                    | 61        |
| 5.5.3    | Sistema de biblioteca: Base de datos SQLite, metadata_utils . . . . .                          | 61        |
| 5.5.4    | Reprodutor web: player.html, controis personalizados . . . . .                                 | 62        |
| 5.6      | Optimizacións específicas . . . . .                                                            | 62        |
| 5.6.1    | <i>Logging e debugging</i> : Sistema de trazabilidade de erros . . . . .                       | 62        |
| 5.6.2    | Cancelación de tarefas con ProcessingCancelledException . . . . .                              | 63        |
| 5.6.3    | Validación de entrada: <i>security_config.py</i> , sanitización de arquivos . . . . .          | 64        |
| 5.6.4    | Xestión de <i>tokens</i> : HuggingFace <i>tokens</i> para <i>speaker diarization</i> . . . . . | 64        |
| <b>6</b> | <b>Validación e probas</b>                                                                     | <b>65</b> |
| 6.1      | Metodoloxía de validación . . . . .                                                            | 65        |
| 6.1.1    | Criterios de avaliación . . . . .                                                              | 65        |
| 6.1.2    | Conxunto de datos de proba . . . . .                                                           | 66        |
| 6.1.3    | Configuración do entorno de probas . . . . .                                                   | 66        |
| 6.2      | Validación por compoñentes . . . . .                                                           | 67        |

|          |                                                                    |           |
|----------|--------------------------------------------------------------------|-----------|
| 6.2.1    | Separación de fontes musicais . . . . .                            | 67        |
| 6.2.2    | Transcrición e aliñamento temporal . . . . .                       | 69        |
| 6.2.3    | <i>Speaker diarization</i> . . . . .                               | 71        |
| 6.2.4    | Renderizado de vídeo e sincronización . . . . .                    | 71        |
| 6.3      | Probas de integración do sistema . . . . .                         | 72        |
| 6.3.1    | Pipeline completo automático . . . . .                             | 72        |
| 6.3.2    | Pipeline con letras manuais . . . . .                              | 73        |
| 6.3.3    | Xestión de erros e recuperación . . . . .                          | 73        |
| 6.3.4    | Rendemento con múltiples usuarios . . . . .                        | 74        |
| 6.4      | Validación en casos de uso reais . . . . .                         | 74        |
| 6.5      | Análise comparativo . . . . .                                      | 75        |
| 6.6      | Limitacións identificadas . . . . .                                | 76        |
| <b>7</b> | <b>Conclusións e traballo futuro</b>                               | <b>77</b> |
| 7.1      | Conclusións . . . . .                                              | 77        |
| 7.2      | Traballo futuro . . . . .                                          | 79        |
| <b>A</b> | <b>Código adicional</b>                                            | <b>81</b> |
| A.1      | Detección automática de GPU . . . . .                              | 81        |
| A.2      | <i>Endpoint</i> principal de WhisperX . . . . .                    | 82        |
| A.3      | Algoritmo de progreso temporal híbrido . . . . .                   | 82        |
| A.4      | Función xeradora de clips de texto con silabificación . . . . .    | 82        |
| A.5      | Estrutura de datos para silabificación . . . . .                   | 82        |
| A.6      | Formato SRT con <i>speaker diarization</i> . . . . .               | 85        |
| A.7      | Configuración GPU para contador principal . . . . .                | 85        |
| A.8      | Coordinación de procesos en <code>celery_tasks.py</code> . . . . . | 85        |
| <b>B</b> | <b>Táboas adicionais</b>                                           | <b>87</b> |
| B.1      | Casos de uso . . . . .                                             | 87        |
| B.2      | Resultados do WER adicionais . . . . .                             | 93        |
| <b>C</b> | <b>Imaxes adicionais</b>                                           | <b>94</b> |
| C.1      | Funcionamento do encoder-decoder de Demucs . . . . .               | 94        |
| C.2      | Análise visual da separación de fontes . . . . .                   | 94        |
| C.3      | Diagrama de fluxo do modo de obtención da instrumental . . . . .   | 94        |
| C.4      | Diagrama da metodoloxía seguida . . . . .                          | 94        |
| C.5      | Diagramas de clases do sistema . . . . .                           | 94        |
| C.6      | Imaxes da interface web adicionais . . . . .                       | 101       |

|                              |            |
|------------------------------|------------|
| <b>Relación de Acrónimos</b> | <b>103</b> |
| <b>Glosario</b>              | <b>105</b> |
| <b>Bibliografía</b>          | <b>106</b> |

# Índice de Figuras

---

|     |                                                                           |    |
|-----|---------------------------------------------------------------------------|----|
| 2.1 | Descrición xeral da Transformada de Fourier a curto prazo . . . . .       | 6  |
| 2.2 | Estrutura da U-Net en Demucs . . . . .                                    | 9  |
| 2.3 | Algoritmo de decodificación CTC . . . . .                                 | 11 |
| 2.4 | Ferramentas de <i>forced alignment</i> . . . . .                          | 13 |
| 2.5 | Diagrama de funcionamento de WhisperX . . . . .                           | 18 |
| 3.1 | Esquema dunha metodoloxía de desenvolvemento iterativa xeral . . . . .    | 25 |
| 3.2 | Diagrama de casos de uso do sistema . . . . .                             | 27 |
| 3.3 | Diagrama de Gantt final do proxecto . . . . .                             | 30 |
| 4.1 | Arquitectura xeral do sistema de karaoke automático . . . . .             | 35 |
| 4.2 | Arquitectura detallada de contedores con volumes compartidos . . . . .    | 37 |
| 4.3 | Fluxo de detección e configuración de hardware . . . . .                  | 37 |
| 4.4 | Fluxo de datos e formatos ao longo do sistema . . . . .                   | 41 |
| 4.5 | Diagrama de secuencia do modo automático . . . . .                        | 46 |
| 4.6 | Diagrama de secuencia do modo manual . . . . .                            | 47 |
| 4.7 | Diagrama UML do patrón Command implementado con Celery . . . . .          | 49 |
| 5.1 | Diagrama de decisión Faster-Whisper vs WhisperX segundo o modo de entrada | 55 |
| 5.2 | Diagrama de fluxo das estratexias de normalización . . . . .              | 55 |
| 5.3 | Diagrama de validación de formularios . . . . .                           | 60 |
| 5.4 | Formularios da aplicación web en modo automático e manual . . . . .       | 61 |
| 5.5 | Interface do reprodutor web . . . . .                                     | 62 |
| C.1 | Funcionamento do encoder-decoder de Demucs . . . . .                      | 95 |
| C.2 | Espectrograma da mezcla orixinal ( <i>mixture</i> ). . . . .              | 95 |
| C.3 | Espectrograma da pista de batería ( <i>drums</i> ). . . . .               | 96 |
| C.4 | Espectrograma da pista de baixos ( <i>bass</i> ). . . . .                 | 96 |

|      |                                                                         |     |
|------|-------------------------------------------------------------------------|-----|
| C.5  | Espectrograma da pista de outros instrumentos ( <i>other</i> ). . . . . | 96  |
| C.6  | Espectrograma da pista vocal ( <i>vocals</i> ). . . . .                 | 97  |
| C.7  | Diagrama de secuencia para a opción de obter a instrumental . . . . .   | 97  |
| C.8  | Esquema da metodoloxía iterativa seguida . . . . .                      | 98  |
| C.9  | Diagrama de clases da aplicación principal parte 1. . . . .             | 99  |
| C.10 | Diagrama de clases da aplicación principal parte 2. . . . .             | 100 |
| C.11 | Cabeceira da páxina principal . . . . .                                 | 101 |
| C.12 | Formulario para o modo de só instrumental da páxina principal . . . . . | 101 |
| C.13 | Interface da biblioteca do sistema . . . . .                            | 102 |



# Índice de Táboas

---

|     |                                                                               |    |
|-----|-------------------------------------------------------------------------------|----|
| 2.1 | Comparación de arquitecturas neuronais para procesamento de audio . . . . .   | 7  |
| 2.2 | Comparación de <i>frameworks</i> para renderizado multimedia. . . . .         | 14 |
| 2.3 | Comparación de sistemas de karaoke existentes . . . . .                       | 16 |
| 3.1 | Resumo dos atrasos identificados no proxecto . . . . .                        | 29 |
| 3.2 | Identificación e análise dos riscos do proxecto. . . . .                      | 31 |
| 3.3 | Resumo de custos totais do proxecto . . . . .                                 | 33 |
| 4.1 | Distribución de recursos por contedor . . . . .                               | 36 |
| 4.2 | Resumo de patróns de deseño implementados . . . . .                           | 48 |
| 5.1 | Estratexias de normalización de vídeo implementadas . . . . .                 | 53 |
| 5.2 | Sistema de cores para diferenciación de speakers . . . . .                    | 57 |
| 5.3 | Capas de composición do vídeo de karaoke . . . . .                            | 60 |
| 6.1 | dataset de validación utilizado nas probas . . . . .                          | 66 |
| 6.2 | Tempos reais de procesamento de Demucs obtidos dos logs dos contedores . .    | 68 |
| 6.3 | Comparación entre diferentes modelos de Whisper no modo automático . . .      | 69 |
| 6.4 | Precisión da detección de falantes . . . . .                                  | 71 |
| 6.5 | Tempos detallados do <i>pipeline</i> automático por canción . . . . .         | 72 |
| 6.6 | Tempos detallados do <i>pipeline</i> con letras manuais por canción . . . . . | 73 |
| 6.7 | Comparación do sistema con ferramentas existentes para cancións en galego .   | 75 |
| B.1 | CU-01: Xeración de karaoke con transcripción automática . . . . .             | 87 |
| B.2 | CU-02: Xeración de karaoke con letra manual . . . . .                         | 88 |
| B.3 | CU-03: Procesamento con arquivo MP4 local . . . . .                           | 88 |
| B.4 | CU-04: Xeración de pista instrumental . . . . .                               | 89 |
| B.5 | CU-05: Xestión de múltiples voces . . . . .                                   | 89 |
| B.6 | CU-06: Reprodución de vídeo de karaoke . . . . .                              | 90 |

|      |                                                                            |    |
|------|----------------------------------------------------------------------------|----|
| B.7  | CU-07: Xestión da biblioteca de cancións . . . . .                         | 90 |
| B.8  | CU-08: Descarga de ficheiros xerados . . . . .                             | 91 |
| B.9  | CU-9: Cancelación de procesamento . . . . .                                | 91 |
| B.10 | CU-10: Configuración de modelo Whisper . . . . .                           | 92 |
| B.11 | Resultados do WER das cancións individuais do dataset utilizadas . . . . . | 93 |

# Índice de códigos

---

|     |                                                                           |    |
|-----|---------------------------------------------------------------------------|----|
| 4.1 | Estrutura de segmentación temporal . . . . .                              | 42 |
| 4.2 | Configuración de diarización por falantes . . . . .                       | 43 |
| 4.3 | Formato solicitude WhisperX . . . . .                                     | 43 |
| 4.4 | Formato resposta WhisperX . . . . .                                       | 43 |
| 4.5 | Configuración Redis para Celery . . . . .                                 | 45 |
| 5.1 | Configuración de volumes compartidos de Docker . . . . .                  | 51 |
| 5.2 | Configuración do worker Celery . . . . .                                  | 51 |
| 5.3 | Configuración de Ngrok . . . . .                                          | 52 |
| 5.4 | Distribución proporcional intelixente . . . . .                           | 56 |
| 5.5 | Función para asignar cada palabra ao seu falante correspondente . . . . . | 58 |
| 5.6 | Algoritmo de renderizado silábico avanzado . . . . .                      | 59 |
| 5.7 | Algoritmo de sincronización multimedia . . . . .                          | 63 |
| 5.8 | Función de chequeo de cancelación . . . . .                               | 63 |
| A.1 | Detección automática de GPU . . . . .                                     | 81 |
| A.2 | Endpoint principal de WhisperX . . . . .                                  | 82 |
| A.3 | Algoritmo de progreso temporal híbrido . . . . .                          | 83 |
| A.4 | Función xeradora de clips de texto con sincronización silábica . . . . .  | 84 |
| A.5 | Estrutura de datos para silabificación . . . . .                          | 85 |
| A.6 | Formato SRT con speaker diarization . . . . .                             | 85 |
| A.7 | Configuración GPU para contador principal . . . . .                       | 86 |
| A.8 | Coordinación de procesos en celery_tasks.py . . . . .                     | 86 |

# Introdución

---

**N**ESTE primeiro capítulo presentase o contexto do proxecto xunto coas motivacións e o interese que sustentan a súa realización. Exporanse os obxectivos xerais e específicos do traballo ademais dunha proposta e pechase coa descrición da estrutura desta memoria.

## 1.1 Motivación

A música galega actual está a vivir un claro momento de auxe. Grupos como Fillas de Cassandra, The Rapants, De Ninghures ou Mondra, enchen salas de concertos e están a adquirir miles de oíntes nas diferentes plataformas musicais. O panorama musical galego actual adáptase a todas as xeracións e estilos de música. Xa é habitual ver artistas esgotando entradas para os seus concertos en poucos días e gañar premios a nivel nacional.

A motivación deste proxecto xorde de formar parte dunha asociación cultural onde se organizan regularmente diferentes eventos, como monólogos, lecturas de libros, festas culturais, exposición de fotografías, etc. Entre estos eventos atópase un karaoke, durante o cal unha persoa é responsable de recibir as solicitudes dos participantes e buscar versións en formato karaoke das cancións que se solicitan en YouTube. Porén, con frecuencia as peticións inclúen cancións moi variadas, a miúdo pouco coñecidas e en distintos idiomas, sobre todo en galego. Isto é normal xa que a asociación atópase nun pequeno pobo de Galicia. O principal problema é que moitas destas cancións non teñen versións de karaoke dispoñibles en YouTube, o que dificulta cumprir coas solicitudes.

Debido á procedencia galega das cancións solicitadas, gran parte delas non teñen versións de karaoke accesibles. Ante a falta de opcións, unha posibilidade podería ser utilizar un creador de vídeos de karaoke online que permite xerar versións a partir de ligazóns de YouTube. Este software xera o vídeo coa pista instrumental e as letras sincronizadas, destacando o texto conforme avanza a canción. Porén, estes programas só funciona correctamente para cancións en español, inglés e outros idiomas maioritarios, mentres que presentaba problemas

significativos con idiomas minoritarios como o galego. Esta limitación espertou o interese en desenvolver unha solución máis robusta e versátil.

## 1.2 Obxectivos

O obxectivo principal deste proxecto é o desenvolvemento dunha plataforma para crear vídeos de karaoke de forma automática, a partir de cancións obtidas de YouTube ou arquivos MP4, e optimizada para funcionar en múltiples idiomas, incluíndo linguas minoritarias como o galego. Este sistema procesará arquivos de vídeo locais ou ligazóns URL e creará vídeos para karaoke. O sistema empregará ferramentas de intelixencia artificial, como modelos de recoñecemento de voz, para transcribir as cancións, sincronizar as letras coa música e crear vídeos con subtítulos animados. Para logralo, defínense os seguintes obxectivos específicos:

1. Desenvolver un sistema de separación de *stems* automático que inclúa as pistas vocais e instrumentais de calquera canción.
2. Integrar un sistema de transcripción automática baseado en WhisperX, capaz de detectar o idioma da canción e xerar subtítulos sincronizados coa música a nivel de palabra.
3. Conseguiir unha sincronización precisa entre o texto da canción (pista vocal) e o tempo da música (pista instrumental).
4. Desenvolver un sistema de xeración de subtítulos cun resaltado progresivo silábico, que mellore a experiencia visual do karaoke adaptándose ao ritmo do cantante
5. Incorporar unha funcionalidade de aliñamento forzado que permita ao usuario introducir manualmente a letra da canción, asegurando que a sincronización sexa precisa incluso en linguas pouco adestradas nos modelos de transcripción de código aberto.
6. Crear unha interface web sinxela e accesible que permita ao usuario escoller entre o modo automático, o modo con letra manual ou o modo de obter só a instrumental, e que xestione o proceso de xeración dos vídeos resultantes, podendo reproducilos na propia web e poder gardar as cancións procesadas nunha biblioteca.
7. Diseñar unha arquitectura baseada en contedores Docker que permita integrar as diferentes ferramentas especializadas, mantendo illadas as súas dependencias para garantir a compatibilidade entre elas e a facilidade de mantemento.
8. Habilitar a posibilidade de realizar diferenciación de voces cando nunha canción existen varios cantantes e resaltar a letra de cada un nunha cor diferente (*speaker diarization*).
9. Abrir un sistema de túneles seguros Ngrok que permita compartir a aplicación publicamente sen necesidade de instalar o entorno completo.

### 1.3 Proposta

Á vista dos obxectivos establecidos na sección 1.2, este proxecto parte da seguinte **hipótese de traballo**: *é posible combinar técnicas modernas de intelixencia artificial para audio (separación de fontes, recoñecemento de fala) con ferramentas de desenvolvemento web para xerar vídeos de karaoke de maneira automática, obtendo resultados comparables en calidade cos karaoke preparados manualmente.*

Para validar isto, requírese demostrar que os modelos profundos utilizados poden ser modificados para integrarse nun mesmo sistema xunto con novos módulos creados especificamente para este obxectivo. É especialmente relevante comprobar se esta aproximación é viable para linguas minoritarias como o galego, onde os modelos de transcripción presentan limitacións significativas.

- **Alcance e innovacións do proxecto:** A proposta diferénciase de solucións existentes en varios aspectos. Primeiro, a integración de múltiples compoñentes nunha soa aplicación web, resolvendo os problemas de compatibilización entre estas tecnoloxías. Segundo, a automatización completa do proceso dende a entrada dunha [URL](#) de YouTube ou ficheiro local ata a xeración do vídeo, eliminando a intervención manual. Terceiro, o soporte especializado para linguas minoritarias mediante a implementación de *forced alignment* que compensa as limitacións dos modelos de transcripción actuais.

Ademais, o proxecto ten varias melloras técnicas específicas que veremos ao longo deste documento.

- **Metodoloxía e validación:** Empregarase unha metodoloxía que combina desenvolvemento incremental con avaliación continua e o sistema construírse seguindo unha arquitectura de microservizos containerizada. A validación realizarase mediante probas cun conxunto diverso de cancións en múltiples idiomas, incluíndo especialmente contido en galego, avaliando a calidade técnica do resultado (precisión da separación, exactitude da transcripción, sincronización temporal, etc).

O éxito do proxecto medirase pola capacidade do sistema para producir vídeos que sexan realmente utilizables por un usuario, que a letra esté perfectamente transcrita e ben sincronizada, e que o resultado sexa comparable a un vídeo de karaoke feito con edición.

Realmente inténtase que este sistema teña un valor diferencial con respecto a outros. Mentres que as solucións comerciais actuais dependen de catálogos limitados, este traballo intenta contribuír ao estado da arte proporcionando unha solución para a creación de contido multimedia en linguas minoritarias grazas ao aliñamento forzado.

## 1.4 Estrutura do documento

Son sete os capítulos nos que está estruturada esta memoria. A continuación describirase cada un deles brevemente:

- **Capítulo 1 - Introducción**

Este capítulo establece o contexto e a xustificación do proxecto, abordando os obxectivos xerais e específicos. Tamén se describe a estrutura completa da memoria, ofrecendo unha visión simplificada da proposta.

- **Capítulo 2 - Fundamentos e estado da arte**

Revísanse os conceptos fundamentais de aprendizaxe profunda aplicada ao audio, as tecnoloxías empregadas e se analiza o estado da arte en sistemas de karaoke automático.

- **Capítulo 3 - Planificación e xestión do proxecto**

Descríbese a metodoloxía empregada, os requisitos do sistema, a planificación temporal e a análise de riscos e custos, mostrando a planificación inicial e o seguimento durante o proceso.

- **Capítulo 4 - Arquitectura do sistema**

Este capítulo proporciona unha visión detallada do deseño da arquitectura, os compoñentes principais, o modelo de datos e a comunicación entre servizos

- **Capítulo 5 - Detalles de implementación**

Preséntanse os detalles técnicos, incluíndo os compoñentes máis complexos do código, as decisións de implementación e as solucións inxenieras.

- **Capítulo 6 - Validación e probas**

Analízanse os resultados obtidos mediante probas con diferentes tipos de cancións e idiomas, incluíndo métricas de calidade e rendemento.

- **Capítulo 7 - Conclusións**

Preséntanse as conclusións do traballo, as limitacións identificadas, as leccións aprendidas e as liñas de traballo futuro.

# Fundamentos e estado da arte

---

NESTE capítulo descríbese o marco teórico e tecnolóxico no que se basea o proxecto. De-tállanse conceptos como aprendizaxe profunda aplicada ao audio e as [redes neuronais](#) para procesamento de sinal, seguido dunha revisión de tecnoloxías específicas empregadas. Finalmente, analízase o estado da arte en sistemas automáticos de karaoke.

## 2.1 Aprendizaxe profunda aplicada ao audio

A aprendizaxe profunda é unha rama da intelixencia artificial que se basea no uso de [redes neuronais](#) artificiais con moitas capas para aprender patróns a partir de grandes cantidades de datos. No contexto do procesamento de audio, estes modelos aprenden a extraer características cada vez máis abstractas: dende patróns espectrais nas primeiras capas ata estruturas musicais e lingüísticas complexas nas capas superiores.

O procesamento de sinais de audio constitúe a base fundamental para a creación automática de vídeos de karaoke. Os sinais de audio son representacións dixitais de ondas sonoras que conteñen información complexa sobre múltiples fontes. Estes sinais presentan características que inflúen no deseño de arquitecturas neuronais. Matematicamente, un sinal de audio discreto pódese representar como unha secuencia temporal  $x[n]$  onde  $n$  indica o índice temporal. A diferenza das imaxes bidimensionais, o audio é unha onda unidimensional no tempo, polo que acostúmase a transformar o audio a representacións tempo-frecuencia antes de alimentalo á rede neuronal.

A transformada de Fourier a curto prazo ([STFT](#)) permite obter unha representación tempo-frecuencia fundamental para o procesamento neuronal. Fragmenta o sinal de audio en seccións máis pequenas mediante unha ventá de tempo deslizante, logo toma a [FFT](#) de cada sección e combínalas. Deste xeito é quen de capturar as variacións de frecuencia ao longo do tempo [1]:



$$X[m, k] = \sum_{n=0}^{N-1} x[n + mH] \cdot w[n] \cdot e^{-j2\pi kn/N}$$

onde  $m$  é o índice temporal,  $k$  o índice frecuencial,  $H$  o salto entre ventás e  $w[n]$  a función ventá aplicada.

Esta dualidade tempo-frecuencia é necesaria para entender como as **redes neuronais** procesan información musical, xa que permite capturar tanto a evolución temporal das notas como a súa estrutura harmónica ao mesmo tempo. Os espectrogramas resultantes facilitan a aplicación de técnicas de visión por computador ao procesamento de audio. Na figura 2.1 pódese observar a descrición xeral da **STFT**.

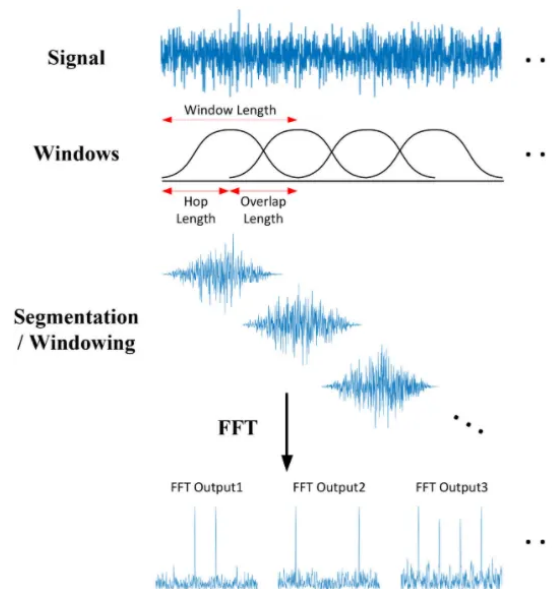


Figura 2.1: Descrición xeral da Transformada de Fourier a curto prazo

As **redes neuronais** profundas teñen unha gran versatilidade para modelar sinais de diferente natureza (audio, imaxe, etc). Existen varias arquitecturas posibles: as redes convolucionais (**CNN**) empréganse para extraer características locais fronte a variacións no sinal, por exemplo aplicando filtros convolucionais sobre espectrogramas ou directamente sobre a onda sonora para capturar patróns de frecuencia. As **CNN** poden aprender filtros que detectan características como a voz, golpes de batería ou outras variantes analizando ventás temporais curtas do sinal. Por outro lado, as redes recorrentes (**RNN**) adoitan facer tarefas onde a dependencia temporal é longa, como o *speech recognition*. Un modelo recorrente pode memorizar contextos previos e modelar secuencias de palabras ou notas musicais ao longo do tempo.

No contexto do **estado da arte** actual, as tecnoloxías de aprendizaxe profunda para audio evolucionaron nos últimos anos. Os modelos *Transformer*, que orixinalmente revolucionaron

| Arquitectura        | Fortalezas principais                   | Limitacións                              | Aplicacións en audio                   |
|---------------------|-----------------------------------------|------------------------------------------|----------------------------------------|
| <i>CNN</i>          | Detección de patróns locais, eficiente  | Contexto temporal limitado               | Clasificación de eventos               |
| <i>RNN/LSTM</i>     | Modelaxe temporal, secuencias variables | Procesamento secuencial lento            | ASR, xeración secuencial               |
| <i>Transformers</i> | Atención global, paralelización         | Complexidade cuadrática, grandes dataset | ASR multilingüe, modelos preentrenados |
| <i>U-Net</i>        | Preserva detalles, mapeo directo        | Para reconstrución                       | Separación de fontes, redución ruído   |
| <i>GANs</i>         | Xeración realista, calidade perceptual  | Adestramento inestable                   | Síntese de voz, mellora de audio       |
| <i>Híbridas</i>     | Combina múltiples vantaxes              | Complexidade de deseño                   | Modelos estado da arte                 |

Táboa 2.1: Comparación de arquitecturas neuronais para procesamento de audio

o procesamento de linguaxe natural, adaptáronse con grande éxito ao dominio do audio.

Unha das tendencias máis prometedoras é a creación de modelos preentrenados que aprenden representacións universais de audio. Seguindo o paradigma das linguaxes naturais, sistemas como wav2vec 2.0 [2] demostran como un modelo pode aprender características xerais do audio usando grandes cantidades de datos. Despois, este coñecemento base pódese adaptar a tarefas específicas, reducindo os requisitos de datos especializados e tempo de adestramento.

As arquitecturas híbridas representan outra innovación, combinando as mellores características de diferentes enfoques. Por exemplo, WaveNet [3] mostra como é posible procesar información a múltiples escalas temporais ao mesmo tempo. Este tipo de modelos permiten capturar tanto os matices instantáneos como os patróns de longo prazo que hai na música. A táboa 2.1 presenta unha comparación das principais arquitecturas neuronais empregadas no procesamento de audio, mostrando as súas características e ámbitos de aplicación.

O procesamento multimodal está emerxendo de xeito interesante, onde os sistemas aprenden a traballar conxuntamente con audio, texto e outras formas de información. Esta capacidade abre as portas a aplicacións que requiren comprensión simultánea do contido sonoro e o seu significado. Neste ámbito, o enfoque cara o aprendizaxe *end-to-end* marca outro cambio fundamental na forma de abordar os problemas de audio. En lugar de deseñar múltiples compoñentes independentes, os sistemas modernos aprenden automaticamente a mellor forma de procesar o audio dende a entrada ata o resultado final.

## 2.2 Separación de fontes musicais

A separación de fontes musicais, tamén coñecida como *source separation*, consiste en extraer os diferentes compoñentes que forman unha gravación musical. No contexto específico

deste proxecto, Débese suprimir a voz do cantante ou cantantes, deixando só a parte musical (*backing track*) para que se poida usar como base para cantar sobre ela. Isto implica desfacer a mestura feita previamente en estudio por un equipo de produción.

A separación de fontes de audio mediante aprendizaxe profundo baséase na capacidade das *redes neuronais* para aprender representacións espectrais de diferentes compoñentes sonoras. Como xa se comentou na sección 2.1, o proceso comeza coa transformación do sinal de audio temporal ao dominio da frecuencia mediante a *STFT*, xerando un espectrograma que contén diversa información.

O problema da separación de fontes pode modelarse como un problema de factorización matricial ou descomposición de sinais. Dado un sinal mixturado  $x(t)$  que representa a suma de múltiples sinais fonte  $s_i(t)$ , o obxectivo é recuperar cada un dos sinais orixinais:

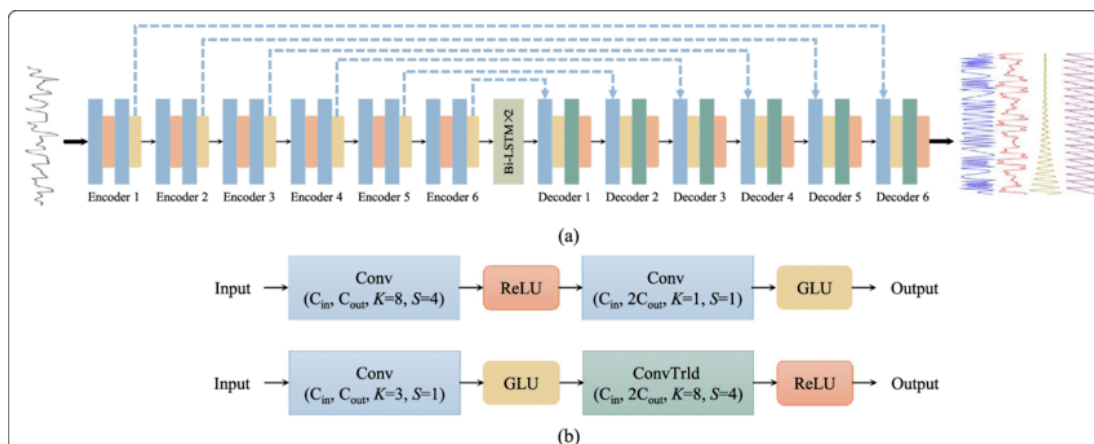
$$x(t) = \sum_{i=1}^N s_i(t) \quad (2.1)$$

onde  $N$  representa o número de fontes presentes na mestura. No caso concreto da separación vocal-instrumental, temos  $N = 2$ , sendo  $s_1(t)$  a pista vocal e  $s_2(t)$  a pista instrumental.

As propiedades acústicas da voz humana son importantes nesta separación. A voz presenta certas características: un rango de frecuencias específico (aproximadamente entre 80 Hz e 8 kHz para a voz humana), patróns temporais asociados coa respiración, etc. Estas características permiten que os algoritmos de separación poidan identificar a compoñente vocal.

O desenvolvemento de técnicas para a separación de fontes musicais experimentou unha evolución significativa nas últimas décadas, dende métodos clásicos baseados en procesamento de sinais ata enfoques modernos que empregan técnicas de aprendizaxe profundo. Os primeiros enfoques baseábanse en técnicas de procesamento de sinais tradicionais. A técnica máis simple foi a subtracción vocal mediante cancelación da canle central, que aproveitaba o feito de que nas gravacións estéreo a voz adoita estar centrada. Este método funciona restando a canle dereita da esquerda, eliminando así os compoñentes que están presentes en ambas canles por igual. Non obstante, esta técnica presenta limitacións importantes, xa que tamén elimina outros instrumentos centralizados como o bombo ou o baixo, e non funciona en gravacións mono ou cando a voz non está perfectamente centrada. Por iso emerxeron ferramentas para descompoñer espectrogramas en compoñentes aditivos:

- Subtracción vocal por cancelación de canle central: baseada na eliminación de compoñentes comúns nas gravacións estéreo.
- A xa mencionada *STFT*: empregada para analizar a evolución das frecuencias no tempo.
- Factorización de Matrices Non-Negativas (*NMF*): permite descompoñer espectrogramas en compoñentes que representan diferentes fontes.

Figura 2.2: Estrutura da U-Net en *Demucs*

A chegada das técnicas de aprendizaxe automático revolucionou o campo da separación de fontes musicais. Os primeiros enfoques baseados en *redes neuronais* empregaban arquitecturas como as Máquinas de Restrición de Boltzmann (RBM) e posteriormente *redes neuronais* profundas tradicionais. Estes métodos melloraron a separación respecto ás técnicas clásicas, sobre todo en escenarios onde moitos instrumentos ocupan rangos de frecuencia semellantes. O **estado da arte** actual está dominado por arquitecturas de aprendizaxe profundo deseñadas para a separación de fontes. Modelos como *Open-Unmix*, *Spleeter* desenvolvido por Deezer, e máis recentemente *Demucs* de Facebook AI Research, usan arquitecturas como redes convolucionais e redes recorrentes.

*Demucs*, o modelo empregado neste proxecto, é unha das aproximacións máis avanzadas. Baseado nunha arquitectura U-Net (figura 2.2) con conexións residuais e atención temporal, *Demucs* procesa directamente formas de onda no dominio temporal, evitando as limitacións das representacións tempo-frecuencia. O modelo foi adestrado en grandes *dataset* de música, permitíndolle aprender representacións complexas das fontes musicais [4].

## 2.3 Recoñecemento e aliñamento da fala

Un paso fundamental para xerar vídeos de karaoke de forma automática é transcribir o canto (a voz) e aliñar a letra co audio. Nesta sección tratarase ambas partes en detalle.

### 2.3.1 Recoñecemento automático da fala

Aquí entran en xogo técnicas de Automatic Speech Recognition (ASR) adaptadas ao caso do canto. A transcripción de cancións presenta retos adicionais respecto do recoñecemento de fala convencional: o vocalista pode xogar co ritmo, afinación e expresión, e a pronunciación ás

veces distórcese intencionadamente pola melodía. O ASR é a tecnoloxía que permite converter sinais de audio que conteñen voz humana en texto escrito. No contexto deste proxecto, o ASR é fundamental para extraer as letras das cancións a partir do audio separado, permitindo así a xeración de subtítulos sincronizados sen intervención manual.

O proceso de recoñecemento da fala pode modelarse matematicamente como un problema de optimización onde, dado un sinal de audio  $X = \{x_1, x_2, \dots, x_T\}$ , o obxectivo é atopar a secuencia de palabras  $W = \{w_1, w_2, \dots, w_N\}$  máis probable:

$$\hat{W} = \arg \max_W P(W|X) \quad (2.2)$$

Os modelos modernos como WhisperX, empregado neste proxecto, seguen unha arquitectura *end-to-end* baseada en *Transformers* que aprende directamente a mapear audio a texto sen necesidade de compoñentes separados. WhisperX usa un codificador que procesa o espectrograma de audio mediante capas de atención, xerando representacións contextuais do contido acústico. O decodificador, tamén baseado en atención, xera secuencialmente os *tokens* de texto correspondentes. [5]

Referente ao **estado a arte**, os primeiros sistemas baseábanse en modelos de Markov ocultos (HMM) combinados con mixturas de gaussianas (GMM) para modelar as características acústicas, requirindo coñecemento experto para o deseño de coeficientes MCFF [6]. A chegada das redes neuronais profundas introduciu os modelos híbridos DNN-HMM que melloraron a modelaxe acústica. Despois, as arquitecturas de redes recorrentes (RNN) e LSTM permitiron capturar dependencias temporais de longo prazo na fala, e a introdución da técnica CTC facilitou o adestramento *end-to-end* sen necesidade de aliñamento previo [7].

Os modelos baseados en atención e *Transformers* marcaron o seguinte salto. Arquitecturas como *Listen, Attend and Spell* demostraron a viabilidade dos enfoques puramente neuronais, eliminando a necesidade de coñecemento fonético experto. Así xorden modelos como Wav2Vec 2.0. OpenAI Whisper [8] representa o estado da arte actual en ASR. Adestrado en 680.000 horas de audio supervisado en varios idiomas, Whisper demostrou capacidades impresionantes.

### 2.3.2 Aliñamento temporal

O *forced alignment* é o proceso de determinar os instantes temporais nos que cada palabra ou fonema aparece nunha gravación de audio. No contexto deste proxecto, esta técnica serve para sincronizar a letra proporcionada manualmente coa pista vocal. O aliñamento temporal baséase na aplicación de algoritmos de programación dinámica (concretamente o algoritmo DTW) [9] e modelos de Markov ocultos (HMM) para determinar a correspondencia temporal entre os elementos textuais (fonemas, palabras ou frases) e as súas realizacións acústicas no

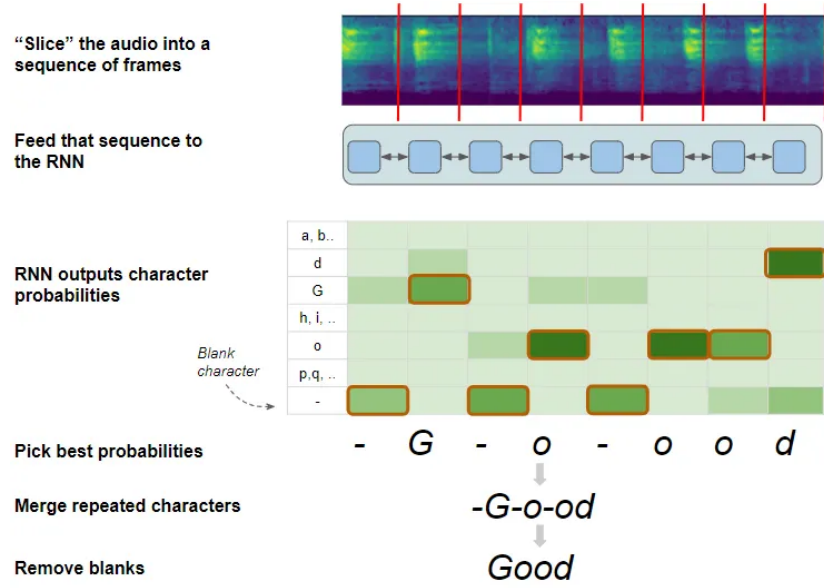


Figura 2.3: Algoritmo de decodificación CTC

sinale de audio [10].

O problema do aliñamento temporal pode formularse do seguinte xeito: dado un sinal de audio  $X = \{x_1, x_2, \dots, x_T\}$  de lonxitude  $T$  e unha secuencia de palabras  $W = \{w_1, w_2, \dots, w_N\}$ , o obxectivo do aliñamento temporal consiste en atopar a función de mapeado óptima:

$$f : W \rightarrow [t_{inicio}, t_{fin}] \subset [1, T] \quad (2.3)$$

onde cada palabra  $w_i$  se asocia cun intervalo temporal  $[t_{inicio}^{(i)}, t_{fin}^{(i)}]$  que maximiza a probabilidade conxunta:

$$P(X, W, A) = \prod_{i=1}^N P(x_{t_{inicio}^{(i)}}^{t_{fin}^{(i)}} | w_i) \quad (2.4)$$

sendo  $A$  o aliñamento temporal e  $x_{t_a}^{t_b}$  o audio comprendido entre os instantes  $t_a$  e  $t_b$ .

O proceso de aliñamento temporal utiliza o algoritmo de Viterbi [11] para atopar a secuencia de estados máis probable nun HMM. Para cada palabra  $w_i$ , defínese un modelo acústico que representa a súa distribución probabilística:

$$\delta_t(j) = \max_{1 \leq k \leq N} [\delta_{t-1}(k) \cdot a_{kj}] \cdot b_j(O_t) \quad (2.5)$$

onde:

- $\delta_t(j)$  é a probabilidade máxima de observar a secuencia de características ata o tempo  $t$  e estar no estado  $j$

- $a_{kj}$  é a probabilidade de transición do estado  $k$  ao estado  $j$
- $b_j(O_t)$  é a probabilidade de emisión do vector de características  $O_t$  no estado  $j$

Este enfoque é especialmente efectivo para o aliñamento de letras manuais, xa que permite compensar variacións no tempo de execución entre a interpretación real e o texto proporcionado. WhisperX mellora este proceso utilizando as activacións internas do modelo Whisper para guiar o aliñamento.

O proceso específico en WhisperX inclúe a segmentación baseada en detección de actividade vocal (VAD) que identifica segmentos de fala activa, seguido do aliñamento palabra por palabra dentro de cada segmento. Isto permite xestionar pausas naturais, respiracións e outros elementos presentes nas cancións.

Referente ao **estado da arte**, o desenvolvemento das técnicas de aliñamento temporal evolucionou xunto cos avances en recoñecemento da fala. Os primeiros sistemas baseábanse en (HMM) con estados correspondentes a unidades fonéticas, requirindo dicionarios de pronunciación manuais e coñecemento lingüístico experto para cada idioma. Despois, a introdución de técnicas baseadas en *redes neuronais* permitiu aprender outros aliñamentos. Os modelos híbridos DNN-HMM melloraron a precisión mediante estimacións de probabilidades de emisión, mentres que técnicas como CTC eliminaron a necesidade de aliñamento previo durante o adestramento.

Montreal Forced Alignment (MFA) é unha das ferramentas máis utilizadas na actualidade. Baseada en modelos Kaldi con arquitecturas neuronais modernas, MFA ofrece modelos preentrenados para varios idiomas e permite aliñamento a nivel de fonema con precisión. Na táboa 2.4 podemos obter a comparación entre varios sistemas de *forced alignment*.

Os sistemas recentes aproveitan modelos de linguaxe preentrenados e técnicas de aprendizaxe autosupervisado. Ferramentas como *Wav2Vec2-based alignment* usan representacións aprendidas de grandes *corpus* de audio para mellorar o aliñamento sen requirir transcripcións manuais durante o adestramento. [12]

## 2.4 Renderizado e contido multimedia

Isto consiste na integración sincronizada de elementos audiovisuais. No contexto deste proxecto, o renderizado céntrase na creación de *clips* de texto dinámicos que se superpoñen ao vídeo orixinal. O renderizado multimedia debe resolver varios desafíos: a creación de subtítulos animados que evolucionen coa música, a integración entre o vídeo de fondo e os elementos textuais superpostos, a xestión da memoria durante o procesamento de vídeos de alta calidade, e a optimización do rendemento para obter tempos de procesamento razoables.

A composición de vídeo multicapa fundaméntase na idea de combinar elementos visuais independentes (*clips*) nunha soa secuencia. Isto pode representarse como unha función de

| Name                                    | Algorithm                    | Supported Language(s) | Interface     |
|-----------------------------------------|------------------------------|-----------------------|---------------|
| <a href="#">aeneas</a>                  | DTW                          | 30+                   | CLI, LIB, Web |
| <a href="#">CMU Sphinx</a>              | HMM (own), RNN               | 11                    | CLI, LIB      |
| <a href="#">DARLA</a>                   | HMM (HTK)                    | English               | Web           |
| <a href="#">FAVE-align</a>              | HMM (HTK)                    | English               | CLI, (Web)    |
| <a href="#">Gentle</a>                  | HMM (Kaldi)                  | English               | CLI, Web      |
| <a href="#">Julius</a>                  | HMM (own)                    | English, Japanese     | CLI, LIB      |
| <a href="#">Kaldi</a>                   | HMM (own), DNN, RNN          | English               | CLI, LIB      |
| <a href="#">kaldi-dnn-ali-gop</a>       | HMM(Kaldi), DNN(Kaldi nnet3) | English               | CLI, LIB      |
| <a href="#">LaBB-CAT</a>                | HMM (HTK)                    | English               | Web           |
| <a href="#">MAUS</a>                    | HMM (HTK)                    | 21                    | CLI, Web      |
| <a href="#">Montreal Forced Aligner</a> | HMM (Kaldi)                  | English               | CLI           |

Figura 2.4: Ferramentas de *forced alignment*

composición [13]:

$$V_{final}(t) = f(V_{fondo}(t), T_1(t), T_2(t), \dots, T_n(t), \alpha_1(t), \alpha_2(t), \dots, \alpha_n(t)) \quad (2.6)$$

onde  $V_{fondo}(t)$  representa o vídeo de fondo no instante  $t$ ,  $T_i(t)$  son os clips de texto ou subtítulos, e  $\alpha_i(t)$  son as funcións de transparencia correspondentes que determinan como se superpoñen os elementos.

A operación de composición máis común é o **alpha blending**, que combina dous elementos visuais baseándose nunha canle de transparencia:

$$C_{final} = \alpha \cdot C_{foreground} + (1 - \alpha) \cdot C_{background} \quad (2.7)$$

onde  $C$  son os valores de cor dos píxeles e  $\alpha \in [0, 1]$  controla o grao de transparencia.

O renderizado de texto para karaoke require técnicas que van máis alá da superposición de texto estático. O efecto de resaltado progresivo faise mediante funcións temporais que modifican as propiedades visuais do texto (cor, tamaño, efectos) en función do progreso temporal:

$$Color_{pixel}(t) = \begin{cases} Color_{resaltado} & \text{se } t \geq t_{inicio\_palabra} \\ Color_{normal} & \text{se } t < t_{inicio\_palabra} \end{cases} \quad (2.8)$$



| Framework                  | Linguaxe   | Nivel | Tempo Real | Program. |
|----------------------------|------------|-------|------------|----------|
| <i>FFmpeg</i>              | C/CLI      | Medio | Si         | Baixa    |
| <i>OpenCV</i> <sup>1</sup> | C++/Python | Medio | Si         | Alta     |
| <i>GStreamer</i>           | C/CLI      | Baixo | Si         | Media    |
| <i>MoviePy</i>             | Python     | Alto  | Non        | Alta     |
| <i>MLT</i>                 | C++        | Medio | Si         | Alta     |

Táboa 2.2: Comparación de *frameworks* para renderizado multimedia.

A sincronización temporal en sistemas multimedia require o manexo de liñas temporais que deben manterse aliñadas durante todo o proceso de renderizado. O sistema debe garantir que: o audio principal (pista instrumental + vocal atenuada) manteña a sincronía coa pista visual e que cada *clip* de texto apareza e desapareza nos instantes exactos determinados polo aliñamento temporal. Isto require o uso de técnicas de sincronización baseadas en *timestamps* que permitan coordinar elementos con diferentes duracións.

Ademáis, o renderizado de vídeo é computacionalmente intensivo, especialmente para resolucións altas e duracións longas. E no contexto do karaoke, os usuarios esperan respostas rápidas ou inmediatas, polo que as principais estratexias de optimización inclúen:

- **Renderizado *lazy*:** Os clips só se procesan cando realmente son necesarios, evitando cargar en memoria elementos que non están activos nun momento dado.
- **Cacheo intelixente:** Elementos que non cambian (como o vídeo de fondo procesado) pódense cachear para evitar reprocesamento.
- **Procesamento por lotes:** Moitos clips de texto poden procesarse conxuntamente para aproveitar mellor os recursos de GPU.

En canto ao **estado da arte**, no contexto do renderizado en karaoke experimentouse unha evolución dende os primeiros sistemas baseados en CDG ata as aplicacións que aproveitan técnicas de visión por computador e procesamento en tempo real. Os primeiros sistemas almacenaban información de subtítulos de baixa resolución xunto co audio en discos compactos. Estes sistemas limitábanse a 300x216 píxeles con 16 *cores*, suficiente para texto básico pero sen capacidades de efectos avanzados. Despois apareceron formatos como KAR (MIDI karaoke) [14] e posteriormente MP3+G, que combinaban audio comprimido con información gráfica. Estes formatos permitían maior flexibilidade pero seguían requirindo creación manual de subtítulos.

<sup>1</sup> OpenCV non é un *framework* multimedia estrito, senón un *framework* de visión por computador con capacidades de procesado de vídeo.

O salto máis grande produciuse coa emerxencia de bibliotecas de procesamento multimedia como FFmpeg [15], OpenCV [16] e posteriormente *frameworks* de alto nivel como MoviePy [17]. Estas ferramentas permitiron acceso a técnicas avanzadas de composición visual, efectos como transicións suaves, renderizado de texto con *antialiasing*, etc. MoviePy, desenvolvido por Zulko, representa unha das solucións máis eficientes. Como é unha das ferramentas elixidas para a realización do proxecto, comentarase máis sobre ela na sección 2.6.2. Para unha visión resumida, a táboa 2.2 presenta unha comparación das principais características destes frameworks no contexto de renderizado automático de karaoke.

As técnicas modernas usan modelos de aprendizaxe profundo para mellorar aspectos como a *frame interpolation* e a xeración automática de efectos visuais. Modelos como RIFE permiten crear transicións suaves entre *frames* [18]. A tendencia actual diríxese cara este tipo de sistemas.

## 2.5 Estado da arte en sistemas de karaoke

O panorama actual dos sistemas de karaoke presenta unha grande diversidade de enfoques, dende aplicacións comerciais que priorizan a experiencia de usuario ata sistemas de investigación que explotan técnicas avanzadas de intelixencia artificial. Esta análise do estado da arte céntrase en clasificar e comparar as diferentes aproximacións existentes.

### 2.5.1 Sistemas comerciais

Poden clasificarse en tres categorías principais: aplicacións móbiles, plataformas web e software de escritorio especializado.

- **Smule** [19] é unha das aplicacións móbiles máis exitosas. Permite colaboracións en tempo real entre usuarios de diferentes localizacións xeográficas. O sistema usa técnicas de cancelación de eco e procesamento de audio en tempo real, pero a súa principal limitación é que se basea nunha biblioteca de pistas instrumentais creadas manualmente.
- **Youka** [20] é unha das mellores solucións. Está asistida por IA e ten funcionalidades como control de tonalidade e tempo. Require subscricións de pago pero o principal problema é a limitación en idiomas minoritarios.
- **Singa** [21] incorpora recoñecemento de voz para avaliar a precisión vocal do usuario e algoritmos de recomendación baseados en preferencias musicais. Non obstante, comparte coa primeira a limitación de depender dunha biblioteca de contido previo.

| Sistema                 | Automática | Idiomas  | Calidade Audio | Custe        |
|-------------------------|------------|----------|----------------|--------------|
| <i>Smule</i>            | Non        | 10+      | Media          | Freemium     |
| <i>Youka</i>            | Si         | Variable | Alta           | Subscription |
| <i>Singa</i>            | Non        | 8+       | Alta           | Freemium     |
| <i>Karaoke Mugen</i>    | Parcial    | Variable | Variable       | Gratuíto     |
| <i>UltraStar Deluxe</i> | Non        | Variable | Variable       | Gratuíto     |
| <i>Este proxecto</i>    | Si         | Variable | Alta           | Gratuíto     |

Táboa 2.3: Comparación de sistemas de karaoke existentes

### 2.5.2 Proxectos de código aberto e investigación

O ecosistema *open source* ten enfoques máis experimentais, destacando proxectos que usan técnicas de aprendizaxe profundo e procesamento automático de audio. Entre eles pódense destacar **Karaoke Mugen**, que permite aos usuarios crear e compartir contido de karaoke. Admite formatos como MP4 con subtítulos incorporados e ofrece xestión de bibliotecas musicais. Malia estes beneficios, o proceso de creación de contido segue sendo máis ben manual [22]. Despois está **UltraStar Deluxe**, que é unha evolución de código aberto do popular xogo SingStar, incorporando recoñecemento de voz e avaliación de rendemento vocal. O sistema permite a importación de contido creado pola comunidade, pero a xeración de novos karaokes require ferramentas externas e coñecementos especializados [23].

A táboa 2.3 presenta unha comparación sistemática das principais características dos sistemas analizados, establecendo o marco de referencia.

### 2.5.3 Limitacións dos enfoques existentes

Grazas ao análise dos sistemas actuais pódense ver varias limitacións que xustifican o desenvolvemento de novas aproximacións:

- Dependencia de contido: A maioría dos sistemas comerciais dependen de bibliotecas de contido creadas manualmente, limitando a diversidade musical e requirindo investimentos en licenciamiento e produción.
- Soporte lingüístico limitado: Malia que algúns sistemas admiten moitos idiomas, a calidade varía considerablemente, especialmente para idiomas minoritarios. Os modelos de recoñecemento de voz adoitan estar optimizados para idiomas maioritarios.
- Separación entre creación e reprodución: Os sistemas existentes xeralmente separan o proceso de creación de karaoke (que require coñecementos técnicos) da experiencia de usuario, limitando a accesibilidade.

## 2.6 Ferramentas e tecnoloxías

Esta sección presenta unha descrición das tecnoloxías, bibliotecas e ferramentas empregadas no desenvolvemento do sistema. A selección destas tecnoloxías baseouse en criterios de rendemento e compatibilidade, ademais da importancia do soporte da comunidade.

### 2.6.1 Librerías de intelixencia artificial

#### Demucs

**Demucs**, proposto por Alexandre Défossez e col. introduciu varias innovacións importantes na separación de música. Como xa se comentou na sección 2.2, en lugar de traballar no dominio da frecuencia, Esta ferramenta opera no dominio do tempo (onda sonora), usando unha arquitectura tipo U-Net convolucional, complementada con capas **LSTM** bidireccionais para capturar dependencias temporais a longo prazo. **Demucs** ten un **encoder** que descompón a onda en representacións de menor dimensión (parecido a extraer características), e un **decoder** que reconstrúe cada fonte a partir desa representación. As conexións entre as capas do **encoder** e **decoder** permiten preservar detalles de alta frecuencia do sinal que normalmente se perderían. Ademais o uso de **LSTM** dentro da arquitectura axuda a que o modelo poida separar fontes tendo en conta o contexto musical global [24]. O esquema do funcionamento pode verse na figura C.1 do apéndice C.

Os resultados reportados polo autor amosan que **Demucs** supera os métodos anteriores en varios conxuntos de datos. En concreto no famoso **dataset** MusDB18, **Demucs** conseguiu unha boa mellora no indicador **SDR** medio, obtendo 6.3 dB de media en catro fontes (baixo, voz, batería, outros) fronte a 5 dB dos mellores modelos previos. Ademais, nunha avaliación con oíntes humanos, o audio separado por **Demucs** foi percibido como máis natural e con menos artefactos en comparación con métodos espectrais, aínda que se detectaba lixeira contaminación (*bleeding*) da voz noutros instrumentos e viceversa. A arquitectura foi posteriormente mellorada introducindo un enfoque híbrido tempo-frecuencia (*Hybrid Demucs*), que combinaba convolucións en onda e en espectrograma, gañando o *Music Demixing Challenge* 2021 e adaptándose ao *Sound Demixing Challenge* 2023 organizado por Sony [25].

#### WhisperX

WhisperX é unha extensión do modelo Whisper de OpenAI que mellora as capacidades de aliñamento temporal para aplicacións que requiren sincronización precisa palabra-audio. Emplea técnicas de segmentación baseadas en actividade vocal (**VAD**) e aliñamento forzado para conseguir precisión a nivel de palabra.

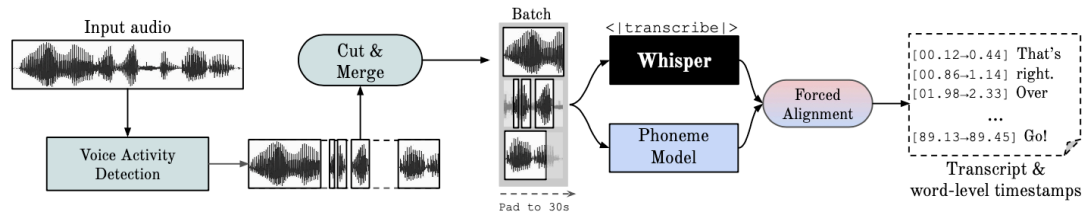


Figura 2.5: Diagrama de funcionamento de WhisperX

Como xa se comentou tamén na sección 2.3, o modelo combina as capacidades multi-língües robustas de Whisper (adestrado en 680.000 horas de audio en 99 idiomas) coa precisión temporal necesaria para aplicacións como o karaoke. WhisperX [5] utiliza unha aproximación en tres etapas (figura 2.5): primeiro realiza a transcripción mediante Whisper, despois aplica segmentación VAD para identificar rexións de fala activa, e finalmente emprega técnicas de aliñamento baseadas en DTW para sincronizar cada palabra co audio correspondente.

No proxecto, WhisperX utilízase tanto para o modo automático (non para transcribir, só aliñar) como para o aliñamento forzado cando se proporcionan letras manuais. Esta flexibilidade permite combatir os casos nos que transcripción automática é imprecisa.

### pyannote.audio

Pyannote.audio é unha biblioteca de código aberto especializada en análise de audio que inclúe detección de actividade vocal, segmentación de falantes e diarización. No contexto deste proxecto, empregase especificamente para a funcionalidade de diarización de falantes (*speaker diarization*). A diarización de falantes permite identificar e separar diferentes voces nunha gravación, respondendo á pregunta "quen fala cando". Esta funcionalidade é especialmente útil en cancións con múltiples cantantes ou en gravacións en vivo onde poden aparecer voces de apoio ou intervencións do público.

Pyannote.audio utiliza modelos de aprendizaxe profundo preentrenados que requiren autenticación mediante tokens de *Hugging Face* debido ás restricións de licenciamiento dos *data-set* de adestramento. O proxecto integra esta funcionalidade como unha opción configurable que mellora a precisión da transcripción.

### 2.6.2 Librerías de Python

#### MoviePy

MoviePy [17] constitúe o núcleo do sistema de renderizado multimedia do proxecto. Esta biblioteca de Python permite a manipulación programática de vídeos mediante unha API baseada en *clips*. Esta arquitectura permítelle ao proxecto crear elementos visuais indepen-

dentes (texto, vídeo de fondo, efectos) que despois se combinan nunha composición final. Cada clip de texto representa unha frase ou palabra con propiedades temporais específicas (inicio, duración) e efectos visuais (resaltado progresivo, posicionamento).

MoviePy xestiona automaticamente aspectos como sincronización audio-vídeo, optimización de memoria durante o renderizado e compatibilidade con diferentes códecs de saída. A biblioteca permite configurar parámetros de calidade como resolución, taxa de fotogramas e compresión para equilibrar calidade e tamaño de arquivo.

### LibROSA

LibROSA [26] é unha biblioteca especializada en análise e procesamento de sinais de audio musicais. No proxecto úsase principalmente para operacións de análise espectral, extracción de características acústicas e manipulación de representacións tempo-frecuencia. A biblioteca proporciona implementacións de transformadas como *STFT*, *mel-spectrograms* e *MCFF*. Tamén inclúe utilidades para manipulación temporal como cambio de velocidade, *pitch shifting* e normalización de audio.

### NumPy

NumPy [27] proporciona a infraestrutura matemática para todo o procesamento do proxecto. Como dependencia básica de practicamente todas as bibliotecas de procesamento científico en Python, NumPy ofrece arrays multidimensionais eficientes e operacións matemáticas optimizadas. No contexto do proxecto, NumPy úsase para manipulación de arrays de audio, operacións de álgebra lineal para o procesamento de sinais e cálculos numéricos en xeral. A súa integración coas outras bibliotecas foi un desafío complexo.

### Pyphen

Pyphen [28] é unha biblioteca para separación silábica que soporta múltiples idiomas. No proxecto utilízase para aplicar resaltado progresivo a sílabas en lugar de palabras completas. Facendo a proba con resaltado silábico e resaltado a nivel de palabra, o resaltado silábico é infinitamente máis cómodo para o lector. Pyphen utiliza dicionarios de patróns de separación específicos para cada idioma.

### PIL (Python Imaging Library)

PIL, na súa implementación moderna Pillow [29], utilízase para o procesamento de imaxes dentro do *pipeline* de renderizado. Aínda que MoviePy xestiona a maior parte das operacións visuais, PIL úsase para tarefas como carga de imaxes de fondo, manipulación de transparen-

cias, redimensionamento, rotación, aplicación de filtros e optimización de imaxes antes da súa integración nos *clips* de vídeo.

### SQLite

SQLite [30] proporciona un sistema de almacenamento persistente. A súa natureza facilita o despregamento sen requirir configuración de servidor de base de datos, mentres que ofrece capacidades SQL completas para consultas, busca e xestión do historial de procesamento.

## 2.6.3 Desenvolvemento web

### Flask

Flask [31] constitúe o *framework* web que proporciona a interface de usuario do sistema. Como *microframework* de Python, Flask crea aplicacións web sen unha estrutura ríxida, e permite personalizar cada aspecto da aplicación segundo as necesidades do proxecto. Cada funcionalidade da interface, das que se falarán nos capítulos 4 e 5, impléntanse como rutas específicas, e xestionanse aspectos como validación de entrada, manexo de erros, xestión de arquivos temporais e envío de resultados ao usuario.

### Bootstrap

Bootstrap [32] proporciona o *framework* CSS que define a aparencia visual da interface web. Garante que a aplicación sexa responsiva e funcione correctamente en diferentes dispositivos e tamaños de pantalla. Tamén facilita a creación de formularios, botóns, indicadores de progreso e outros elementos. Bootstrap inclúe compoñentes JavaScript para funcionalidades interactivas como *tooltips*.

### JavaScript

JavaScript [33] é utilizado para mellorar a experiencia de usuario mediante funcionalidades interactivas. Impléntanse *scripts* para a validación de formularios, *feedback* visual durante o procesamento de arquivos, e a xestión de estados da aplicación. As funcionalidades JavaScript inclúen validación de URL de YouTube, indicadores de progreso durante o procesamento, manexo de erros asíncronos, confirmacións de acción e navegación entre diferentes modos da aplicación, etc.

### 2.6.4 Ferramentas de desenvolvemento

#### Docker

Docker permite illar as dependencias conflitivas entre diferentes compoñentes do sistema. O proxecto utiliza unha arquitectura multi-contedor que aparta as funcionalidades de separación de fontes ([Demucs](#)) e procesamento de fala (WhisperX) en contedores independentes.

Esta separación resolve problemas de compatibilidade entre bibliotecas que requiren versións diferentes de dependencias como PyTorch, [CUDA](#) e diversas bibliotecas de procesamento de audio. Cada contedor ten o seu propio entorno completamente configurado, facilitando o despregamento en diferentes sistemas. Tamén se configuran volumes compartidos para intercambio de datos, mapeamento de portos para comunicación entre servizos e configuración de [GPU](#) para aceleración por hardware cando está dispoñible [34].

#### yt-dlp

Yt-dlp [35] é unha biblioteca de Python para a descarga de contido multimedia dende plataformas como YouTube. Utilízase no proxecto para descargar automaticamente vídeos mediante [URL](#), permitindo aos usuarios procesar contido sen necesidade de descargalo manualmente. A biblioteca ofrece opcións avanzadas de configuración para seleccionar a calidade de vídeo, os formatos de saída e os metadatos asociados. No proxecto configúrase de xeito flexible para garantir a compatibilidade co resto do *pipeline*.

Yt-dlp tamén xestiona aspectos como extracción de metadatos (título, duración, autor), manexo de listas de reprodución e adaptación a cambios nas [API](#) das plataformas de vídeo.

#### FFmpeg

FFmpeg [15] é un software que actúa como o motor de procesamento multimedia a baixo nivel que soporta moitas das operacións de audio e vídeo do proxecto. Malia que bibliotecas como MoviePy proporcionan interfaces de alto nivel, FFmpeg realiza as operacións de codificación, decodificación e transformación real.

No contexto do proxecto, FFmpeg utilízase para a conversión entre formatos de audio, a normalización de vídeos a resolucións estándar, a extracción de pistas de audio dende vídeos e o renderizado final de vídeos con múltiples pistas de audio e subtítulos. A configuración de FFmpeg no proxecto inclúe soporte para *códecs* modernos como H.264 para vídeo e [AAC](#) para audio, así como optimizacións para aceleración *hardware* cando está dispoñible.



## Ngrok

Ngrok [36] é unha plataforma que permite crear túneles seguros para expor a aplicación local á Internet sen configuración da rede. Esta funcionalidade permite compartir a aplicación publicamente para demostracións ou para terceiros, creando [URLs](#) temporais que redirixen ao servidor local.

### 2.6.5 Documentación

#### LaTeX

LaTeX [37] emprégase para a redacción da memoria do proxecto, proporcionando capacidades de formateo, xestión automática de referencias, numeración de seccións, xeración de índices e ecuacións matemáticas.

#### Editores e entornos de desenvolvemento

Para código Python utilízase Visual Studio Code [38] con extensións específicas para desenvolvemento Python, Docker e xestión de git. Para a documentación LaTeX emprégase Overleaf [39], que proporciona facilidades para compilar esta linguaxe (compilación automática, previsualización en tempo real, colaboración multiusuario, etc).

#### Diagramas e visualización

A documentación técnica inclúe diagramas de arquitectura e fluxos de proceso creados mediante a ferramenta **Mermaid** [40]. Mermaid é unha ferramenta que converte texto e código en diagramas e visualizacións orientado a desenvolvedores. Mermaid axuda a documentar o sistema e permite ter os diagramas actualizados.

### 2.6.6 Outras ferramentas

Ademais das bibliotecas principais descritas anteriormente, o proxecto emprega unha serie de ferramentas auxiliares que proporcionan funcionalidades específicas:

- **PyTorch**: *Framework* de aprendizaxe profundo que forma a base de funcionamento tanto para [Demucs](#) como para WhisperX. Contén as bases matemáticas e o soporte para a aceleración [GPU](#) mediante [CUDA](#). É necesario para o rendemento dos modelos de [IA](#) empregados no proxecto.
- **Werkzeug**: Biblioteca de utilidades web que forma parte da infraestrutura de Flask. No proxecto emprégase especificamente para funcionalidades de seguridade.

- **ImageMagick:** Ferramenta para procesar imaxes de baixo nivel que MoviePy usa internamente para operacións de renderizado. O proxecto inclúe configuración específica para detectar automaticamente a instalación de ImageMagick en diferentes sistemas operativos.
- **SoundFile:** Biblioteca para lectura e escritura de arquivos de audio en varios formatos. Fai de *backend* para LibROSA e outras bibliotecas de audio, proporcionando compatibilidade con WAV, FLAC e OGG.
- **subprocess:** Permite executar comandos externos do sistema operativo dende Python. Úsase para invocar FFmpeg para operacións de conversión de audio e para executar Demucs mediante liña de comandos cando é necesario.
- **shutil:** Biblioteca para operacións de alto nivel con arquivos e directorios. O proxecto emprégaa para tarefas como copiado de arquivos entre directorios, limpeza de arquivos temporais e xestión de permisos de arquivos.
- **re (expresións regulares):** Módulo para procesamento de texto mediante patróns. Utilízase na normalización das letras proporcionadas polos usuarios, eliminando etiquetas, caracteres especiais e outras elementos que poidan interferir co aliñamento.
- **logging:** Sistema de rexistro de eventos. Permite supervisar e identificar erros durante o procesamento. Especialmente importante para depurar problemas relacionados coa GPU e a compatibilidade de bibliotecas, entre outros.
- **GitHub:** Sistema de control de versións para o desenvolvemento do proxecto. Facilita o seguimento de cambios, as colaboracións e o mantemento de diferentes versións ou ramas do código. Todo o código desenvolto está dispoñible en: <https://github.com/gaelfernandez1/KaraokeProject> [41].
- **Celery:** Celery é o sistema de procesamento asíncrono que permite xestionar tarefas de longa duración sen bloquear a interface de usuario.
- **Redis:** Emprégase como *message broker*, almacenando as tarefas pendentes e os resultados de procesamento. Isto permite a cancelación de tarefas en tempo real, o seguimento de progreso e o procesamento de solicitudes simultáneas [42].

# Planificación e xestión do proxecto

---

NESTE capítulo abóndase aspectos da planificación do proxecto, analizaranse os requisitos do sistema, a metodoloxía seguida, a planificación temporal, así como a xestión dos recursos dispoñibles e a avaliación de riscos e custes. Tamén se describe o seguimento realizado durante o proceso e os casos de uso máis relevantes.

## 3.1 Metodoloxía de desenvolvemento

Para levar a cabo o proxecto, adoptouse unha **metodoloxía iterativa** nun entorno áxil, un dos métodos de desenvolvemento evolutivo. Esta metodoloxía permitiu ir construíndo o sistema por fases e incorporar melloras progresivas a medida que se ían completando os módulos e ían xurdindo ideas. É complicado planificar todo dende o inicio, pero isto aporta vantaxes como realizar un bo deseño a pesar de que o ámbito era previamente descoñecido. Comézase o proceso cunha implementación sinxela de requisitos e vanse mellorando as versións do sistema. Non se tenta iniciar o desenvolvemento despois da especificación completa do sistema, senón que comeza só con parte do produto de software. Neste modelo repetimos certos pasos, como probas de deseño e implementación en cada iteración [43].

As actualizacións constantes das librerías de [IA](#) utilizadas implican que o proxecto non é estático: require revisións frecuentes e cambios en [API](#). Todo encaixa ca metodoloxía iterativa/incremental, que permite flexibilidade para adaptar o plan de proxecto en función dos resultados obtidos e das necesidades temporais. Esta metodoloxía dividiu o traballo en ciclos chamados *sprints* de duración fixa (2 semanas), nos que en cada un se deben completar tarefas específicas. Os *sprints* pódense definir como fases curtas e repetibles que dividen un proxecto en pequenas partes. O equipo planifica un *sprint* á vez e adapta os seguintes segundo o resultado do anterior [44]. Os *sprints* son validados a partir de reunións semanais nas que os integrantes presentan melloras e discuten posibles solucións. A figura [C.8](#) do apéndice [C](#) ilustra o modelo iterativo utilizado, e a figura [3.1](#) un esquema dunha metodoloxía iterativa

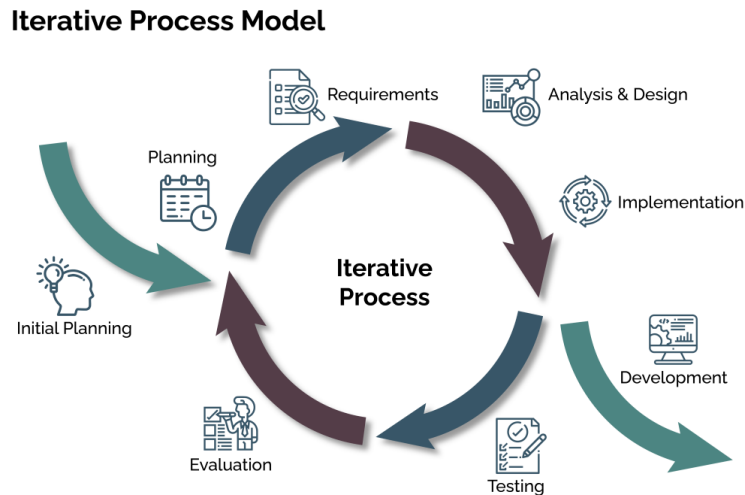


Figura 3.1: Esquema dunha metodoloxía de desenvolvemento iterativa xeral

xeral.

## 3.2 Análise de requisitos

Os requisitos son as características que debe posuír o sistema para cumprir os requirimentos e as expectativas dos clientes ou usuarios. Nesta sección vanse describir os requisitos funcionais e non funcionais que guían o desenvolvemento do proxecto. No caso deste proxecto, os requisitos do sistema son determinados polos mesmos creadores e non son impostos por ningunha entidade externa.

### 3.2.1 Requisitos funcionais

Os requisitos funcionais definen as accións que o sistema debe ser capaz de realizar. Nas reunións periódicas establecidas entre o alumno e o titor establecéronse os seguintes requisitos a alto nivel para o sistema:

- RF-1 Permitir ao usuario introducir unha ligazón de vídeo de YouTube ou subir un ficheiro local en formato [MP4](#).
- RF-2 Extraer o audio do vídeo proporcionado e separar a pista vocal da instrumental mediante técnicas de separación de fontes.
- RF-3 Transcribir automaticamente a voz do cantante a texto, empregando modelos multilingües de recoñecemento automático da fala.
- RF-4 Xerar a sincronización entre a letra e a música a nivel de sílaba.

- RF-5 Xerar un vídeo final de karaoke no que se amose a letra sincronizada cunha animación que destaque as palabras a medida que son cantadas.
- RF-6 O usuario debe poder descargar o resultado.
- RF-7 Permitir empregar letras manuais para optimizar a sincronización cando sexa necesario.
- RF-8 Seleccionar opcionalmente a diferenciación de voces (*speaker diarization*)
- RF-9 Seleccionar o modelo de Whisper desexado na propia web.
- RF-10 Reproducir o vídeo na propia interface web permitindo a configuración de parámetros
- RF-11 Extraer simplemente a pista instrumental e descargala.

### 3.2.2 Requisitos non funcionais

Os requisitos non funcionais establecen criterios de calidade, rendemento e outras propiedades que se desexa que cumpra o sistema:

- RNF-1 O sistema debe ofrecer unha experiencia de usuario sinxela e intuitiva, cunha interface web clara e funcional ofrecendo *feedback para fallos*.
- RNF-2 O tempo de procesado para cancións estándar (3-5 minutos) non debe superar un tempo razoable, mesmo tendo en conta a carga computacional das ferramentas de [IA](#).
- RNF-3 O sistema debe ser compatible con distintos idiomas, incluíndo linguas minoritarias como o galego.
- RNF-4 A solución debe ser modular, de xeito que se poida actualizar ou substituír facilmente calquera dos compoñentes (p.ex., cambiar Whisper por outro sistema de transcripción).
- RNF-5 A aplicación debe ser despregable nun contedor Docker para facilitar a súa instalación, reproducibilidade e portabilidade.

## 3.3 Casos de uso

O sistema está deseñado para abordar varios escenarios de uso. A súa flexibilidade permite adaptarse a diferentes tipos de usuarios e necesidades, propoñendo tanto funcionalidade automatizada completa como control manual para casos específicos que requiren maior precisión. Nas táboas que se mostran no apéndice [B](#), táboas [B.1](#) a [B.10](#), detállanse todos os casos de uso do sistema, e na figura [3.2](#) móstrase o diagrama dos principais casos de uso do sistema.

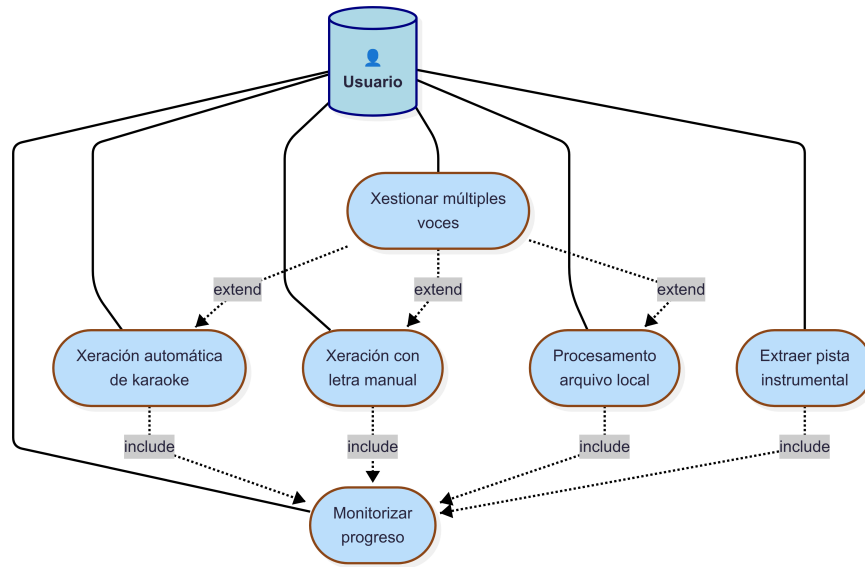


Figura 3.2: Diagrama de casos de uso do sistema

O sistema ten utilidade para diferentes contextos. Nun contexto educativo, serve para crear material didáctico para os estudantes utilizando cancións tradicionais galegas. Nun contexto familiar ou cultural, a utilidade que levou a desenvolver este proxecto, pode servir para organizar eventos de karaoke nos que os participantes poidan escoller libremente a canción desexada, contribuíndo á preservación da música galega.

### 3.4 Planificación e seguimento

Nesta sección descríbense as fases e a división de tarefas do proxecto, resultado de aplicar un desenvolvemento progresivo. Tamén se describe o seguimento do proxecto comentando os obstáculos técnicos que provocaron atrasos.

#### 3.4.1 Planificación do proxecto

A continuación preséntase a división do proxecto nos diferentes *sprints*. Como se comentou na sección 3.1, cada *sprint* está enfocado facer una mellora conforme ao *sprint* anterior e en avanzar dende a investigación inicial ata a implantación final do sistema.

- **Sprint 1 – Documentación:** Neste primeiro *sprint* realizouse unha análise das tecnoloxías dispoñibles para o proxecto, como WhisperX, Demucs, MoviePy ou Flask, así como a súa viabilidade técnica. Fíxose unha análise exhaustiva da problemática e as posibles solucións.

- **Sprint 2 – Integración do pipeline básico e interface web:** Tráballose nun primeiro fluxo funcional que permitise extraer o audio dun vídeo e procesalo. Implementouse a separación de pistas con [Demucs](#) e a transcripción automática do audio coa ferramenta Whisper, obtendo así os primeiros resultados sincronizados e as primeiras transcricións. Desenvolveuse tamén unha aplicación web empregando Flask, que permitise ao usuario subir ligazóns de YouTube ou ficheiros locais. Creouse unha interface base para ir engadindo novas funcionalidades a medida que avanzara o proxecto.
- **Sprint 3 – Procesamento de texto e sincronización con WhisperX:** Fíxose un cambio efectivo da ferramenta Whisper por WhisperX, software que permite xerar transcricións a nivel de palabra e non de frase, funcionalidade moi útil para xerar un resultado progresivo das letras. Ademais implementouse a diarización de falantes.
- **Sprint 4 – Dockerización e despregamento:** A medida que o proxecto adquire dependencias, agrupalas e facelas compatibles entre elas é cada vez máis complicado. Este sprint centrouse na creación dun contorno dockerizado que agrupase todas as partes do sistema. Construíronse os ficheiros Docker e docker-compose e realizouse a integración completa do *pipeline* co servidor web.
- **Sprint 5 – Xeración de vídeo karaoke:** Centrouse na creación do vídeo final de karaoke a partir do audio instrumental e as letras sincronizadas. Utilizouse MoviePy para renderizar o vídeo, aplicando estilos visuais como o subliñado progresivo ou a cor activa nas palabras, mellorando así a experiencia visual.
- **Sprint 6 – Forced Alignment:** Isto pode ser considerada a clave do proxecto. Posto que os modelos de transcricións actuais están lonxe de ser perfectos, integrouse a posibilidade de empregar letras manuais para asegurar que a letra que se mostra en pantalla é perfecta, e a partir de aí elaborar scripts e módulos para mellorar o axuste temporal, buscando un aliñamento máis preciso co audio da canción.
- **Sprint 7 – Validación e melloras:** Probouse o sistema con distintas cancións e idiomas, incluído o galego. Axustáronse tempos e mellorouse o rendemento. Ademais implementáronse funcionalidades adicionais, tales como un reprodutor propio da interface web, unha base de datos para engadir cancións a unha biblioteca e a creación dun túnel Ngrok [36] para permitir o uso da interface por terceiros.
- **Sprint 8 – Documentación e preparación da memoria:** Finalmente, adicouse tempo á redacción da memoria e á creación de diagramas. Tamén se preparou a defensa do traballo e realizouse unha revisión do proxecto para asegurar a súa calidade final.

| Problema                       | Atraso           | Solución Implementada                   |
|--------------------------------|------------------|-----------------------------------------|
| Limitacións WhisperX en galego | 2 semanas        | FallBack (es) + <i>scripts</i>          |
| Conflitos de dependencias      | 2 semanas        | Arquitectura Docker con microservizos   |
| <b>Total</b>                   | <b>4 semanas</b> | <b>Arquitectura robusta e escalable</b> |

Táboa 3.1: Resumo dos atrasos identificados no proxecto

### 3.4.2 Seguimento

Durante o desenvolvemento do presente proxecto identificáronse varios obstáculos técnicos que provocaron atrasos significativos na planificación inicial. A maioría de fases leváronse a cabo no tempo previsto, pero hai dúas fases concretas que tiveron atrasos debido a que o problema era máis grande do estimado. Na figura 3.3 pódese visualizar a parte do diagrama de Gantt afectada, e na sección 3.5 detalláranse os custos a maiores provocados por estes contratempos. A continuación descríbense as principais dificultades atopadas e as solucións implementadas.

#### Limitacións no soporte multiidioma de WhisperX

Un dos atrasos máis significativos do proxecto débese ás limitacións de WhisperX para o idioma galego. Durante a fase de implementación descubriuse que WhisperX non dispón de modelos de aliñamento (*alignment models*) específicos para o galego, o que supuxo un obstáculo importante dado que un dos obxectivos principais do proxecto era crear un sistema de karaoke que funcionara eficazmente neste idioma. Esta limitación provocou un atraso de **2 semanas** no cronograma inicial, xa que foi necesario investigar alternativas para o procesamento de audio en galego e realizar probas con diferentes configuracións.

Para solventar esta limitación decidiuse establecer o modo de aliñamento español (*es*) como *fallback* (ademais de módulos externos para mellorar o funcionamento). Esta solución funciona razoablemente ben, tendo en conta as similitudes lingüísticas entre os dous idiomas, aínda que non é óptimo debido ás diferenzas fonéticas entre eles.

#### Incompatibilidades entre dependencias de software

Durante a fase de integración atopáronse conflitos entre as dependencias das diferentes librerías utilizadas no proxecto. A realidade é que estas ferramentas non están optimizadas para funcionar entre elas, e concretamente púidose atopar:

- **Conflitos entre versións de PyTorch:** *Demucs* requiría de certas versións de PyTorch mentres que as versións máis recentes de WhisperX necesitaban versións anteriores, e non se atopou ningunha versión desta ferramenta que fose compatible con ambas.



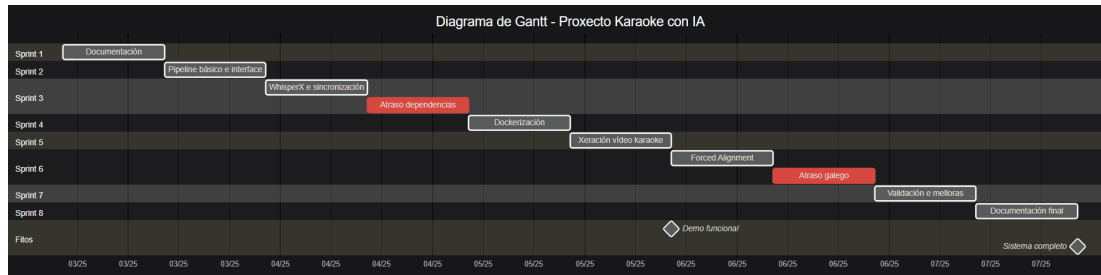


Figura 3.3: Diagrama de Gantt final do proxecto

- **Incompatibilidades CUDA:** Diferentes librarías requirían versións específicas de **CUDA** que entraban en conflito. O resultado final é que o noso sistema está optimizado con **GPU** en menos partes das desexadas.
- **Problemas con librarías de audio:** Conflitos entre librosa, soundfile e as dependencias internas de MoviePy, entre outras.

Este problema técnico causou un forte impacto no desenvolvemento. Concretamente un atraso adicional de **2 semanas** e incluíu: moitas reinstalacións do entorno de desenvolvemento, perda de tempo en *debugging* de erros de dependencias e a necesidade de reescribir partes do código para adaptarse a versións específicas.

A solución adoptada foi a containerización mediante Docker, separando as funcionalidades en varios contedores independentes. Aínda que inicialmente non estaba previsto, resultou ser unha decisión acertada que mellorou a estabilidade do sistema. Na táboa 3.1 pódese observar un resumo dos principais problemas encontrados e as súas correspondentes solucións.

### 3.4.3 Diagrama de Gantt

Para proporcionar unha representación visual máis sinxela da planificación temporal final, creouse un diagrama de Gantt, que se mostra a continuación na figura 3.3. A data de inicio está marcada o 7 de marzo de 2025, e o diagrama mostra tanto a organización das actividades ao longo do tempo como os retrasos nos que o proxecto se viu involucrado.

## 3.5 Análise de riscos e custos

Nesta sección vaise tratar a parte de viabilidade do sistema. En todo proxecto é preciso realizar un análise de prevención ante posibles problemas que se poidan manifestar, e a forma de poder evitalos será anticiparse, identificando os riscos e os seus plans de prevención. Por suposto a estimación de custes tamén é fundamental para a efectividade do proxecto. Analizaranse tanto os custes materiais como os dos recursos utilizados.

| Risco                                                         | Probab. | Impacto |
|---------------------------------------------------------------|---------|---------|
| R1. Incompatibilidades entre dependencias de software         | Alto    | Alto    |
| R2. Calidade deficiente da transcripción automática           | Alto    | Baixo   |
| R3. Rendemento computacional insuficiente sen GPU             | Medio   | Alto    |
| R4. Erros na sincronización temporal entre audio e subtítulos | Medio   | Alto    |
| R5. Fallos na separación de stems con Demucs                  | Baixo   | Alto    |
| R6. Problemas de escalabilidade con múltiples usuarios        | Baixo   | Medio   |

Táboa 3.2: Identificación e análise dos riscos do proxecto.

### 3.5.1 Análise de riscos

Nesta subsección levarase a cabo a identificación e xestión dos riscos aos que o sistema está exposto. Farase unha elaboración de plans de prevención no caso de que algún destes riscos se manifeste realmente. Para isto elaborouse a táboa 3.2, na que se pode ver a lista de posibles riscos contemplados e o seu grao de probabilidade e impacto.

Unha vez establecidas as posibilidades, os riscos teñen cadanseu plan de prevención e a continuación elabórase unha lista nos que se mostran:

- **R1:** Isto mitigaríase buscando un xeito de illar as ferramentas utilizadas e unindo mediante postprocesamento o resultado de cada unha delas, ou ben buscando librerías alternativas que cumplan o mesmo fin.
- **R2:** Este risco ten unha alta probabilidade de facerse efectivo. Para evitalo, vaise implementar un sistema que combina transcripción automática con *forced alignment* mediante a introdución de letras manual, garantindo a precisión da transcripción.
- **R3:** O sistema incluírá detección automática das capacidades de hardware de cada equipo e configurará os mellores parámetros para cada situación e implementaranse *fallbacks* automáticos de GPU a CPU en caso de erros.
- **R4:** Desenvolveranse algoritmos de suavizado e *buffers* de anticipación para mellorar a sincronización. O sistema permitirá axustar todo tipo de parámetros e implementaranse *scripts* de postprocesamento para detectar segmentos incorrectos nos resultados de sincronización.
- **R5:** Implementarase detección de calidade para os resultados da separación e aparecerán mensaxes informativos cando a calidade sexa insuficiente.
- **R6:** A arquitectura con Celery e Redis permite procesamento asíncrono e xestión de colas. Para un entorno académico, considerárase suficiente o rendemento obtido, pero a arquitectura permitirá escalamento engadindo máis *workers*.

### 3.5.2 Recursos utilizados

Esta sección describe os recursos utilizados para levar a cabo o proxecto, os cales poden dividirse en tres categorías: humanos, materiais e de *software*. A continuación móstrase máis en detalle cada un deles:

#### Recursos humanos

Este proxecto realízase cunha persoa (o estudante) que adquire múltiples roles ao longo do desenvolvemento, coa colaboración dun consultor externo:

- **Consultor:** Un profesor da facultade será o encargado da supervisión e asesoramento ao longo do proxecto, mediante reunións semanais que servirán para guiar ao alumno e asegurar a planificación establecida.
- **Xefe de proxecto-Analista-Programador:** O alumno asume cada un destes tres roles e terá que definir os requisitos do sistema en base aos requerimentos, tomar as decisións e realizar a implementación das diferentes partes do sistema.

#### Recursos materiais

- **Ordenador persoal do investigador-programador:** Portátil ASUS VivoBook Pro 15 OLED M6500QC-L1010W coas seguintes especificacións:
  - Procesador: AMD Ryzen 7 5800H with Radeon Graphics (3.20 GHz)
  - RAM instalada: 16,0 GB (15,4 GB usable)
  - Controlador gráfico: NVIDIA GeForce RTX 3050 4GB GDDR6
  - Almacenamento: 512 GB SSD
  - Sistema operativo: Windows 11 Home 64 Bits

#### Recursos de *software*

Para poñer en práctica as diferentes partes do sistema, empregouse unha serie de ferramentas de código aberto, todas as cales están explicadas en detalle na sección 2.6 desta mesma memoria. Na próxima sección 3.5.3 comentarase o análise de custos relacionado con estas ferramentas.

### 3.5.3 Análise de custos

Neste apartado presentarase o desglose final de custos do proxecto. Dada a natureza do traballo, os custes en material *software* son inexistentes xa que o seu desenvolvemento baséase

| Concepto     | horas        | Custo (€/h)   | Custo total (€) |
|--------------|--------------|---------------|-----------------|
| Consultor    | 38           | 50            | 1.900           |
| Xefe         | 50           | 50            | 2.500           |
| Analista     | 90           | 40            | 3.600           |
| Programador  | 210          | 30            | 6.300           |
| Equipamento  | (20 semanas) |               | 65              |
| Software     |              | (open source) | 0               |
| <b>Total</b> | <b>388</b>   |               | <b>14.365</b>   |

Táboa 3.3: Resumo de custos totais do proxecto

no uso de librerías e programas exclusivamente *open source*. Podemos entón afirmar que o custe real neste aspecto son 0€.

Para a realización do traballo non foi necesaria a adquisición de *hardware* adicional. O equipo persoal do estudante foi suficiente para cumprir cos requisitos do proxecto. Como o prezo de compra do equipo foi de 1014,70€, e asumindo unha vida útil de 6 anos, calcúlase que a amortización anual é de 169,12€. Polo tanto, o custo asociado ás 20 semanas de duración do proxecto calcúlase nuns 65€.

Baseándose no diagrama de Gantt da figura 3.3, o proxecto ten unha duración total de 20 semanas. Como se comentou na sección 3.5.2, o estudante asume múltiples roles, mentres que o profesor actúa como consultor. En base a isto podemos determinar os custes de recursos humanos partindo das horas traballadas en cada rol.

Para o consultor estableceuse unha tarifa de 50€ por hora. Considerando reunións semanais dunha hora de duración que totalizan 20 horas de supervisión directa, 10 horas para a corrección da memoria e 8 horas adicionais reservadas para resolver dúbidas específicas e asesoramento, resulta nun total de 38 horas de dedicación. O que supón un custo de 1.900€.

Para o xefe de proxecto, estableceuse unha tarifa de 50€ por hora. O estudante dedica 50 horas a este rol, o que resulta nun custo de 2.500€. Para o analista, a tarifa é de 40€/h nas que o estudante dedica 90 horas, o que resulta nun custo de 3.600€. Para o programador, estableceuse unha tarifa de 30€/h para o que se dedica 210 horas, o que resulta nun custo de 6.300€. O tempo total dos roles do alumno son 17,5 horas semanais distribuídas de luns a venres, equivalentes a 3,5 horas diarias de traballo. Isto son 350 horas, resultando nun custo total, tendo en conta os diferentes roles, de 12.400€.

Para unha visión máis clara do total de custes, realizouse un breve resumo na táboa 3.3.

# Arquitectura do sistema

---

ESTE capítulo presenta o deseño arquitectónico do sistema. Descríbense os compoñentes principais, as súas interaccións e as decisións de deseño que guiaron o desenvolvemento. Este capítulo céntrase no "QUÉ": arquitectura, compoñentes e como se comunican. A arquitectura proposta resolve os retos identificados no capítulo anterior, proporcionando unha solución eficiente.

## 4.1 Visión xeral da arquitectura

A arquitectura do sistema adopta un enfoque de **microservizos containerizados** que permite illar as dependencias conflitivas entre as ferramentas empregadas. Os microservizos representan un cambio con respecto ao desenvolvemento tradicional de aplicacións monolíticas. En lugar de crear unha aplicación grande, os desenvolvedores crean subaplicacións especializadas, cada unha cunha función [45]. Esta decisión arquitectónica foi motivada polos problemas de compatibilidade descritos na sección 3.1, onde se documentaron conflitos entre versións de PyTorch, [CUDA](#) e outras bibliotecas.

O sistema estrutúrase en cinco contedores Docker independentes que se comunican mediante protocolos estándar e volumes compartidos. Esta separación garante que cada compoñente funcione no seu propio entorno, evitando as incompatibilidades que impedían o funcionamento conxunto. Na figura 4.1 pódese observar o diagrama de arquitectura xeral deseñado para o sistema.

### 4.1.1 Principios de deseño adoptados e xustificación

A arquitectura do sistema fundamentase en catro principios fundamentais que guían todas as decisións de implementación. A elección desta arquitectura responde a necesidades que xurdiron durante o desenvolvemento. Poden definirse deste xeito:

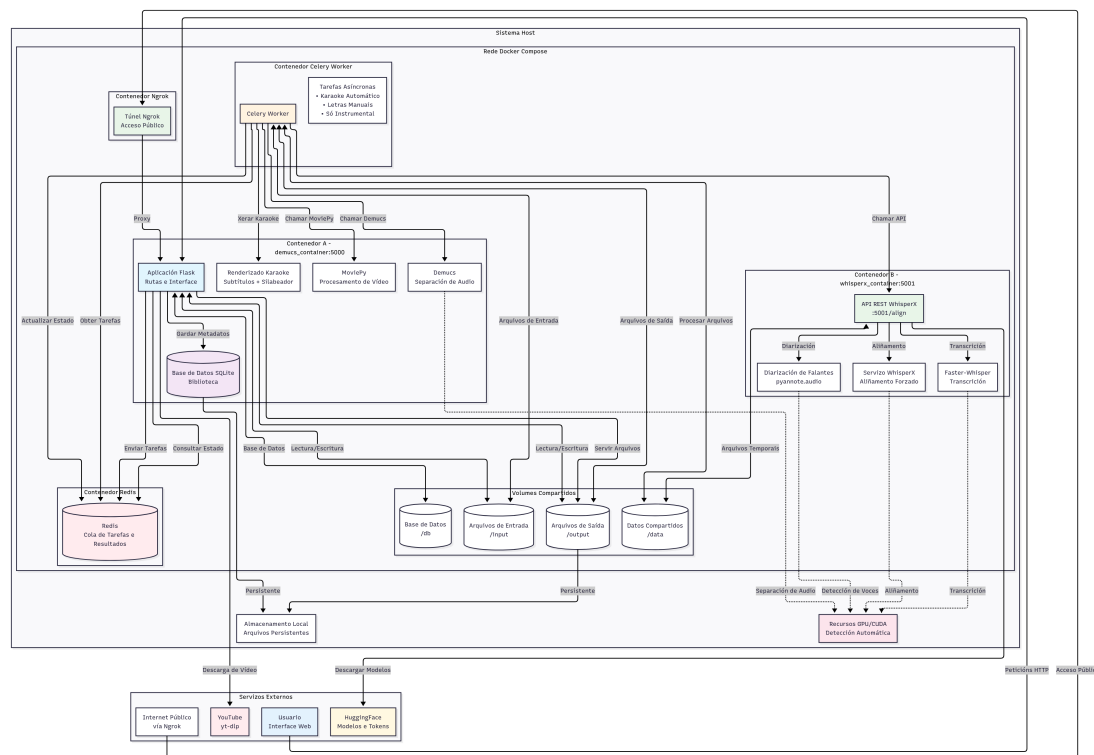


Figura 4.1: Arquitetura xeral do sistema de karaoke automático

1. **Separación de responsabilidades:** Cada contedor ten unha función ben definida. O contedor Flask xestiona a interface de usuario, o contedor **Demucs** realiza a separación de fontes, WhisperX procesa as transcricións e Celery coordina o procesamento asíncrono. Como se documenta na sección 3.5, as ferramentas empregadas requiren versións incompatibles de PyTorch, **CUDA** e outras. A containerización permite que cada ferramenta funcione coa súa versión optimizada.
2. **Desacoplamento mediante interfaces:** Os compoñentes comunícanse a través de protocolos estándar (**HTTP REST**, Redis) e formatos de intercambio ben definidos (**JSON**, **SRT**, **WAV**), permitindo substituír calquera compoñente sen afectar ao resto do sistema.
3. **Tolerancia a fallos e optimización:** O sistema implementa múltiples estratexias de *fallback*, como se viu na sección 2.3, que permiten continuar o funcionamento aínda que algún compoñente presente problemas. Incluso si diferentes operacións usan configuracións *hardware* distintas (**CPU** ou **GPU**), os contedores independentes permíteno.
4. **Escalabilidade horizontal:** A arquitectura baseada en colas de Redis permite ter varios *workers* de procesamento para manexar cargas de traballo maiores. Ademais a separación facilita actualizar ferramentas individuais sen afectar á estabilidade do sis-

| Contedor      | Acceso GPU    | CPU Cores | Memoria | Prioridade |
|---------------|---------------|-----------|---------|------------|
| Flask/Demucs  | Si (completo) | Todos     | Alta    | Alta       |
| Celery Worker | Si (completo) | Todos     | Alta    | Alta       |
| WhisperX      | Si (opcional) | 2-4       | Media   | Media      |
| Redis         | Non           | 1         | Baixa   | Alta       |
| Ngrok         | Non           | 1         | Baixa   | Baixa      |

Táboa 4.1: Distribución de recursos por contedor

tema completo. Isto é especialmente relevante no campo da [IA](#), onde as bibliotecas evolucionan rapidamente.

## 4.2 Arquitectura de contedores

Como se comentou na sección [4.1](#), o sistema está construído sobre unha arquitectura Docker multi-contedor. Nesta sección vaise amosar un esquema sobre dita arquitectura e cómo se comunican as súas partes, ademais de tratar a xestión de recursos *hardware*. Na figura [4.2](#) visualízase un diagrama detallado dos contedores con volumes.

### 4.2.1 Xestión de recursos e hardware

A arquitectura permite configurar os recursos asignados a cada contedor. A táboa [4.1](#) mostra un resumo da distribución de recursos de cada contedor. O sistema inclúe unha lóxica para detectar automaticamente as capacidades GPU de cada equipo (figura [4.3](#)).

Esta estrutura xurdiu de probar o entorno docker nun equipo portátil externo (diferente ao ordenador persoal no que se implementou o sistema dende cero) e producirse un erro debido á diferenza de capacidade *hardware*. Con esta arquitectura calquera persoa pode instalar o entorno en calquera equipo a partir do repositorio de Github. Está implementada no arquivo *gpu\_utils.py*, cuxo código se pode ver no listado [A.1](#) do apéndice [A](#).

### 4.2.2 Redes e comunicación entre contedores

Docker Compose crea automaticamente unha rede interna que permite a comunicación entre contedores usando os seus nomes como *hostnames*. Isto proporciona illamento de rede:

- **Flask ↔ WhisperX:** Comunicación [HTTP REST](#) no porto 5001
- **Flask ↔ Redis:** Conexión [TCP](#) no porto 6379 para colas Celery
- **Celery Worker ↔ Redis:** Mesma configuración que Flask para coordinación

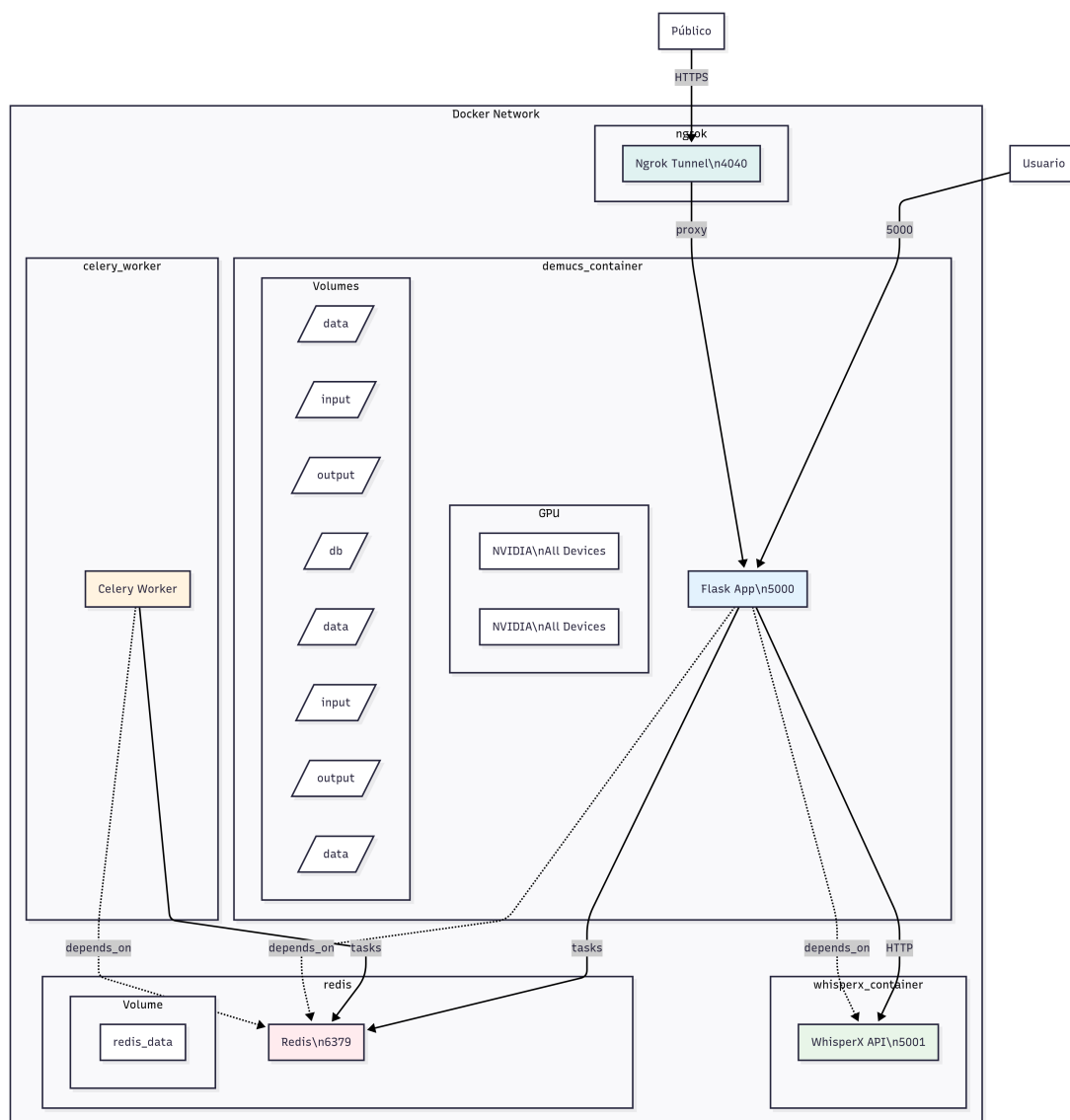


Figura 4.2: Arquitectura detallada de contenedores con volumes compartidos

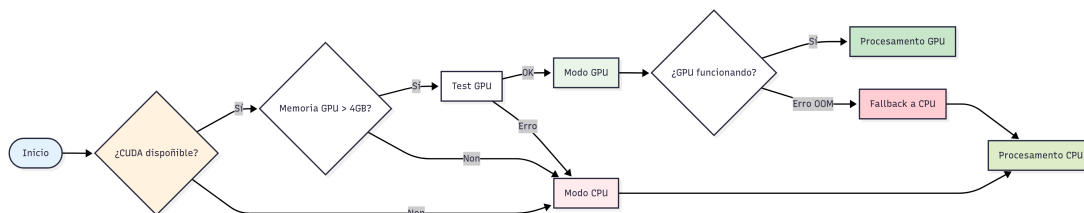


Figura 4.3: Flujo de detección e configuración de hardware



- **Usuario** ↔ **Ngrok**: [HTTPS](#) público que redirixe ao contedor Flask

### 4.3 Compoñentes principais do sistema

O sistema pódese organizar nunha arquitectura de cinco capas que facilita a separación de responsabilidades. Esta arquitectura *layered* [46] permite que cada capa teña unha función específica, facilitando modificacións sen afectar outras partes do sistema.

Nas figuras C.9 e C.10 do apéndice C móstrase o diagrama de clases dos compoñentes do proxecto dividido en dúas partes para mellor visualización: a principal e a de procesamento.

#### 4.3.1 Capa de presentación

A capa de presentación implementa a interface de usuario e a lóxica de controladores web. Está construída sobre Flask, un *microframework* web para Python e aplicacións de tamaño medio como esta. Esta capa xestiona múltiples responsabilidades: validación da entrada do usuario (URLs YouTube, arquivos MP4, letras manuais), coordinación con tarefas asíncronas de procesamento, xestión de estado das tarefas e renderizado da interface web responsiva.

O arquivo *app.py* contén o centro da aplicación web, implementando as rutas que corresponden aos casos de uso dos que se fala na sección 3.3. As rutas principais inclúen procesamento automático (*/generate*), procesamento con letras manuais (*/generate\_manual*) e a extracción de instrumental (*/generate\_instrumental*).

O sistema de *templates* utiliza Jinja2 para xerar contido [HTML](#) dinámico, cun *template* base para a estrutura común e varios específicos para cada funcionalidade. Os *templates* principais inclúen a páxina central con formularios de entrada (*index.html*), a interface para modo con letras manuais (*manual\_lyrics\_form.html*), a monitorización en tempo real do procesamento (*progress.html*), o reprodutor web integrado (*player.html*) e a biblioteca de cancións procesadas (*library.html*).

A capa contén JavaScript que implementa funcionalidades *client-side* [47] como validación de formularios, chequeo do progreso das tarefas, manexo de estados, etc. O sistema de progreso usa [polling](#) cada 2 segundos para actualizar o estado das tarefas sen recargar a páxina.

#### 4.3.2 Capa de procesamento asíncrono

Esta capa é para manter a responsividade da web implementando procesamento non bloqueante con Celery e Redis. Permite que as operacións que poden durar varios minutos (separación de *stems*, transcripción, renderizado) se executen de fondo mentres a interface web segue funcionando.

Celery actúa como xefe de tarefas asíncronas e usa Redis como *broker* de mensaxes e *backend* para almacenar resultados. A configuración está optimizada para o contexto deste proxecto, con serialización **JSON** para que haxa compatibilidade entre os contedores, e configuración de *timezone* **UTC**.

O arquivo *celery\_tasks.py* implementaría as tarefas principais: procesamento automático (*process\_automatic\_karaoke*), procesamento con letras manuais (*process\_manual\_karaoke*) e extracción de só instrumental (*process\_instrumental\_only*). Cada tarefa ten monitorización de progreso grazas a *self.update\_state()*, permitindo que a interface mostre información en tempo real ao usuario. Ademais, o sistema inclúe mecanismos de cancelación de tarefas utilizando excepcións personalizadas como *ProcessingCancelledException*.

#### 4.3.3 Capa de servizos de IA

Esta capa contén as ferramentas de intelixencia artificial, sendo o centro do sistema. Agrupa tres servizos principais: separación de fontes con **Demucs**, procesamento de fala con WhisperX e diarización con pyannote.audio.

O proceso con **Demucs** está implementado no arquivo *audio\_processing.py*, que xestiona a separación de *stems* mediante liña de comandos. O sistema implementa detección automática de capacidades **GPU** e adaptación de parámetros, con *fallback* automático a **CPU** se o procesamento con gráfica falla. A función *separate\_stems\_cli()* contén toda esta lóxica, devolvendo as rutas das pistas vocal e instrumental separadas.

O servizo WhisperX funciona como unha **API REST** independente implementada en *whisperx\_service\_api.py*. Este servizo expón **endpoint** para transcripción automática e *forced alignment*, xestionando tanto o modo automático (onde Whisper transcribe a letra automaticamente e alinea con WhisperX) como o modo manual (onde se proporcionan letras e se realiza aliñamento temporal).

A integración con pyannote.audio para *speaker diarization* é opcional e require autenticación con *tokens* de Hugging Face debido ás restricións de licenciamento. Cando está habilitada, pódense diferenciar múltiples voces nunha canción e asignar cores diferentes nos subtítulos. Detallaranse máis estes procesos en capítulos posteriores.

#### 4.3.4 Capa de renderizado

A capa de renderizado encárgase da xeración dos elementos visuais do karaoke, implementando o sistema de resaltado progresivo e do vídeo final. O motor de renderizado de texto, implementado en *karaoke\_rendering.py*, posúe un renderizado silábico avanzado mediante a biblioteca pyphen para poder resaltar sílabas en lugar de palabras, o que resulta infinitamente máis cómodo para o lector. O sistema calcula o progreso de resaltado combinando progreso

temporal e por segmentos, usando pesos configurables (*PESO\_PROGRESO\_TEMPORAL* e *PESO\_PROGRESO\_SEGMENTO*) para crear transicións suaves. Os detalles de implementación disto poderanse observar na sección 5.4.1.

#### 4.3.5 Capa de persistencia

A capa de persistencia xestiona o almacenamento de datos e metadatos do sistema, e grazas a ela hai unha axeitada persistencia estruturada e xestión de arquivos. A base de datos SQLite, implementada en *database.py*, almacena metadatos das cancións procesadas permitindo crear unha biblioteca. O esquema da base de datos inclúe información de: títulos, nomes de arquivos orixinais e procesados, tipo de fonte (YouTube/upload), parámetros de procesamento e información de diarización. Este deseño permite recuperar cancións procesadas anteriormente sen necesidade de reprocesar, mellorando a experiencia de usuario.

A xestión de arquivos baséase nunha estrutura de directorios que mellora o intercambio entre os contedores e o mantemento do sistema. Os directorios principais inclúen */input* para arquivos de entrada, */output* para resultados finais, */separated* para arquivos temporais de separación e */data* para intercambio entre contedores (SRT, metadatos). Esta organización, ademais dos volumes compartidos de Docker, permite que cada contedor acceda só aos datos que necesita. Ademais, o sistema ten utilidades para limpar automaticamente arquivos temporais e resultados antigos, xestionando eficientemente o espazo de almacenamento.

### 4.4 Modelo de datos

O modelo de datos do sistema define as estruturas de información que se intercambian entre compoñentes e como se almacenan tanto temporalmente como de forma persistente. Este modelo garante a coherencia dos datos ao longo do *pipeline* de procesamento e facilita a comunicación. A figura 4.4 mostra cómo flúen os tipos de datos dende a entrada ata a saída.

#### 4.4.1 Datos internos compartidos

O sistema mantén varias estruturas de datos internas que se comparten entre módulos e contedores.

##### Metadatos de cancións

As cancións procesadas almacénanse como rexistros na base de datos SQLite cun esquema completo que permite a recuperación posterior e a xestión da biblioteca. A estrutura de metadatos inclúe *id* única, información básica (título, nome do arquivo orixinal), arquivos

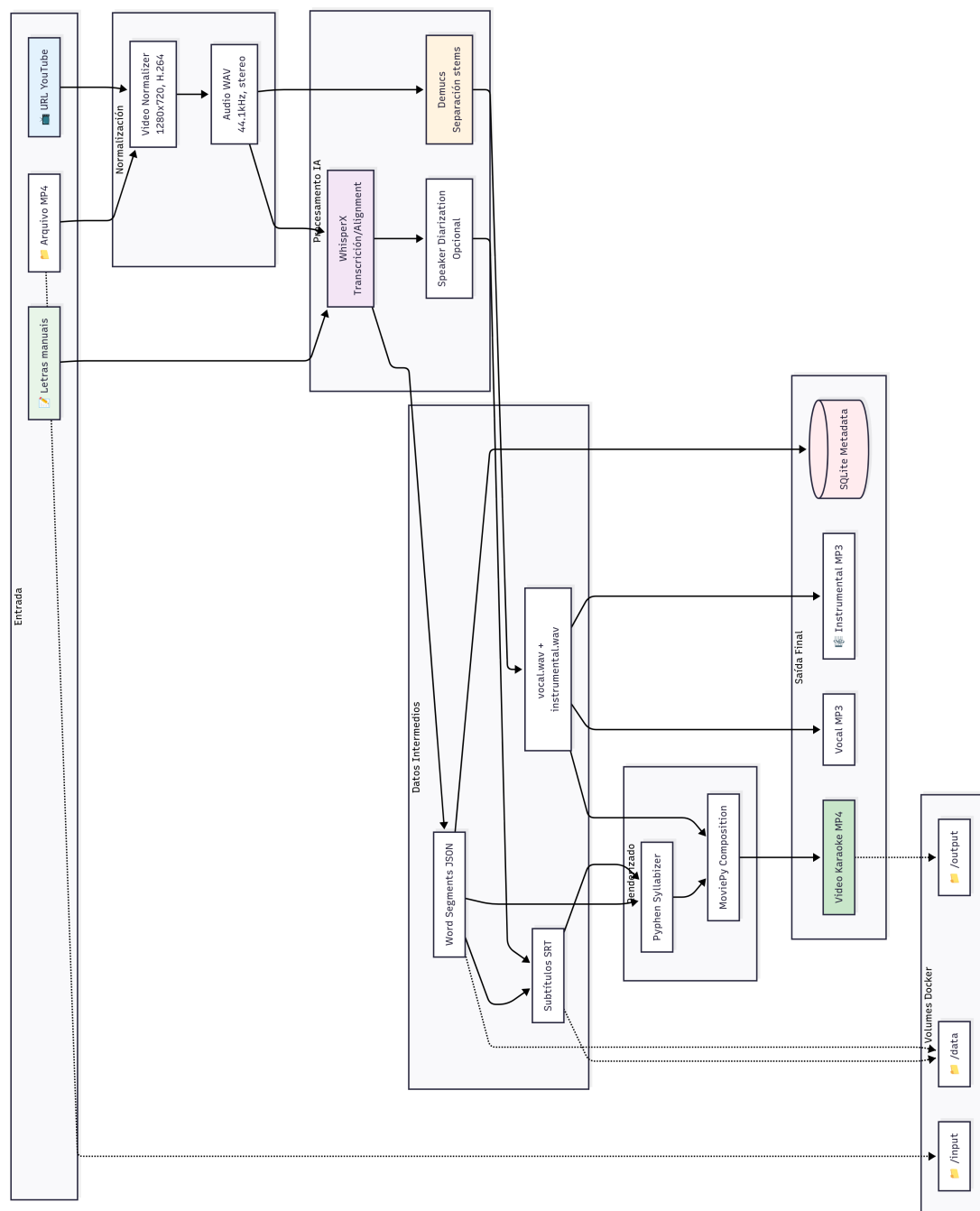


Figura 4.4: Flujo de datos e formatos a lo largo del sistema

xerados (karaoke final, só vídeo, vocal, instrumental), información da fonte (tipo: YouTube/upload, [URL](#) orixinal), parámetros de procesamento (tipo: automático/manual, letras manuais se as hai, idioma detectado), configuracións especiais (diarización habilitada), propiedades de arquivos (tamaño do arquivo, duración) e *timestamp* de creación.

Esta estrutura permite operacións como consultar cancións procesadas anteriormente, reutilizar resultados, xestionar almacenamento e proporcionar historial detallado ao usuario.

### Información de segmentación temporal

A información de segmentación temporal determina cando se resalta cada palabra ou sílaba. Os segments de palabras (*word\_segments*) teñen a seguinte estrutura:

```

1 word_segment = {
2     "word": "palabra",           # Texto da palabra
3     "start": 12.45,             # Tempo de inicio (segundos)
4     "end": 13.21,              # Tempo de fin (segundos)
5     "confidence": 0.95,         # Confianza da transcripción
6     "speaker": "SPEAKER_01"    # ID do falante (opcional)
7 }
```

Código 4.1: Estrutura de segmentación temporal

Estes *segments* úsanse para calcular o progreso de resaltado silábico, determinar cando cambiar entre liñas de texto e sincronizar cos efectos visuais. Hai algoritmos de suavizado que corrixen *segments* anormalmente longos ou curtos, como se documenta na función *clean\_abnormal\_segments()* do arquivo *utils.py*.

### Configuracións de *speaker diarization*

Cando está habilitada a diarización de falantes, o sistema mantén un mapeado de identificadores de falante a propiedades visuais. Esta configuración permite que o sistema de renderizado asigne cores diferentes segundo o falante actual, mellorando a experiencia en cancións con múltiples cantantes, como se pode observar no código 4.2.

#### 4.4.2 Formatos de intercambio

O sistema utiliza formatos de intercambio diferentes segundo o tipo de comunicación e almacenamento.

#### JSON para comunicación entre contedores

As comunicacións [HTTP REST](#) entre Flask e WhisperX utilizan [JSON](#) como formato de intercambio. As mensaxes principais inclúen formatos de solicitude e resposta, amosados nos

```

1 speaker_config = {
2     "SPEAKER_00": {
3         "color": "#FFFFFF",           # Cor branca para o principal
4         "highlight_color": "#FFFF00", # Amarelo para resaltado
5         "priority": 1 },             # Prioridade de visualización
6
7     "SPEAKER_01": {
8         "color": "#87CEEB",           # Azul para segundo falante
9         "highlight_color": "#FFB6C1", # Rosa para resaltado
10        "priority": 2 }
11 }

```

Código 4.2: Configuración de diarización por falantes

seguintes códigos:

#### Solicitud de procesamiento:

```

1 {
2     "audio_path": "/shared_data/audio_file.wav",
3     "manual_lyrics": "Letra da canción proporcionada ...",
4     "enable_diarization": true,
5     "hf_token": "hf_token_if_available",
6     "language": "es"
7 }

```

Código 4.3: Formato solicitud WhisperX

#### Resposta con resultados:

```

1 {
2     "success": true,
3     "srt_content": "contido SRT xerado ...",
4     "word_segments": [...],
5     "language_detected": "es",
6     "processing_time": 45.2,
7     "speakers_detected": [ "SPEAKER_00", "SPEAKER_01" ]
8 }

```

Código 4.4: Formato resposta WhisperX

### SRT para subtítulos temporais

Os arquivos [SRT](#) son o formato estándar para almacenar subtítulos con información temporal. O sistema xera [SRT](#) con extensións adicionais para soportar diarización. Para saber cómo é o formato dun arquivo [SRT](#) xerado pódese visualizar no código [A.6](#) do apéndice [A](#).

Este formato sae modificado cando WhisperX entra en escena para facer un resultado a nivel de palabra e non de frase, o que permite xerar o resaltado dinámico das palabras, e despois das sílabas grazas ao silabeador implementado.

### Formatos de audio e vídeo

O sistema procesa e xera múltiples formatos multimedia:

- **Audio WAV:** Utilizado para procesamento intermedio pola súa calidade sen perda e compatibilidade con ferramentas de [IA](#). Os arquivos [WAV](#) mantéñense en 44.1kHz, 16-bit, stereo.
- **Audio MP3:** Xerado para descarga de pistas instrumentais e vocais separadas. O códec utiliza [VBR](#) para optimizar tamaño vs calidade.
- **Vídeo MP4:** Formato final para os vídeos de karaoke, usando códec H.264 para vídeo e [AAC](#) para audio. Para a reprodución web faise cunha resolución de 1280x720 a 30fps.

#### 4.4.3 Protocolos de comunicación

A comunicación entre compoñentes utiliza protocolos estándar adaptados ás necesidades do sistema.

#### HTTP REST entre Flask e WhisperX

A comunicación entre o contedor principal Flask e o contedor WhisperX utiliza [HTTP REST](#) síncrono. As peticións usan POST con *payload* [JSON](#) e as respostas inclúen códigos de estado [HTTP](#) estándar: 200 para procesamento exitoso, 400 para erros de validación de entrada, 500 para erros internos de procesamento e 503 para servizo non dispoñible. Ademais o sistema implementa reintentos automáticos con *backoff* exponencial para evitar erros temporais.

#### Redis para cola de tarefas Celery

Como se comentou na sección [4.3.2](#), Redis actúa como *broker* de mensaxes para Celery. A configuración do código [4.5](#) produce que as tarefas se executen secuencialmente evitando sobrecargarse.

#### Volumes compartidos para arquivos grandes

Para arquivos grandes (vídeos, audios separados), o sistema utiliza volumes compartidos de Docker en lugar de transferencia por rede. Así redúcese a latencia e limitacións de tamaño

```
1 CELERY_CONFIG = {  
2     'broker_url': 'redis://redis:6379/0',  
3     'result_backend': 'redis://redis:6379/0',  
4     'task_serializer': 'json',  
5     'result_serializer': 'json',  
6     'task_track_started': True,          # Rastrear inicio de tarefas  
7     'result_expires': 3600,             # Resultados expiran en 1h  
8     'worker_prefetch_multiplier': 1     # Un task por worker  
9 }
```

Código 4.5: Configuración Redis para Celery

en [HTTP](#). Os volumes principais organízanse segundo o tipo de contido: *shared\_input* para arquivos de entrada (vídeos descargados, *uploads*), *shared\_output* para resultados finais, *shared\_data* para intercambio intermedio ([SRT](#), metadatos) e *separated* para arquivos temporais de [Demucs](#).

## 4.5 Fluxo de compoñentes principais

O sistema implementa tres *pipelines* de procesamento que comparten infraestrutura común pero cambian as súas estratexias segundo as necesidades do usuario. Cada *pipeline* está optimizado para un caso de uso específico.

### 4.5.1 Pipeline de procesamento automático

Este *pipeline* é completamente automático dende que se introduce unha [URL](#) de YouTube ata formar o vídeo de karaoke. O fluxo comeza cando o usuario envía unha solicitude POST á aplicación Flask cunha [URL](#). A aplicación valida a entrada, mete a tarefa *process\_automatic\_karaoke.delay()* ao *worker* de Celery e devolve un *task\_id* para monitorizar. O *worker* descarga o vídeo mediante *yt-dlp*, extrae o audio en formato [WAV](#) e chama a [Demucs](#) para separar as pistas obtendo *vocal.wav* e *instrumental.wav*. Despois realiza unha chamada [HTTP](#) POST ao contedor WhisperX que executa un *pipeline* de tres fases: transcripción automática mediante o modelo Whisper elixido, *forced alignment* para sincronizar cada palabra co audio e *speaker diarization* mediante *pyannote.audio* de xeito opcional.

O resultado final é un arquivo [SRT](#) con información temporal e un [JSON](#) con segmentos de palabras. O *worker* agrupa as palabras en frases mediante un algoritmo intelixente que se comentará en próximas seccións e procede ao renderizado. Para entender mellor o funcionamento deseñouse o diagrama de fluxo da figura 4.5.



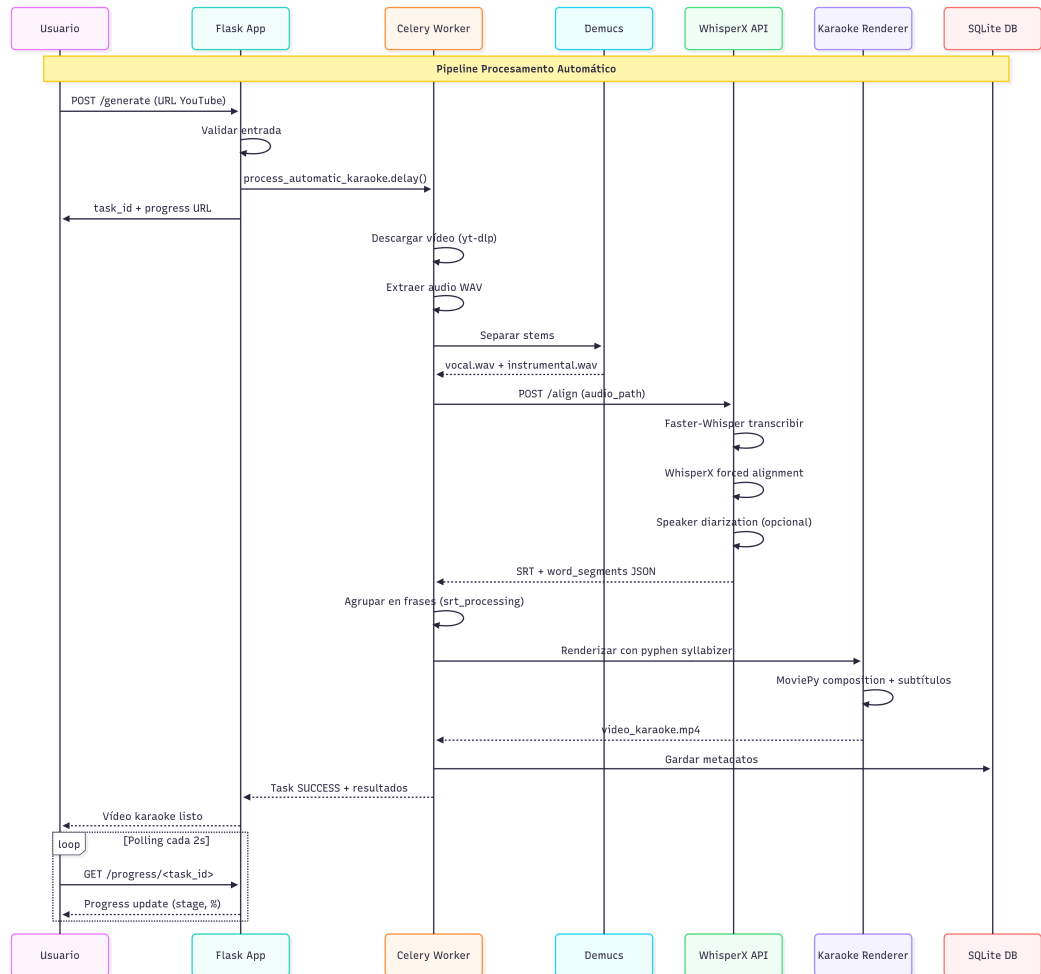


Figura 4.5: Diagrama de secuencia do modo automático

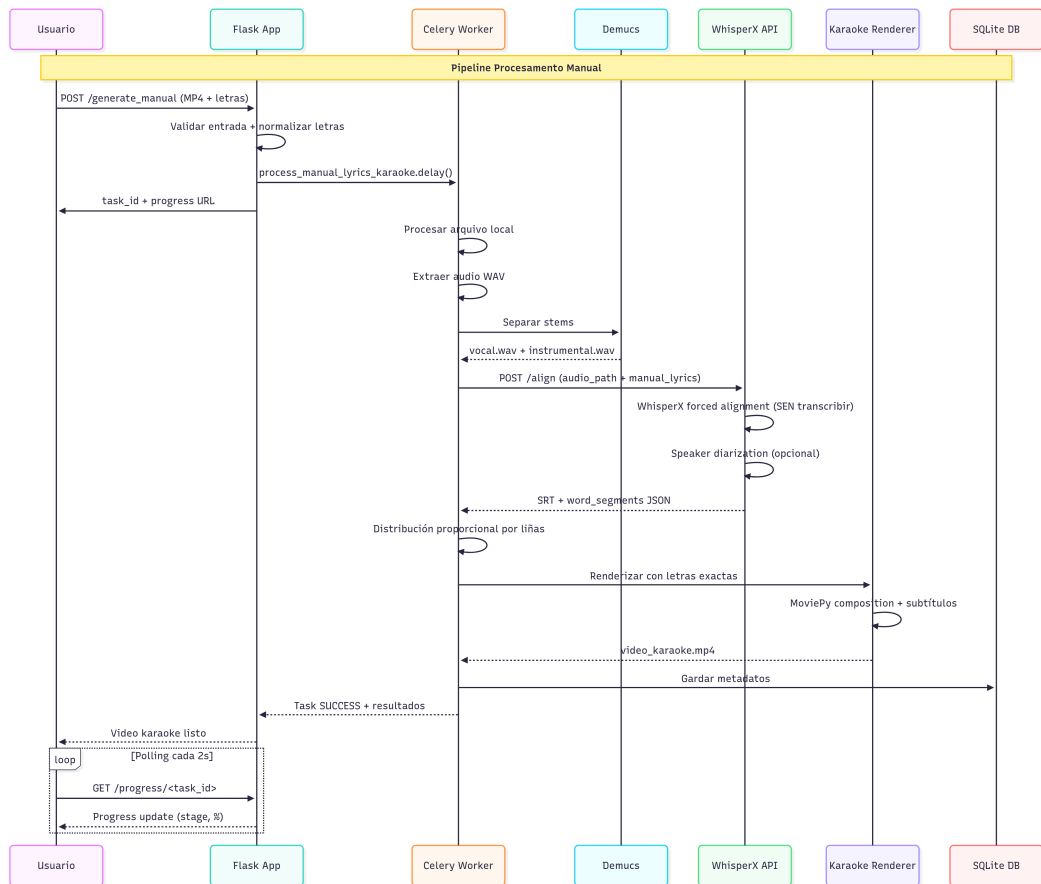


Figura 4.6: Diagrama de secuencia do modo manual

### 4.5.2 Pipeline de procesamento manual

O *pipeline* manual está deseñado para casos onde a transcripción automática non é boa, especialmente para idiomas minoritarios como o galego. Comeza cunha solicitude POST que inclúe un arquivo MP4 local ou a URL e o texto das letras proporcionadas polo usuario. A aplicación realiza validación e normalización das letras mediante *text\_processing.py*.

O procesamento de audio segue o mesmo patrón: extracción de WAV e separación mediante Demucs. A diferenza é que se omite completamente a transcripción automática. En lugar de transcribir, o *worker* envía tanto a ruta do audio como as letras manuais ao servizo WhisperX, que executa o *forced alignment*. A distribución en frases segue unha estratexia onde cada liña do texto de entrada se converte nunha frase de subtítulos, respectando a estrutura orixinal das letras marcada polo autor. É dicir, cada salto de liña detectado converteuse nunha frase. Na figura 4.6 móstrase este fluxo.

| Patrón                 | Implementación        | Vantaxes                         | Casos de uso           |
|------------------------|-----------------------|----------------------------------|------------------------|
| <i>Command</i>         | Tarefas Celery        | Execución asíncrona, cancelación | Todos os procesamentos |
| Strategy               | Modos procesamento    | Intercambiabilidade              | Automático/Manual      |
| <i>Template Method</i> | <i>Pipeline</i> común | Reutilización código             | Normalización vídeo    |

Táboa 4.2: Resumo de patróns de deseño implementados

### 4.5.3 Pipeline de só instrumental

O *pipeline* instrumental é unha versión simplificada para usuarios que unicamente necesitan separar a voz da música sen xerar subtítulos. Pódese visualizar o diagrama de secuencia da figura C.7, no apéndice A.

O fluxo executa as fases iniciais de obtención de vídeo, extracción de audio e separación de *stems*. En lugar de proceder coa transcripción, o *worker* converte directamente a pista instrumental ao formato MP3. Este *pipeline* tipicamente complétase en 1-2 minutos.

## 4.6 Patróns de deseño implementados

Nesta sección vanse comentar os patróns de deseño que proporcionan a base para o desenvolvemento e permiten que o código sexa máis estruturado e fácil de entender [48]. A continuación detállanse os principais patróns implementados e como contribúen ao sistema. A táboa 4.2 resume os patróns implementados e as súas vantaxes no contexto do sistema.

### 4.6.1 Patrón *Command*

O patrón *Command* encapsula unha solicitude como un obxecto, permitindo parametrizar aos clientes con diferentes operacións, poñer solicitudes en cola e soportar operacións que se poden desfacer [49]. No sistema este patrón está implementado mediante as tarefas Celery definidas en *celery\_tasks.py*. Cada modo de procesamento (automático, manual e instrumental) impleméntase como un comando separado que pode ser executado de forma asíncrona.

Este patrón proporciona varias vantaxes [49]: desacoplamento entre quen invoca a operación e quen a executa, flexibilidade para engadir novos tipos de procesamento, rastrexamento do estado de cada operación, cancelación controlada de operacións, etc. Na figura 4.7 pódese visualizar como se implementa este patrón no contexto do sistema.

### 4.6.2 Patrón *Strategy*

O patrón *Strategy* define unha serie de algoritmos, encapsula cada un deles e fainos intercambiáveis. Este patrón permite que o algoritmo varíe independentemente dos clientes que o

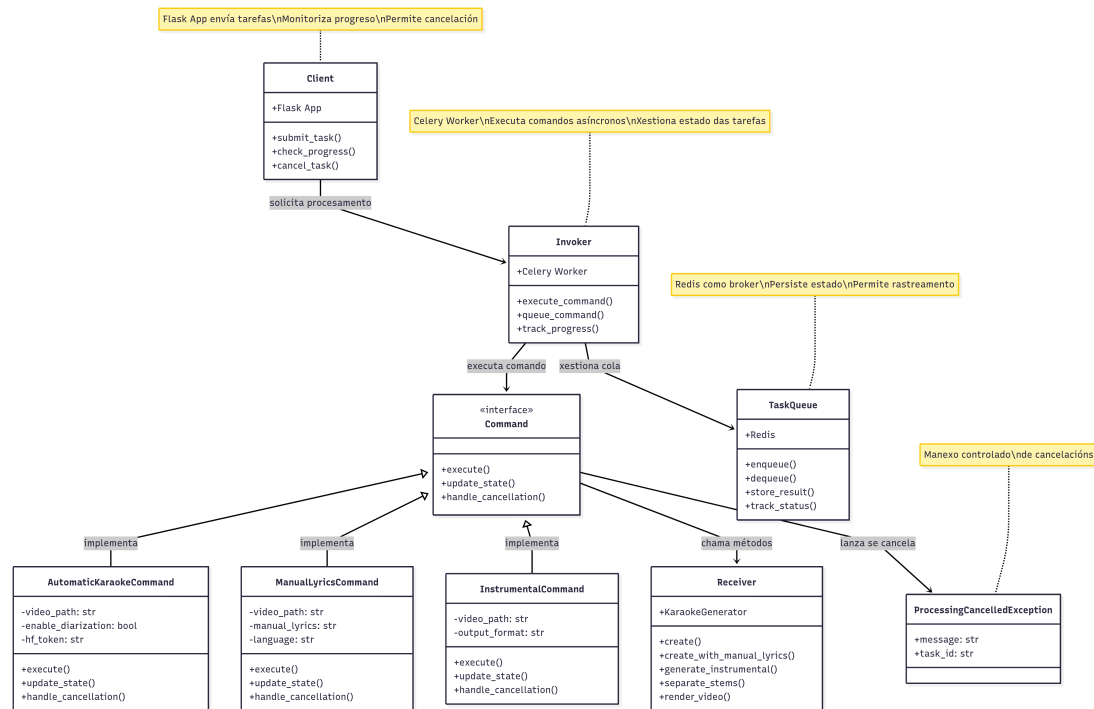


Figura 4.7: Diagrama UML do patrón Command implementado con Celery

utilizan [50]. No sistema está implementado principalmente na xestión dos diferentes modos de procesamento e nas estratexias de normalización de vídeo.

O sistema implementa tres estratexias principais de procesamento que comparten unha interface común pero executan algoritmos diferentes segundo as necesidades do usuario. Como se comentou na sección 4.4.2, diferentes códecs de vídeo requiren estratexias de procesamento distintas. A función *normalize\_video()* implementa o patrón Strategy para xestionar estas diferenzas. Isto garante compatibilidade con practicamente calquera formato de vídeo de entrada.

### 4.6.3 Patrón *Template Method*

O patrón *Template Method* define o esqueleto dun algoritmo nunha superclase, permitindo que as subclases sobrescriban pasos específicos do algoritmo sen cambiar a súa estrutura [51]. No sistema está implementado no *pipeline* común de procesamento que todos os modos seguen cun conxunto de variacións específicas. O sistema de renderizado tamén implementa un *template method* para crear clips de texto con variacións segundo o tipo de contido.

# Detalles de implementación

---

NESTE capítulo preséntanse os detalles técnicos da implementación do sistema. Mentres que o capítulo anterior centrouse na arquitectura e o deseño xeral do sistema, este capítulo aborda o "COMO": algoritmos concretos, as optimizacións implementadas e os detalles técnicos. Tamén se describen os compoñentes máis complexos do código, as decisións de implementación e as solucións surxidas. Todo o código desenvolto está dispoñible na seguinte ligazón: <https://github.com/gaelfernandez1/KaraokeProject> [41].

## 5.1 Implementación de contedores

Nesta sección vaise comentar máis en detalle a configuración do Docker e a implementación de cada un dos cinco contedores nos que está separado o sistema.

### 5.1.1 Configuración do ecosistema Docker

O arquivo *docker-compose.yml* define completamente os contedores, establece as relacións entre servizos, a configuración de volumes e a xestión de recursos. A configuración utiliza volumes compartidos para o intercambio de datos entre contedores, evitando a necesidade de transferir arquivos grandes a través da rede (código 5.1).

### 5.1.2 Contedor Flask principal (demucs\_container)

O contedor principal ten a aplicación Flask e o procesamento con [Demucs](#). Este é o punto de entrada principal do sistema e xestiona toda a lóxica de negocio. A súa configuración inclúe:

- **Porto exposto:** 5000 para a interface web
- **Dependencias:** Redis e WhisperX deben estar activos antes do seu arranque

```
1 volumes :  
2   shared_data :  
3     driver: local  
4   shared_input :  
5     driver: local  
6   shared_output :  
7     driver: local  
8   shared_db :  
9     driver: local  
10  redis_data :  
11    driver: local
```

Código 5.1: Configuración de volumes compartidos de Docker

```
1 command: celery -A celery_app worker --loglevel=info  
2 environment:  
3   - CELERY_BROKER_URL=redis://redis:6379/0  
4   - CELERY_RESULT_BACKEND=redis://redis:6379/0
```

Código 5.2: Configuración do worker Celery

- **Volumes montados:** Acceso a todos os volumes compartidos para xestionar ficheiros
- **Configuración GPU:** Acceso completo aos recursos NVIDIA para aceleración de *Demucs*. Pódese ver esta configuración no código A.7 do apéndice A.

A configuración do hardware é especialmente importante neste contedor, xa que *Demucs* presenta unha diferenza de tempo significativa entre o procesamento con CPU e GPU, gañando este último por bastante.

### 5.1.3 Contedor *worker* de Celery (*celery\_worker\_container*)

Este contedor executa as tarefas de procesamento asíncrono, así a interface web permanece responsiva mentres se procesan os vídeos. Utiliza a mesma imaxe que o contedor principal pero cun comando específico para executar o *worker* de Celery, como se ve no código 5.2.

O *worker* ten acceso aos mesmos volumes que o contedor principal, permitindo procesar os arquivos xerados pola aplicación web e compartir resultados a través do sistema de arquivos.

### 5.1.4 Contedor WhisperX (*whisperx\_container*)

Este contedor funciona como un servizo independente que expón unha API REST no porto 5001. A separación é especialmente importante porque WhisperX require versións específicas de dependencias que entran en conflito coas de *Demucs*. O *endpoint* principal de WhisperX pode apreciarse axeitadamente no código A.2 do apéndice A.

```
1 command: http demucs:5000 --log stdout
2 ports:
3   - "4040:4040"
4 environment:
5   - NGROK_AUTH_TOKEN=${NGROK_AUTH_TOKEN}
```

Código 5.3: Configuración de Ngrok

### 5.1.5 Contedor Redis (`redis_container`)

Redis funciona como *broker* [42] de mensaxes para Celery e como *backend* para almacenar resultados de tarefas. A súa configuración é estándar, utilizando a imaxe oficial de Redis con persistencia de datos:

- **Porto exposto:** 6379 (estándar Redis)
- **Volume de datos:** `redis_data` para persistencia
- **Política de reinicio:** `unless-stopped`

### 5.1.6 Contedor Ngrok (`ngrok_container`)

Este contedor proporciona un túnel [HTTPS](#) público que permite compartir a aplicación con usuarios externos sen necesidade de configurar infraestrutura de rede. É especialmente útil para demostracións e probar o sistema sen necesidade de instalar o entorno completo. A configuración Ngrok pode verse no código 5.3.

## 5.2 Pipeline de procesamiento de audio

O procesamiento de audio é un dos núcleos do sistema. Esta sección detalla as implementacións que permiten procesar calquera vídeo de entrada e extraer as pistas de audio para o resto do pipeline.

### 5.2.1 Normalización de vídeo

Precísase garantir que vídeos con formatos moi diferentes poidan ser procesados de forma consistente. Durante o desenvolvemento atopáronse problemas cando diferentes formatos de entrada (especialmente códecs modernos) producían subtítulos mal postos debido ás diferenzas na estrutura temporal dos *frames*.

A solución está en `video_processing.py`, que aplica varias estratexias de normalización que se proban unha por unha ata que unha funcione. Na figura 5.2 pode verse o diagrama de fluxo. Na táboa 5.1 móstrase as diferentes estratexias implementadas e os seus casos de uso.

| Estratexia          | Códecs obxectivo          | Características                 | Prioridade |
|---------------------|---------------------------|---------------------------------|------------|
| Recodificación AV1  | AV1, VP9                  | Optimizada para códecs modernos | 1          |
| Recodificación H264 | HEVC, códecs propietarios | Compatibilidade ampla           | 2          |
| Escalado simple     | MP4 estándar              | Mínima recodificación           | 3          |
| MoviePy fallback    | Calquera                  | Máxima compatibilidade          | 4          |

Táboa 5.1: Estratexias de normalización de vídeo implementadas

A estratexia de MoviePy como *fallback* é especialmente importante porque utiliza unha aproximación completamente diferente, calculando o redimensionamento mediante operacións de composición con *letterbox* automático [17].

### 5.2.2 Extracción e conversión de audio

Xestiónanse tanto arquivos de vídeo MP4 como arquivos de audio MP3. O sistema implementa xestión de memoria mediante a liberación de recursos AudioClip, evitando *memory leaks* que poden producirse durante o procesamento de arquivos grandes.

### 5.2.3 Separación de fontes con Demucs

A función `separate_stems_cli()` implementa un sistema de detección automática de hardware con *fallbacks* GPU→CPU, optimizando dinámicamente os parámetros segundo as capacidades dispoñibles.

$$\text{segment\_size} = \begin{cases} 5 & \text{se GPU\_memory} < 5\text{GB} \\ 8 & \text{se } 5 \leq \text{GPU\_memory} < 6\text{GB} \\ 15 & \text{se GPU\_memory} \geq 8\text{GB} \\ 10 & \text{en caso contrario} \end{cases} \quad (5.1)$$

Esta optimización é fundamental porque o tamaño de segmento afecta tanto á velocidade como ao uso de memoria de Demucs. Segmentos máis grandes melloran a calidade da separación pero requiren máis memoria GPU.

Unha vez completada a separación, o sistema ten que xestionar os arquivos resultantes. Demucs pode xerar diferentes formatos dependendo da configuración (*stems* individuais ou agrupados), polo que o código implementa a lóxica para unir os resultados mediante combinación FFmpeg. Esta implementación asegura que o sistema sempre produce dous arquivos finais: a pista vocal e a pista instrumental, preparadas para as seguintes fases.

Cómpre aclarar que Demucs produce tres resultados ou *stems*, *bass*, *drums*, e *other*, que hai que combinar cun *script* manual para crear a instrumental completa porque Demucs non o fai



automaticamente. De feito, nas figuras C.2, C.3, C.4, C.6 e C.5 ubicadas no anexo C, pódense visualizar os espectogramas resultantes de todas as partes que son producidas a partir dun mesmo audio de 5 segundos tras unha execución do programa [52].

## 5.3 Sistema de transcripción e aliñamento

O sistema de transcripción e aliñamento é o compoñente máis crítico. Esta sección describe a implementación que combina varias tecnoloxías de recoñecemento de fala e aliñamento temporal, dando solucións tanto para o modo automático como para o modo de letras manuais.

### 5.3.1 Transcripción automática: Integración Whisper + WhisperX

Este modo combina dúas ferramentas complementarias: Whisper para a transcripción inicial e WhisperX para o aliñamento temporal. Esta arquitectura híbrida xurdiu para solucionar os problemas de precisión que apareceron no desenvolvemento, concretamente a detección de actividade vocal (VAD) que se configura con parámetros específicos para música, onde as pausas son máis curtas que na fala normal.

O algoritmo selecciona automaticamente entre modo automático (transcripción + aliñamento) e modo manual (só aliñamento forzado), tal e como indica o diagrama da figura 5.1.

### 5.3.2 Forced alignment: Algoritmos de aliñamento temporal

O aliñamento forzado implementa algoritmos que sincronizan texto proporcionado manualmente co audio correspondente, determinando exactamente cando se pronuncia cada palabra/fonema. WhisperX carga un modelo específico para o idioma detectado, e estes modelos coñecen a fonética do idioma (cómo se pronuncian as palabras). A continuación o audio convértese en espectogramas de mel (representación visual do son) e extraíense características acústicas *frame* por *frame* (cada 20ms). Despois execútanse os seguintes pasos:

- Usa DTW ou algoritmos similares
- Compara as características acústicas cos fonemas esperados do texto
- Encontra a mellor correspondencia temporal

O aliñamento forzado proceso é esencial para idiomas como o galego, onde a transcripción automática non sae ben. A función `execute_whisperx_alignment()` implementa o algoritmo cun sistema de *fallbacks* multilingües: galego→español→inglés, proporcionando compatibilidade mesmo cando os modelos específicos non están dispoñibles.

Para o manexo de discrepancias texto-audio, a función `group_word_segments` é especialmente sofisticada e pódese ver a distribución temporal intelixente no código 5.4.

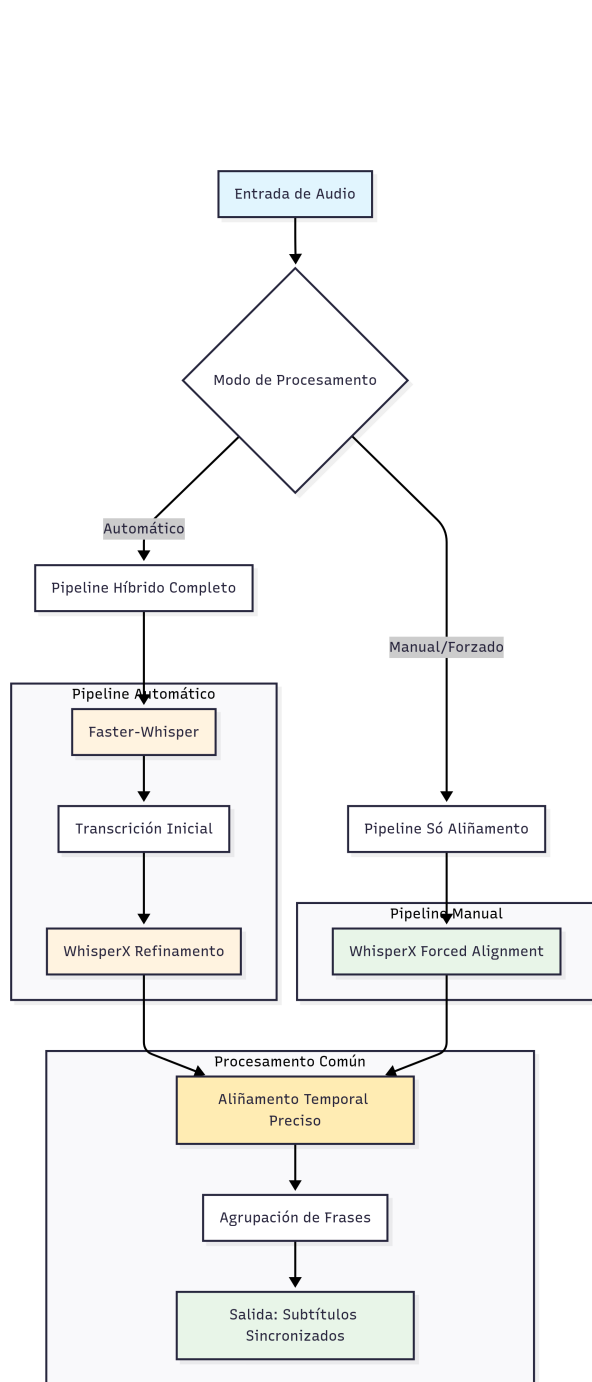


Figura 5.1: Diagrama de decisión Faster-Whisper vs WhisperX segundo o modo de entrada

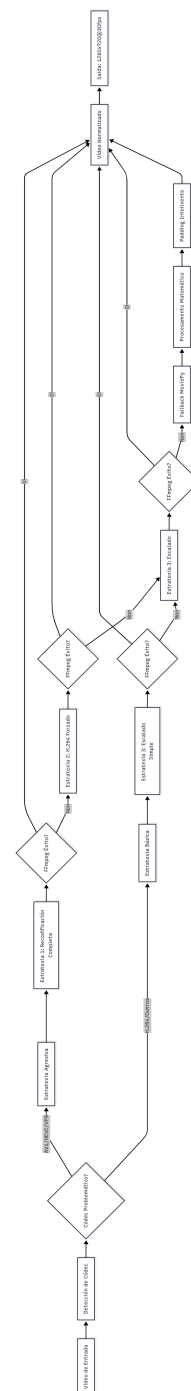


Figura 5.2: Diagrama de flujo das estrategias de normalización

```

1 # Conta palabras esperadas vs palabras reais transcritas
2 contador_tokens_manual = [len(línea.split()) for línea in líneas]
3 total_esperado = sum(contador_tokens_manual)
4 total_real = len(word_segments)
5
6 proporcional = [(cnt/total_esperado) * total_real for cnt in
7                 contador_tokens_manual]
8
9 #Para asegurar que cada liña teña 1 palabra ou mais se corresponde
10 while diferencia != 0:
11     if diferencia > 0:
12         # Aumentar donde o decimal sexa maior
13         índice = max(range(len(decimales)), key=lambda i:
14                       decimales[i])
15         asignados[indice] += 1

```

Código 5.4: Distribución proporcional intelixente

Estas discrepancias son problemáticas porque o cantante pode repetir palabras que non están no texto, saltar palabras ou as pausas poden non coincidir cos saltos de liña. Tamén é probable que a letra non se corresponda coa realidade. Por exemplo, moitas páxinas de líricas aforran espazo nos coros repetitivos, engadindo un x3 por exemplo a unha frase que se repite 3 veces, ou incluso non indican esas repeticións.

Todas estas deficiencias poden afectar á sincronización temporal nesa parte da canción, pero o sistema recupérase en puntos posteriores cando as letras volven a coincidir. Todo dependendo da potencia do modelo utilizado. Será deber do usuario asegurar a veracidade das letras que se proporcionan.

Finalmente o *forced alignment* xunto co silabeador transforma esta entrada:

"Estamos ben aquí" (texto) + audio.wav

Nesta saída:

|      |      |      |      |      |      |
|------|------|------|------|------|------|
| Es-  | ta-  | mos  | ben  | a-   | quí  |
| 0.5s | 1.2s | 1.8s | 2.3s | 2.9s | 3.4s |

O sistema de renderizado (*karaoke\_rendering.py*) usa eses *timestamps* ou marcas temporais para o resultado progresivo sílaba por sílaba, creando o efecto de karaoke profesional.

### 5.3.3 Procesamento de texto: Normalización de letras manuais

As letras proporcionadas polo usuario normalízanse, eliminando elementos que poden interferir co aliñamento temporal (etiquetas [HTML](#), caracteres de formato, signos de puntuación problemáticos) mentres preserva con coidado caracteres especiais útiles para pronunciar correctamente. Tamén se implementan solucións para as discrepancias texto-audio que se

| Speaker    | Cor normal       | Cor resaltado     |
|------------|------------------|-------------------|
| SPEAKER_00 | #FFFFFF (Branco) | #FFFF00 (Amarelo) |
| SPEAKER_01 | #87CEEB (Branco) | #FFB6C1 (Rosa)    |
| SPEAKER_02 | #98FB98 (Branco) | #FFA500 (Laranxa) |
| SPEAKER_03 | #DDA0DD (Branco) | #FF69B4 (Rosa)    |

Táboa 5.2: Sistema de cores para diferenciación de speakers

comentou no anterior apartado, pero se a diferenza entre a letra proporcionada e a realidade é moi esaxerada os resultados poden non ser eficientes.

#### 5.3.4 *Speaker diarization*: Implementación con pyannote.audio

A diarización de falantes permite diferenciar as diferentes voces nunha canción, asignando cores diferentes nos subtítulos para cada cantante. O código usa pyannote.audio con configuración para música (non fala), axustando parámetros de [clustering](#) e segmentación temporal. A asignación de cores está feita para asegurar o contraste visual.

O sistema implementa lóxica de *fallback* cando pyannote.audio non está dispoñible ou cando os *tokens* de Hugging Face non son válidos. Así o sistema funciona correctamente mesmo sen diarización, simplemente usando unha soa cor para todos os subtítulos.

O núcleo do sistema está na función *assign\_speaker\_to\_word*. Úsase o punto medio de cada palabra como referencia principal, evitando problemas con palabras que se estendan máis alá de *boundaries* de *speakers*. Nos casos onde varios *speakers* teñen validez temporal, priorízase aquel con mais duración de solapamento (código 5.5).

En segmentos onde moitas voces cantan ao mesmo tempo, a diarización pode ser ambigua. O algoritmo xestiona esta ambigüidade priorizando a consistencia temporal sobre a precisión do *speaker*. Por outra banda, Pyannote.audio está optimizado para conversacións, non música. Como se comentou ao principio desta sección, o sistema mitiga esta deficiencia mediante o axuste de parámetros. Outra alternativa sería executar a diarización sobre a parte vocal da canción unha vez eliminada a instrumental. Pero, como veremos no capítulo 6, a solución proposta supera a calidade dos resultados producidos pola diarización da parte sen música.

## 5.4 Motor de renderizado de karaoke

Encárgase de transformar a información temporal dos subtítulos nun vídeo de karaoke cun resaltado fluído. Esta sección describe as técnicas implementadas para facer os vídeos.

```

1 def assign_speaker_to_word(word_segment, speaker_segments):
2     word_start = word_segment["start"]
3     word_end = word_segment["end"]
4     word_center = (word_start + word_end) / 2 #Punto temp. central
5     best_speaker = None
6     best_overlap = 0
7
8     for speaker_seg in speaker_segments:
9         # Cálculo de superposición temporal
10        overlap_start = max(word_start, speaker_seg["start"])
11        overlap_end = min(word_end, speaker_seg["end"])
12        overlap_duration = max(0, overlap_end - overlap_start)
13
14        # Criterio asignación: centro + máxima superposición
15        if (speaker_seg["start"] <= word_center <=
16            speaker_seg["end"] and
17            overlap_duration > best_overlap):
18            best_overlap = overlap_duration
19            best_speaker = speaker_seg["speaker"]
20    return best_speaker

```

Código 5.5: Función para asignar cada palabra ao seu falante correspondente

#### 5.4.1 Renderizado silábico: Algoritmo pyphen + resaltado progresivo

Mentres que a maioría de sistemas de karaoke automáticos resaltan palabras completas, o noso sistema divide cada palabra en sílabas e resalta cada unha individualmente, proporcionando un seguimento moito máis fluído do ritmo da canción. O algoritmo utiliza pyphen para dividir, calcular duracións proporcionais e crear segmentos temporais para cada sílaba:

$$\text{progreso\_combinado}(t) = \alpha \cdot P_{\text{temporal}}(t) + (1 - \alpha) \cdot P_{\text{segmental}}(t) \quad (5.2)$$

con

$$P_{\text{temporal}}(t) = \min \left( 1.0, \max \left( 0.0, \frac{t - t_{\text{inicio}}}{t_{\text{fin}} - t_{\text{inicio}}} \right) \right) \quad (5.3)$$

$$P_{\text{segmental}}(t) = \frac{N_{\text{completos}}}{N_{\text{total}}} + \frac{f_{\text{parcial}}(t)}{N_{\text{total}}} \quad (5.4)$$

$$\alpha = 0.2 \text{ (peso temporal)}, \quad (1 - \alpha) = 0.8 \text{ (peso segmental)} \quad (5.5)$$

onde  $f_{\text{parcial}}(t)$  é o progreso dentro do segmento actual:

$$f_{\text{parcial}}(t) = \begin{cases} 0 & \text{se } t < t_{\text{seg\_inicio}} \\ \min \left( 1.0, 1.2 \cdot \frac{t - t_{\text{seg\_inicio}}}{t_{\text{seg\_fin}} - t_{\text{seg\_inicio}}} \right) & \text{se } t_{\text{seg\_inicio}} \leq t < t_{\text{seg\_fin}} \\ 0 & \text{se } t \geq t_{\text{seg\_fin}} \end{cases} \quad (5.6)$$

```

1 def render_line_image(line_info: dict, t_offset: float):
2     # Cálculo de progreso combinado temporal e por segmentos
3     progreso_temporal = min(1.0, max(0.0, t_offset /
4     duracion_total_liña))
5     progreso_final = calcular_progreso_por_segmentos(word_segments,
6     t_offset)
7     progreso_combinado = (progreso_temporal *
8     PESO_PROGRESO_TEMPORAL) + \
9     (progreso_final * PESO_PROGRESO_SEGMENTO)
10
11     # Aplicación de resaltado silábico con pyphen
12     for palabra in texto_completo.split():
13         silabas_palabra = dic_pyphen.inserted(palabra).split('-')

```

Código 5.6: Algoritmo de renderizado silábico avanzado

Esta fórmula permite que o resaltado avance suavemente no tempo pero tamén se axuste aos límites reais dos segmentos, evitando que o resaltado se anticipe ou atrase demasiado respecto ao real. O algoritmo de progreso temporal calcula cantas sílabas resaltar baseándose no tempo transcorrido, nos segmentos de audio completados, no *buffer* de anticipación para a última sílaba e no factor de aceleración progresiva. O código completo do cálculo do progreso temporal A.3 amósase no apéndice A, onde se pode comprobar a súa complexidade.

Por outra banda, como se comentou na sección 4.3.4, o código 5.6 amosa como o sistema calcula a combinación de progresos usando pesos configurables para conseguir un renderizado silábico avanzado. Por último, a silabificación require un procesamento adicional para mapear correctamente as sílabas cos tempos de audio (ver código A.5 do apéndice A).

#### 5.4.2 Xeración de clips de texto: `create_karaoke_text_clip()`

A función `create_karaoke_text_clip()` contén un sistema de renderizado *per-frame* que calcula o progreso (temporal + segmental) e aplica os efectos visuais en tempo real. A función execútase para cada frame (é dicir, a 30 FPS). Manipúlanse directamente arrays Numpy e compóñense as matrices multidimensionais. Ademais manéxanse ao mesmo tempo varios offsets temporais: *advance*, *duration\_padding* e *offset\_visualizacion*. Calquera ineficiencia multiplícase por 30 frames/segundo. A función completa xeradora de clips de texto con sincronización silábica aparece no listado A.4, no apéndice A.

#### 5.4.3 Composición multicapa: `MoviePy CompositeVideoClip`

A composición final do vídeo usa MoviePy para combinar moitas capas visuais nunha soa secuencia. O sistema utiliza MoviePy CompositeVideoClip para facelo: vídeo de fondo escurecido (30%), subtítulos principais con resaltado e ademais con tipografía dinámica (axústase automaticamente o tamaño de fonte segundo o contido), previsualización da seguinte

| Capa                  | Contido                         | Z-Index | Posición               |
|-----------------------|---------------------------------|---------|------------------------|
| Fondo                 | Vídeo orixinal escurecido (30%) | 0       | Pantalla completa      |
| Subtítulos principais | Texto con resaltado silábico    | 1       | Centro-inferior        |
| Previsualización      | Seguiente liña (texto tenue)    | 2       | Centro-inferior+offset |

Táboa 5.3: Capas de composición do vídeo de karaoke

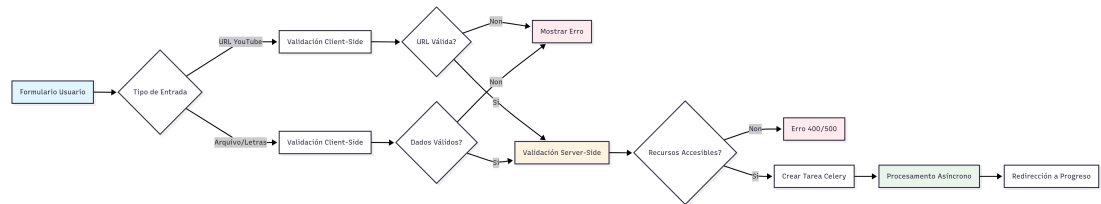


Figura 5.3: Diagrama de validación de formularios

liña, a cal se renderiza simultaneamente coa principal, entre outros. A táboa 5.3 describe as diferentes capas e a súa función no vídeo final.

## 5.5 Interface web e xestión de sesións

A interface web é o punto de entrada principal do sistema para os usuarios finais. Esta sección detalla a implementación da aplicación Flask, o sistema de xestión de sesións asíncronas e a biblioteca que permite aos usuarios reproducir os seus karaokes xerados.

### 5.5.1 Arquitectura Flask: Rutas, *templates*, formularios

A aplicación implementa rutas especializadas con validación multinivel: verificación de URLs YouTube, estrutura de letras manuais e asíncronía con Celery para tarefas longas. O sistema de *templates* usa Jinja2 con arquitectura de bloques e Bootstrap 5 para o deseño responsivo. A validación de formularios implementa lóxica, tanto no *frontend* como no *backend*, para a integridade dos datos. Podemos observar na figura 5.3 o fluxograma de validación de formularios mostrando validación *client-side* → *server-side* → procesamento.

Nas figuras 5.4a e 5.4b móstrase a estética dos formularios implementados para a interface web, incluíndo os módulos de xeración automática e manual. Para máis información deste aspecto, pódense consultar nas figuras C.11 e C.12 do apéndice C as estéticas da cabeceira da páxina e o formulario de xeración de instrumental.

(a) Formulario modo automático páxina principal (b) Formulario individual modo de letra manual

Figura 5.4: Formularios da aplicación web en modo automático e manual

### 5.5.2 Procesamento asíncrono: Integración Celery + tracking de progreso

O sistema implementa **polling** con frecuencia adaptativa e reintentos automáticos para actualizar a interface en tempo real. Detecta tarefas soltas tras posibles fallos do sistema, mantén a consistencia entre Redis e os procesos activos e xestiona os *timeouts* e dalle *feedback* ao usuario. Ademais, a coordinación entre o contedor principal e WhisperX comprende reintentos lóxicos, validar a integridade dos datos, sincronizar os volumes compartidos, etc. O sistema ten que coordinar procesos con características moi diferentes: a transcripción e o aliñamento, o renderizado, o procesamento GPU, etc. No listado A.8 do apéndice A está parte do código da coordinación de procesos en *celery\_tasks*.

### 5.5.3 Sistema de biblioteca: Base de datos SQLite, metadata\_utils

O sistema de biblioteca permite aos usuarios xestionar todos os karaokes xerados, proporcionando funcionalidades de busca, reprodución e eliminación. A implementación utiliza SQLite como base de datos pola súa simplicidade. O esquema SQLite implementa unha táboa completa con metadatos de procesamento, configuracións de diarización, propiedades de arquivos e índices para consultas. É necesario rastrexar moitos arquivos relacionados por cada canción procesada e manter consistencia entre o sistema de arquivos e a base de datos. Na figura C.13 do apéndice C pódese observar a estética da interface da biblioteca.



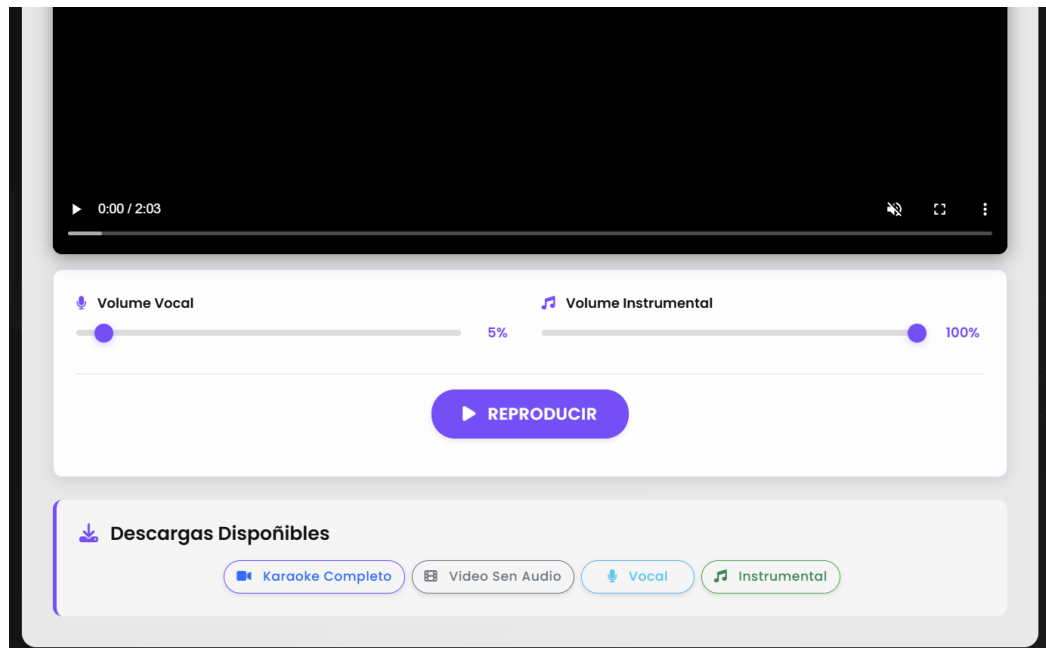


Figura 5.5: Interface do reprodutor web

#### 5.5.4 Reprodutor web: `player.html`, controis personalizados

O reprodutor web implementa [HTML5](#) con controis superpostos, funcionalidades de karaoke como aumentar e diminuir o volume do cantante, sincronización dual de audio (vocal + instrumental) e carga asíncrona de pistas. A complexidade radica na sincronización de tres pistas de audio en tempo real. Sincronízase cada 100ms entre vídeo muteado + 2 pistas de audio independentes (código [5.7](#)). Para ter unha idea xeral de como é visualmente a interface do reprodutor, pódese ver a figura [5.5](#). Nela pódese ver a funcionalidade de controlar os audios vocal e instrumental de xeito independente.

### 5.6 Optimizacións específicas

Nesta sección trátanse aspectos específicos interesantes do proxecto. Malia que non sexan tan importantes como o resto de funcionalidades, precísanse para o óptimo funcionamento do sistema.

#### 5.6.1 *Logging e debugging*: Sistema de trazabilidade de erros

Existen varias optimizacións que adaptan o funcionamento ás capacidades específicas de cada entorno, mellorando tanto o rendemento como a experiencia do usuario. O sistema implementa *logging* estruturado con múltiples niveis que facilita o *debugging* e a monitorización

```

1 syncAudioToVideo() {
2     const videoTime = this.video.currentTime;
3     if (Math.abs(this.vocalAudio.currentTime - videoTime) > 0.2) {
4         this.vocalAudio.currentTime = videoTime;
5     if (Math.abs(this.instrumentalAudio.currentTime - videoTime) >
6         0.2) {
7         this.instrumentalAudio.currentTime = videoTime;}}
7 setupSynchronization() {
8     setInterval(() => {
9         if (this.isPlaying) {this.syncAudioToVideo();}
10    }, 100);
11 }

```

Código 5.7: Algoritmo de sincronización multimedia

```

1 def check_if_cancelled():
2     if current_task.request.id in active_processes:
3         status =
4         active_processes[current_task.request.id].get('status')
5         if status in ['REVOKED', 'CANCELLED']:
6             raise ProcessingCancelledException("Tarefa cancelada")
7
8     if current_task.request.called_directly is False:
9         try:
10             # Obter o estado actual desde Redis
11             task_result =
12             celery.AsyncResult(current_task.request.id)
13             if task_result.state == 'REVOKED':
14                 raise ProcessingCancelledException("Tarefa
15                 cancelada")
16         except:
17             pass

```

Código 5.8: Función de chequeo de cancelación

do rendemento. Cada compoñente ten o seu propio *logger* configurado. O sistema de trazabilidade permite seguir completamente o procesamento dunha canción dende a entrada ata a saída final, facilitando a identificación de problemas.

### 5.6.2 Cancelación de tarefas con `ProcessingCancelledException`

O sistema permite cancelar tarefas en execución de forma limpa, liberando recursos e realizando *cleanup* automático. A cancelación está integrada con Celery mediante sinais periódicos e permite ao usuario deter o procesamento en calquera momento sen deixar recursos bloqueados (código 5.8).

### 5.6.3 Validación de entrada: *security\_config.py*, sanitización de arquivos

O sistema contén capas de seguridade que protexen contra ataques comúns, validan todas as entradas do usuario e limitan o uso de recursos. Son aproximacións de seguridade bastante normais. *Security\_config.py* é o módulo que centraliza todas as validacións de seguridade. Existe tamén un *rate limiting* para previr abuso e que haxa sempre dispoñibilidade:

- *Rate limiting* por IP: Máximo 1000 solicitudes por hora por IP (xeral)
- *Rate limiting* por *endpoint*: 3 procesamentos por minuto para karaoke, 5 para instrumental
- Validación de arquivos: Límite de 100MB, sanitización de nomes de arquivo
- *Timeouts* de seguridade: 60 minutos para tarefas Celery, 10 minutos para comunicación con WhisperX
- *Headers* de seguridade: X-Content-Type-Options, X-Frame-Options, X-XSS-Protection

### 5.6.4 Xestión de *tokens*: HuggingFace *tokens* para *speaker diarization*

A diarización de falantes require *tokens* de HuggingFace que se xestionan de forma segura. Os tokens almacénanse cifrados e expiran automaticamente despois de 1 hora, minimizando o risco de exposición. O sistema nunca almacena *tokens* en *logs* ou bases de datos permanentes.

# Validación e probas

---

ESTE capítulo presenta a validación completa do sistema. Móstranse os resultados das probas para verificar o funcionamento de cada compoñente, así como a avaliación global do sistema en escenarios reais. A validación colle varios aspectos técnicos, así como a experiencia de usuario final.

## 6.1 Metodoloxía de validación

Nesta sección vaise explicar brevemente como se deseñaron as probas, que métricas se usaron, que conxunto de datos de proba se usaron e a configuración do entorno de probas.

### 6.1.1 Criterios de avaliación

A validación do sistema realizouse seguindo unha metodoloxía sistemática que trata tanto a funcionalidade individual de cada compoñente como o rendemento global. Definíronse estas métricas para cada compoñente:

- **Separación de fontes:** Relación sinal-distorsión ([SDR](#)), relación sinal-interferencia ([SIR](#)) e relación sinal-artefactos ([SAR](#)) [53]
- **Transcrición:** Precisión de palabras ([WER](#) - *Word Error Rate*), precisión temporal (desviación media en milisegundos)
- **Aliñamento:** Sincronización palabra-audio (desviación estándar), fluidez do resaltado silábico
- **Rendemento:** Tempo de procesamento, uso de memoria e [CPU/GPU](#)
- **Experiencia de usuario:** Facilidade de uso, calidade visual, tempo de resposta da interface

| Categoría                            | Título                          | Artista             | Duración |
|--------------------------------------|---------------------------------|---------------------|----------|
| Galego, varias voces simultáneas     | Terra                           | Tanxugueiras        | 2:59     |
| Galego, voz feminina, coros          | A fonte                         | Lontreira           | 3:06     |
| Galego, pausas longas                | Tempestades de sal              | Sés                 | 3:53     |
| Galego, Rock                         | Como Che Quero Nena             | Herdeiros da Crus   | 2:14     |
| Galego, dialecto                     | La favorita                     | The Rapants         | 2:57     |
| Galego, tradicional                  | Pirimpimpim                     | Malvela             | 3:35     |
| Galego, múltiples voces              | En fin, que máis dá             | De Ninghures        | 3:10     |
| Galego, coros longos                 | Varre Vasoira                   | Fillas de Cassandra | 3:29     |
| Español, rápida                      | Ocre                            | Luisaker            | 2:03     |
| Español, dueto, signos de puntuación | Te Regalo                       | Rels B              | 2:28     |
| Español, pop clásico, varias voces   | Resistiré                       | Dúo Dinámico        | 4:04     |
| Español, longa duración              | Entre dos tierras               | Héroes del Silencio | 6:10     |
| Español, gravación en vivo           | La vereda de la puerta de Atrás | Extremoduro         | 3:52     |
| Inglés, voz prominente               | Someone Like You                | Adele               | 4:45     |
| Inglés, guitarra                     | Sweet Child O' Mine             | Guns N' Roses       | 5:03     |
| Inglés, voz procesada                | Slalom                          | Healy               | 3:53     |
| Inglés, harmonías vocais             | Don't Stop Me Now               | Queen               | 3:33     |
| Inglés, electrónica                  | The Nights                      | Avicii              | 3:10     |

Táboa 6.1: [dataset](#) de validación utilizado nas probas

### 6.1.2 Conxunto de datos de proba

Seleccionouse un conxunto diverso de 18 cancións que abarcan diferentes idiomas, xéneros e características técnicas. A táboa 6.1 mostra a distribución do [dataset](#) de validación. Este conxunto foi seleccionado estratéxicamente para facer os diferentes tipos de probas dos que se falarán neste capítulo.

### 6.1.3 Configuración do entorno de probas

Todas as probas foron feitas no equipo persoal do estudante mencionado na sección 3.5.2. Ademais, fixéronse pequenas probas nun equipo diferente con outras características, sobre todo para probar o funcionamento do entorno nun equipo no que non foi desenvolvido o proxecto, para ver se o sistema é compatible con outras máquinas. De todos os xeitos, as probas principais das que falaremos en próximas seccións, foron realizadas no equipo portátil do estudante. Antes de comezar, cómpre aclarar que todas as probas a continuación son feitas baixo o modelo *medium* de Whisper, tanto para a transcripción como para a aliñación. Todas se fixeron tamén co equipo conectado á corrente e cunha conexión Wifi estable de 300 mbps.

## 6.2 Validación por compoñentes

Nesta sección avaliarase o funcionamento compoñente por compoñente. Probas de calidade, métricas, opinión persoal, etc.

### 6.2.1 Separación de fontes musicais

En canto ao funcionamento concreto desta parte, analizarase a calidade da separación, o rendemento computacional e a robustez ante os diferentes xéneros musicais.

#### Calidade da separación vocal-instrumental

Os resultados producidos por [Demucs](#) adóitanse medir mediante as métricas [SDR](#) (cuantifica a relación entre a potencia da sinal obxectivo e a potencia do erro na saída esperada), [SIR](#) (mide o grao no que se suprime a interferencia doutras fontes), [SAR](#) (avalía a cantidade de distorsión artificial introducida durante o proceso de separación) [53]. Porén, para este traballo optouse por facer unha valoración subxectiva dos resultados, posto que a validación destas métricas pode verse e está documentado na rede. Para máis información pódese comprobar o xa mencionado na sección [2.6.1 Sound Demixing Challenge](#) [25]

En canto á calidade dos resultados tendo en conta o [dataset](#) utilizado, pódese concluír que a separación para ditas cancións é moi bo. Apenas se aprecia ruído e todas as partes instrumentais parecen ser separadas correctamente. Cómpre aclarar que [Demucs](#) produce como saída tres compoñentes instrumentais que se teñen que mezclar mediante un *script* implementado manualmente (máis información na sección [5.2.3](#)), o que pode alterar a calidade da validación. De todas as cancións do [dataset](#), as seguintes presentaron algunha incidencia reseñable na calidade da separación:

- **Pirimpimpim:** A partir do minuto 1:45, durante os coros cantados, os instrumentos da gaita e do acordeón parecen soar distorsionados, volvendo á normalidade a partir do minuto 2:00.
- **A fonte:** A instrumental escóitase perfectamente sen perdas con respecto á pista orixinal, porén, a parte vocal dos coros non foi completamente subtraída e pódese apreciar a moi baixo volume en certas partes.

#### Rendemento computacional

En canto ao rendemento computacional, esta é claramente a ferramenta do proxecto máis beneficiada polo uso de recursos [GPU](#). Polo tanto as probas de rendemento faranse tan só no apartado da separación. As probas amosan que o tempo aforrado en comparación co procesado en [CPU](#) ascenden ao orde de minutos se a canción é longa. A táboa [6.2](#) mostra os

| Canción            | Duración | Tempo GPU | Tempo CPU | $\bar{\Delta}t$ |
|--------------------|----------|-----------|-----------|-----------------|
| Ocre               | 2:03     | 0:15 min  | 0:53 min  | 38 s            |
| Tempestades de sal | 3:53     | 0:23 min  | 1:05 min  | 42 s            |
| Someone Like You   | 4:45     | 0:29 min  | 1:22 min  | 53 s            |
| La favorita        | 2:57     | 0:17 min  | 0:49 min  | 32 s            |
| The nights         | 3:10     | 0:20 min  | 1:06 min  | 46 s            |

Táboa 6.2: Tempos reais de procesamento de *Demucs* obtidos dos logs dos contedores

tempos reais de separación de *stems* para cinco cancións específicas do *dataset*, comparando o rendemento con GPU (RTX 3050) versus CPU (AMD Ryzen 7 5800H).

Esta eficiencia pode apreciarse na interface web grazas á monitorización do progreso. Aínda que é unha comprobación indirecta a partir dos *logs* dos contedores en tempo real, que mostran a tardanza de cada proceso de xeito máis preciso. Os resultados foron cronometrados manualmente polo desenvolvedor. Cómpre aclarar que o sistema non permite executar con procesamento CPU de xeito opcional, simplemente salta a este modo automaticamente cando se produce algún tipo de erro co procesamento GPU. Porén, ningunha canción do *dataset* completo produce erros co procesamento gráfico en *Demucs*, polo tanto para realizar esta proba foi preciso modificar manualmente o código fonte orixinal.

Os tempos e o uso de memoria GPU varían segundo a configuración de *segment\_size*, parámetro que se axusta automaticamente segundo a memoria gráfica dispoñible. O *dataset* de cancións é aleatorio e non ten en conta a complexidade computacional de cada canción. Pero se pode calcular o tempo que lle leva a maiores ao procesamento CPU:

$$\bar{\Delta}t = \frac{1}{n} \sum_{i=1}^n (t_{CPU_i} - t_{GPU_i}) = 42.2 \text{ segundos} \quad (6.1)$$

De media, o procesamento con CPU tarda aproximadamente 42 segundos máis que o procesamento con GPU: unha mellora do 150% (3 veces máis rápido na GPU).

### Robustez ante diferentes xéneros musicais

Como se pode ver na táboa 6.1, o *dataset* inclúe diversos xéneros musicais. No que corresponde a este apartado, a diferenza de separación de *stems* en canto ao xénero apenas varía. Realmente a diferenza entre a eficiencia en diferentes casos non depende realmente demasiado do xénero musical polo que se puido comprobar, senón que depende de algún instrumento concreto. Aínda que xénero e instrumento adoiten ir da man, tras varias probas non se determina ningún xénero concreto que funcione peor. Básicamente o rendemento varía dependendo dun instrumento e un momento determinados, tal e como se ve na sección anterior

| Modelo           | WER Español (%) | WER Inglés (%) | WER Galego (%) |
|------------------|-----------------|----------------|----------------|
| Whisper small    | 10.0            | 6.4            | 31.1           |
| Whisper medium   | 5.2             | 3.4            | 24.4           |
| Whisper large v2 | 3.4             | 2.2            | 20.5           |

Táboa 6.3: Comparación entre diferentes modelos de Whisper no modo automático

coas cancións que se probaron que supuxeron problemas. Por suposto ás veces hai baixóns de rendemento en cancións con moito ruído, como se describe na documentación oficial [4].

### 6.2.2 Transcrición e aliñamento temporal

Nesta sección avalíanse tanto as capacidades de transcrición automática como o aliñamento forzado. Realmente a utilidade do sistema radica no modo con letras manuais, polo que as probas para o modo automático non son exhaustivas, xa que se engadiu ao pipeline como unha opción secundaria.

#### Comparación entre modelos de Whisper na transcrición

A calidade dos sistemas deste tipo avalíase mediante o Word Error Rate (**WER**), que se calcula comparando a transcrición de referencia coa transcrición xerada polo sistema:

$$WER = \frac{S + D + I}{N} \times 100\% \quad (6.2)$$

onde  $S$  representa o número de substitucións (palabras incorrectas),  $D$  o número de palabras omitidas,  $I$  o número de palabras engadidas incorrectamente e  $N$  o número total de palabras na transcrición de referencia. **WER** baixos indican mellor rendemento [54].

A sincronización temporal non vai ser avaliada no modo automático xa que funciona axeitadamente en todos os casos. As probas fixéronse con 10 cancións (aleatorias) do [dataset](#), 4 en galego e 3 en español e inglés, o que supón un total de 30 execucións. Na táboa 6.3 amósase a media aritmética dos **WER** para os tres idiomas e tres modelos de Whisper. Os resultados do **WER** de cada canción pódense ver a táboa B.11 do apéndice B. Pero a decisión de usar cada modelo depende do usuario. Lóxicamente modelos máis potentes requiren máis capacidades computacionais e tempo de procesamento.

#### Soporte multilingüe

Como está documentado na páxina oficial [8], Whisper está entrenado en multitude de idiomas. Porén, para o idioma principal para o que está pensado este traballo, o galego, WhisperX non ten modelos de aliñamento como si os ten para o español ou o inglés, polo que usa



o modelo español como *fallback* para o galego. O sistema implementa *scripts* e algoritmos para combatir esta desventaxa. Con isto o sistema ten un amplo soporte multilingüe. As probas amosan claramente que o galego funciona peor. Neste capítulo faranse probas con tres idiomas: español, inglés e galego, aínda que o sistema funciona en calquera outro.

### ***Forced alignment con letras manuais***

Nesta sección non se avaliará a precisión da transcripción xa que ao proporcionar manualmente as letras, os subtítulos saen sempre cun 100% de precisión.

Os algoritmos de suavizado e *buffers* de anticipación implementados reducen as desviacións temporais, e ademais, en caso de producirse algunha desincronización, o sistema volve a axustarse intelixentemente nalgún punto posterior, polo que a desincronización non é acumulativa. Pero como sempre é posible detectar erros, na seguinte lista móstranse as incidencias recollidas na sincronización de 8 cancións do [dataset](#) elixidas aleatoriamente:

- **Ocre:** Sincronización perfecta.
- **Varre Vasoira:** Desincronización xeral. Neste caso illado a sincronización é desastrosa e aínda non se coñece o motivo. Todas as letras saen de corrido ao principio do vídeo sen ningún tipo de pauta. Saen dende o segundo 0 ata o minuto 1:30, malia que a canción dure 3:28 min. Probablemente, a razón sexa algún tipo de coro de fondo que non sae reflexado nas líricas, e o sistema o detecte e se desestabilice.
- **Te regalo:** Sincronización perfecta.
- **En fin, que máis dá:** Pequena desincronización nalgúns partes debido aos coros e a pausas longas de instrumental. Para este caso realizouse unha proba para entender o que ocorre dependendo de cómo o usuario proporcione as letras.  
Fíxeronse dúas execucións: unha coa letra principal e os coros e outra só coa letra principal. Comprobouse que o sistema se atasca na parte dos coros, xa que non ten referencias claras, pero volve a axustarse en canto rematan e soa a letra principal.  
Polo tanto, é mellor pasarlle só a letra principal sen coros e que o sistema se ocupe de axustarse automaticamente. Deste xeito os coros non saen como subtítulos de xeito desincronizado, senón que os subtítulos desaparecen nos intres nos que soan os coros.
- **Sweet Child O' Mine:** Sincronización perfecta.
- **A fonte:** Pequeno desaxuste por mor dos coros. Desviación a partir do 0:53 e axústase outra vez no minuto 1:00. Despois prodúcese outra desviación no 2:12, reaxustándose no 2:19. Isto supón unha desviación total de 14 segundos.

| Tipo de Canción                     | Falantes | Detectados | Observacións                       |
|-------------------------------------|----------|------------|------------------------------------|
| Ocre (voz única)                    | 1        | 1          | Detección perfecta                 |
| Te Regalo (dueto claro)             | 2        | 2          | Detección perfecta                 |
| Resistiré (dueto con solapamento)   | -        | -          | Erro en procesamento (diarización) |
| Terra (solapamento)                 | 3        | 1          | Sobredetección común               |
| Pirimpimpim (coros con solapamento) | coros    | 1          | Moi complexo                       |

Táboa 6.4: Precisión da detección de falantes

- **Tempestades de sal:** Sincronización perfecta.
- **Como che Quero Nena:** Sincronización perfecta.

### 6.2.3 *Speaker diarization*

Neste apartado a validación centrouse en cancións con diversidade de cantantes. Probáronse varias cancións con múltiples voces para avaliar a precisión da detección de falantes. A táboa 6.4 mostra os resultados: a diarización funciona perfectamente se as voces cantan por separado, e presenta dificultades cando se solapan, neste caso marcando como se fose sempre o falante 1.

### 6.2.4 **Renderizado de vídeo e sincronización**

O sistema de renderizado é responsable de crear o vídeo final, combinando resaltado silábico e calidade visual.

#### **Silabeado automático con pyphen**

O silabeador funciona en todos os casos (nos idiomas nos que foi testado), non hai palabra que apareza nos subtítulos (independentemente de se está ben transcrita ou non) que non esté dividida en sílabas correctamente. Quizáis se presenten limitacións en palabras compostas ou neoloxismos, pero non se deu o caso.

En canto á comparación de comodidade visual respecto ao resaltado a nivel de palabra, únicamente se pode dar unha opinión subxectiva. A realidade é que o resultado final vese moito máis fluido co resaltado silábico, mellorando a experiencia de usuario.

#### **Calidade visual do vídeo final**

As características do vídeo final saen comentadas na sección 4.4.2. O vídeo de fondo é o do vídeo de procedencia, lixeiramente escurecido e con calidade reducida para diminuír o tempo de procesamento. Os subtítulos vense cómodamente, aparecen sobre un fondo negro para

| Canción                         | Desc./Norm. | Separación | Transcripción | Renderizado | Total    |
|---------------------------------|-------------|------------|---------------|-------------|----------|
| Te regalo                       | 18 s        | 52s        | 1:48 min      | 3:38 min    | 6:36 min |
| Don't Stop Me Now               | 21s         | 1:01 min   | 1:44 min      | 4:03 min    | 7:09 min |
| The Nights                      | 23s         | 47s        | 2:33 min      | 3:09 min    | 6:52 min |
| Como Che Quero Nena             | 16s         | 30s        | 2:18 min      | 2:58 min    | 6:02 min |
| La vereda de la puerta de atrás | 17s         | 27s        | 1:02 min      | 3:41 min    | 5:27 min |

Táboa 6.5: Tempos detallados do *pipeline* automático por canción

contrastar co vídeo de fondo, e o resaltado faise cunha cor amarilla que difire axeitadamente do branco das letras. A composición multicapa de MoviePy funciona eficientemente, mantendo a sincronización entre o vídeo de fondo e os subtítulos animados.

A previsualización da seguinte liña mellora a experiencia de usuario, permitindo prepararse para as próximas palabras. A outra opción era facer aparecer a letra uns segundos antes de que se cante, pero optouse por previsualizar a seguinte liña. A razón é que se o usuario non coñece a letra da canción, o cerebro necesita procesar a frase uns milisegundos antes para poder cantala ao mesmo tempo que no vídeo.

As probas de compatibilidade realizáronse en varios dispositivos e navegadores:

- Navegadores web modernos (Chrome, Firefox, Safari, Edge): funciona correctamente en todos os navegadores probados.
- Dispositivos móbiles: o reprodutor non funciona correctamente. Non se pode visualizar o vídeo mediante o reprodutor integrado pero sí descargalo no dispositivo.
- Reproductores multimedia estándar: ás veces o reprodutor do Windows Media Player non é compatible cos códecs e non deixa visualizar. En VLC funciona sempre.

## 6.3 Probas de integración do sistema

Nesta sección compróbase o funcionamento de todo o sistema, probando os *pipelines* completos dende a entrada ata a saída final, e o seu comportamento ante situacións de erro e/ou con carga de traballo.

### 6.3.1 Pipeline completo automático

Realizáronse probas con cinco cancións representativas do *dataset* para medir tempos específicos de cada fase do procesamento. Na táboa 6.5 móstranse os resultados. O tempo medio total con cancións promedio oscila entre 4 e 7 minutos. A fase que máis tempo con-

| Canción             | Desc./Norm. | Separación | Aliñamento | Renderizado | Total    |
|---------------------|-------------|------------|------------|-------------|----------|
| En fin, que máis dá | 14s         | 25s        | 14s        | 5:09 min    | 6:03 min |
| Resistire           | 16s         | 26s        | 22s        | 6:04 min    | 7:09 min |
| Tempestades de sal  | 17s         | 30s        | 1:48 min   | 6:05 min    | 8:40 min |
| Terra               | 20s         | 23s        | 1:12 min   | 3:54 min    | 5:50 min |
| Slalom              | 18s         | 41s        | 1:51 min   | 5:17 min    | 8:08 min |

Táboa 6.6: Tempos detallados do *pipeline* con letras manuais por canción

sume depende de varios factores, pero os máis relevantes son a separación por mor dunha instrumental complexa e a transcripción e aliñamento por unha letra complexa.

Como era de esperar, os tempos varían segundo a duración da canción e a complexidade do audio. Cancións máis longas requiren máis tempo en todas as fases, podendo dar lugar incluso a erros de procesamento. Isto obviamente supón un problema, pero o sistema está deseñado para funcionar tamén en equipos máis potentes, nos que sí sería posible acadar o procesamento destas cancións.

### 6.3.2 Pipeline con letras manuais

O modo manual elimina a fase de transcripción automática substituíndoa polo aliñamento forzado con WhisperX, polo que a calidade do aliñamento é significativamente superior. Tamén é certo que o traballo centrouse sobre todo nesta modalidade. A fase máis problemática é o aliñamento con frases complexas nun idioma no que WhisperX non teña modelo de aliñación.

Fixéronse probas con cinco cancións que presentaron dificultades na transcripción automática, especialmente cancións en galego e casos con ruído. O tempo de procesamento varía aínda que se procese a mesma canción nas mesmas condicións. Depende de factores como: a velocidade de internet (descarga de Youtube), da letra (si a letra é complexa e desordenada requerirá tempo de normalización), se o equipo está conectado á corrente, etc. En xeral, o modo manual é algo máis rápido que o automático, excepto en cancións con letras difíciles de aliñar.

### 6.3.3 Xestión de erros e recuperación

No procesamento poden ocorrer diversidade de erros: GPU, memoria, códecs, etc. O sistema implementa algoritmos de *fallbacks* para que se recupere automaticamente sin que o usuario teña que xestionar nada. Isto por suposto implica máis tempo de procesamento. Dediciuse non informar ao usuario no caso de que o sistema se recupere dun erro.

A interface de usuario conta con cancelación de tarefas e *logs* detallados para manter

informado ao usuario de todo o que ocorre no proceso. Se finalmente o procesamento acaba non sendo posible, a web informa axeitadamente do erro producido ao usuario, e pode volver a intentalo ou probar outra canción sen problemas na mesma sesión. Para máis información pódese instalar o entorno e observar os *logs* de cada contedor.

A continuación, móstrase a lista coas excepcións máis importantes<sup>1</sup>:

- **Introduza unha URL de YouTube ou sube un arquivo MP4:** Validación cando non se proporciona *input*.
- **Introduce unha URL válida:** Cando a URL de YouTube non é válida.
- **Erro descoñecido:** Erro non especificado desde o *backend*.
- **Erro en procesamento:** Cando a tarefa falla durante o procesamento.
- **Procesamento cancelado:** Cando o usuario cancela a tarefa.
- **Erro de comunicación:** Erro de conexión co servidor.

#### 6.3.4 Rendemento con múltiples usuarios

O sistema está deseñado para soportar múltiples usuarios concorrentes. Para realizar as probas ábreanse varias sesións ngrok dende diferentes dispositivos. Porén, para o equipo no que se están realizando as probas, é inviable a carga computacional soportada por varios procesos concorrentes. O máximo que o equipo persoal foi quen de soportar foron 2 cancións de corta duración e pouca complexidade dende 2 equipos diferentes (sempre respetando o *rate limiting* comentado na sección 5.6.3). Isto non é suficiente para avaliar a capacidade baixo carga, pero polo menos permitiu comprobar o correcto funcionamento da arquitectura con Celery e Redis, a cal xestiona correctamente as colas de tarefas, garantindo que ningunha solicitude se perde aínda que o tempo de resposta aumente. En resumidas contas, para avaliar o sistema baixo carga é preciso contar cun equipo máis potente.

## 6.4 Validación en casos de uso reais

Esta sección presenta a validación do sistema en escenarios reais de uso. As probas realizáronse con música de diferentes xéneros, idiomas e características técnicas.

Segundo unha valoración persoal tras decenas de execucións, conclúese que o sistema non produce diferenzas de calidade segundo o xénero musical (pop, rock, rap, música tradicional galega, electrónica, ...), senón que máis ben a diferenza radica nas características especiais de cada canción, como pode ser o ruído, os cambios de ritmo ou as pausas instrumentais. Quizáis

<sup>1</sup> Móstranse máis excepcións, pero nesta memoria só se documentan as máis comúns para abreviar.

| Sistema                 | La Favorita                                                                                                           | Tempestades de sal                                                                        |
|-------------------------|-----------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------|
| <b>Sistema proposto</b> | Sincronización aceptable nos primeiros 1:30 minutos, despois desacompásase lixeiramente por mor do "seseo" e "gheada" | Aliñamento preciso durante toda a canción                                                 |
| <b>Youka</b>            | Saída incorrecta. Produce vídeo sen subtítulos.                                                                       | Sincronización correcta con lixeiras desviacións                                          |
| <b>UltraStar</b>        | Require moita intervención manual. Sincronización moi imprecisa e silabeado incorrecto                                | Proceso pouco automatizado, transcripción correcta e sincronización lixeiramente desviada |

Táboa 6.7: Comparación do sistema con ferramentas existentes para cancións en galego

a diferenza entre o xénero musical se note máis na parte da separación vocal e instrumental, pero non na parte de transcripción nin aliñamento.

- **Múltiples voces e coros:** Presentan dificultades para a transcripción, o aliñamento e a diarización de falantes. Na parte de separación, coas cancións que se probaron, non se toparon dificultades neste caso.
- **Cambios de ritmo e tempo:** Cancións con cambios de tempo probaron a robustez do aliñamento temporal. O sistema mantén unha precisión aceptable en cambios graduais pero ten dificultades con cambios abruptos superiores ao 20% do tempo orixinal. E sobre todo presenta dificultades ante períodos longos de sós de instrumental.
- **Audio con ruído de fondo:** Probáronse varias versións das mesmas cancións pero ruído engadido, ou con grabacións da mesma canción pero de peor calidade. Os resultados amosaron que introducindo ruído lixeiro o WER aumenta aproximadamente nun 5% e introducindo ruído alto aumenta aproximadamente nun 15%.

De cara ao aliñamento e a separación, malia que non se pode medir con métricas obxectivas, nótase o empeoramento destas fases introducindo ruído, tanto na imprecisión da transcripción como no desfase da aliñación.

- **Gravacións en vivo:** Os resultados con audio en directo son semellantes ao punto anterior, xa que a debilidade destas gravacións é efectivamente o ruído de fondo: audiencia, reverberación da sala, etc.

## 6.5 Análise comparativo

Nesta sección compárase o sistema con outros comerciais e de código aberto.

Partindo da base que o karaoke perfecto sexa aquel feito manualmente con edición de vídeo (letra e sincronización perfectas), os resultados que ofrece o sistema proposto son com-

parables con respecto a solucións existentes. Na táboa 2.3 da sección 2.5 xa se fala da comparativa entre os sistemas de karaoke máis famosos. Estes karaokes comerciais son mellores en canto a estética e sincronización. Podemos diferenciar dous tipos diferentes: os que teñen unha base de datos con un número limitado de cancións, e os que xeran a canción de xeito automático con IA. Estes últimos teñen a desvantaxe de funcionar mal en galego, cousa que pretende solucionar o sistema proposto.

A táboa 6.7 amosa un pequeno resumo da comparación entre o noso sistema e as dúas mellores ferramentas no contexto do estado da arte actual: Youka e UltraStar. As cancións avaliadas son **La Favorita** e **Tempestades de sal**.

A valoración da experiencia de usuario pasa a ser un tema subxectivo. A interface, comparada coa das alternativas coñecidas, é cómoda e sinxela. Cada sistema ten os seus apartados e opcións ao igual que a do sistema proposto. Os casos de uso de todos os sistemas son semellantes, salvo aqueles que teñen unha lista limitada de cancións, nos que o usuario ten que buscar a canción. A velocidade da xeración de contido tamén é semellante.

Como novidade, comentar que o noso proxecto permite xogar coas pistas instrumental e vocal, cousa que non é útil para o usuario promedio, pero sí para o desenvolvedor para comprobar a calidade da sincronización temporal.

## 6.6 Limitacións identificadas

Esta sección documenta as limitacións inherentes ao sistema que afectan o seu rendemento en certos escenarios. En xeral, o sistema require recursos computacionais significativos que limitan a súa escalabilidade. Aínda que WhisperX soporta múltiples idiomas, existen diferencas significativas no rendemento segundo o idioma de procesamento. Prodúcese problemas sobre todo en variantes dialécticas do galego, coloquialmente coñecidas como o "seseo" e a "gheada". Por último, se a parte vocal vai moi rápido, prodúcese un efecto "pouco suave" ao final de cada oración, cambiando de frase demasiado bruscamente.

# Conclusións e traballo futuro

---

NESTE capítulo da memoria vanse presentar as conclusións ás que se chegou unha vez o proxecto estivo rematado e as posibles liñas futuras do traballo.

## 7.1 Conclusións

Tras completar o proxecto pódese afirmar que se acadaron os obxectivos iniciais, creando un sistema que xera vídeos de karaoke de forma automática mediante técnicas de intelixencia artificial. Finalmente puido desenvolverse unha arquitectura de microservizos que presenta múltiples propiedades. A separación de responsabilidades implementouse mediante cinco contedores independentes: Flask/[Demucs](#) para interface e separación de fontes, [WhisperX](#) para transcripción e aliñamento, Celery Worker para procesamento asíncrono, Redis para xestión de colas e [Ngrok](#) para acceso externo. Existen decenas de funcionalidades que foron implementadas e documentadas ao longo deste documento e funcionan conxuntamente nun mesmo sistema. O desacoplamento acadouse a través de protocolos estándar e volumes compartidos Docker, permitindo substituír calquera compoñente sen afectar o resto.

Durante o desenvolvemento puideron aplicarse os coñecementos adquiridos ao longo da carreira, ademais de explorar novas tecnoloxías no campo da [IA](#) aplicadas a audio e vídeo. A realización deste proxecto proporcionou unha mellor comprensión sobre as capacidades e limitacións das tecnoloxías de [IA](#). A experiencia coas limitacións dos modelos ensinou a importancia de non depender cegamente das solucións de [IA](#), senón de entender as súas vantaxes e desvantaxes para crear solucións adaptadas ás necesidades de cada problema. Foi nese punto onde xurdiu o desenvolvemento das estratexias híbridas que propón este traballo.

Dos 11 requisitos funcionais establecidos na sección [3.2](#), o sistema cumpre con 10 deles, mentres que o requisito número 4 (sincronización temporal) cúmprese parcialmente, xa que non é posible asegurar a calidade da aliñación en absolutamente todas as cancións. O [dataset](#) está composto por cancións estratexicamente elixidas para a avaliación, o que supón que



conteñen características técnicas complexas para probar a fondo as capacidades do sistema. Porén, tamén se probou con decenas de cancións de xeito aleatorio e pódese concluír que o sistema ten unha efectividade arredor do 80%.

En canto aos requisitos non funcionais, todos se cumpren axeitadamente a excepción do requisito número 2 (tempo de procesado). O portátil empregado para as probas necesita entre tres e cinco minutos para procesar unha canción, pero hai casos excepcionais onde o tempo pode superar os sete minutos. Todo o código desenvolto está dispoñible de forma pública no repositorio: <https://github.com/gaelfernandez1/KaraokeProject> [41].

Implementar algoritmos como a distribución proporcional para letras manuais ou a diarización de falantes foi todo un desafío. Concretamente, a parte que resultou ser máis complexa foi o sistema de renderizado multicapa con resaltado progresivo, porque supuxo resolver infinidad de *bugs* que ían xurdindo conforme avanzaba o proxecto. É preciso coordinar múltiples procesos simultáneos que deben executarse con moita precisión para conseguir o efecto visual desexado.

As probas realizadas cun *dataset* de 18 cancións en tres idiomas (galego, español e inglés) demostraron a eficacia do sistema. En separación de fontes, *Demucs* mostrou un rendemento excelente con só 2 de 18 cancións presentando incidencias menores de distorsión en instrumentos específicos. O rendemento computacional amosou unha mellora significativa: o procesamento con *GPU* foi 3 veces máis rápido que con *CPU* (20 segundos de media en *GPU* fronte a 56 segundos e *CPU* en cancións de 3 minutos). En transcripción automática os modelos Whisper demostraron diferentes niveis de precisión: Whisper large v2 acadou un WER do 2.2% en inglés, 3.4% en español e 20.5% en galego; Whisper medium obtivo 3.4% en inglés, 5.2% en español e 24.4% en galego. O aliñamento forzado con letras manuais mostrou unha sincronización temporal axeitada na maioría das cancións probadas, con certas desviacións en casos complexos con coros e dependendo da calidade da letra que se proporciona.

Os resultados obtidos sitúan o sistema nun bo nivel competitivo respecto a solucións comerciais, especialmente considerando que é o único que ofrece soporte nativo para galego. Ademais, a flexibilidade para aceptar varios formatos de entrada e a súa interface web intuitiva fan que non se requiran coñecementos técnicos avanzados nin intervención manual esaxerada. Por suposto a solución está lonxe de ser perfecta, xa que como se puido comprobar facendo as probas exhaustivas, a diversidade musical actual é inmensa e aínda aparecen problemas con certas cancións con complexidade técnica, pero este traballo senta unha base sólida para futuras investigacións.

## 7.2 Traballo futuro

Aínda que os resultados obtidos son positivos, quedan varias áreas que se poden mellorar. Unha das principais liñas de traballo futuro sería ampliar o sistema con soporte para dialectos. Podería dar bos resultados o *fine-tuning* específico dos modelos de transcripción e aliñamento en galego. Agora mesmo, os modelos que se usan (Whisper e WhisperX) están entrenados principalmente con datos en inglés e outras linguas maioritarias, o que pode facer que funcionen peor co galego, especialmente en música onde a pronunciación pode ser diferente do galego falado *standard*. Facendo un *fine-tuning* con *dataset* específicos de cancións en galego, podería mellorarse bastante a precisión da transcripción automática e, polo tanto, a calidade final dos vídeos de karaoke que se xeran.

En canto ás técnicas de *IA* que se usan, poderíanse probar outras arquitecturas de modelos para ver se dan mellores resultados ou funcionan mellor que as actuais. Por exemplo, sería interesante investigar modelos *transformer* máis avanzados para a transcripción automática, ou explorar técnicas de *deep learning* para o renderizado de subtítulos.

Para conseguir mellores métricas e resultados, outra liña de investigación sería desenvolver algoritmos máis avanzados de detección de pausas e segmentación musical. O sistema actual podería beneficiarse de técnicas de análise musical que detecten automaticamente estruturas como estrofas ou estribillos, permitindo unha segmentación máis intelixente dos contidos.

Outro aspecto a ter en conta en futuros traballos sería mellorar a interface de usuario engadindo funcionalidades como o seguimento da evolución da calidade de voz do usuario ao longo do tempo, ou a posibilidade de personalizar os estilos visuais dos subtítulos segundo as preferencias de cada un. E sobre todo, para automatizar por completo o proceso, considerar a posibilidade de realizar *scraping* nas webs de líricas e implementando un buscador integrado na web, xestionando todos os problemas legais que iso podería levar.

A longo prazo, a idea é que esta ferramenta non só sirva para crear karaoke caseiro, senón que tamén sexa útil para profesionais da música, facilitando a creación de contidos para espectáculos, ensaios ou fins educativos. Para dito fin é importante seguir mellorando os sistemas de *IA* que se teñen actualmente. Con tantas liñas de investigación por abrir, pódese ver o potencial que ten este traballo. Deixa moitas preguntas novas, e outras que se exploraron aquí con moito máis recorrido.

# **Apéndices**

# Código adicional

---

## A.1 Detección automática de GPU

Na sección 4.2.1 comentouse que o sistema inclúe una lóxica para detectar automáticamente as capacidades GPU de cada equipo. Aquí está o código que o implementa. [A.1](#)

```
1 def detect_gpu_capability():
2     try:
3         if torch.cuda.is_available():
4             device_count = torch.cuda.device_count()
5             current_device = torch.cuda.current_device()
6             gpu_name = torch.cuda.get_device_name(current_device)
7             gpu_memory = torch.cuda.get_device_properties(
current_device).total_memory / (1024**3)
8             return {
9                 'has_cuda': True,
10                'device_count': device_count,
11                'gpu_name': gpu_name,
12                'gpu_memory': gpu_memory,
13                'recommended_device': 'cuda'
14            }
15     except Exception as e:
16         return {
17             'has_cuda': False,
18             'recommended_device': 'cpu',
19             'error': str(e)
20         }
```

Código A.1: Detección automática de GPU

## A.2 *Endpoint principal de WhisperX*

O código A.2 que se mostra a continuación amosa o *endpoint* principal configurado de WhisperX tal e como se comenta na sección 5.1.4

```
1 @app.route('/align', methods=['POST'])
2 def align_audio():
3     data = request.get_json()
4     audio_path = data.get('audio_path')
5     manual_lyrics = data.get('manual_lyrics')
6     enable_diarization = data.get('enable_diarization', False)
7     hf_token = data.get('hf_token')
8
9     # Procesamento con WhisperX
10    result = process_whisperx_alignment(
11        audio_path, manual_lyrics, enable_diarization, hf_token
12    )
13
14    return jsonify(result)
```

Código A.2: Endpoint principal de WhisperX

## A.3 Algoritmo de progreso temporal híbrido

O sistema divide cada palabra en sílabas e resalta cada unha individualmente tal e como se menciona na sección 5.4.1. O listado A.3 amosa cómo se calcula cantas sílabas resaltar baseándose en diversos factores.

## A.4 Función xeradora de clips de texto con silabificación

Na sección 5.4.2 coméntase que o sistema contén un sistema de renderizado *per-frame* que calcula o progreso e aplica os efectos visuais en tempo real. Pois o listado A.4 mostra cómo se implementa.

## A.5 Estrutura de datos para silabificación

Como se menciona na sección 5.4.1 a silabificación require un procesamento adicional para mapear correctamente as sílabas cos tempos de audio, e móstrase cómo se fai no código A.5

```

1 # Cálculo do progreso temporal
2 if duracion_total_liña > 0:
3     progreso_temporal = min(1.0, max(0.0, t_offset /
4     duracion_total_liña))
5     segmentos_pasados = 0
6     bonus_progreso_segmento = 0.0
7     # Buffer para anticipar a última sílaba
8     buffer_anticipacion = BUFFER_ANTICIPACION
9
10    for i, seg in enumerate(line_info["words"]):
11        tiempo_fin_efectivo = seg["end"]
12        if i == len(line_info["words"]) - 1:
13            tiempo_fin_efectivo = max(seg["start"]+0.1,
14                                     seg["end"]-buffer_anticipacion)
15
16        if tempoActual >= tiempo_fin_efectivo:
17            segmentos_pasados += 1
18        elif tempoActual >= seg["start"]:
19            duracion_seg = tiempo_fin_efectivo - seg["start"]
20            if duracion_seg > 0:
21                progreso_seg = (tempoActual - seg["start"]) /
22                duracion_seg
23                progreso_seg = min(1.0, progreso_seg * 1.2)
24                bonus_progreso_segmento = progreso_seg /
25                len(line_info["words"])
26                break
27
28        progreso_segmento = (segmentos_pasados /
29        len(line_info["words"]))
30        if line_info["words"] else 0.0
31        progreso_final = min(1.0, progreso_segmento +
32        bonus_progreso_segmento)
33
34    # Combinación ponderada dos progresos
35    progreso_combinado = (progreso_temporal *
36    PESO_PROGRESO_TEMPORAL) +
37    (progreso_final * PESO_PROGRESO_SEGMENTO)

```

Código A.3: Algoritmo de progreso temporal híbrido

```

1 def create_karaoke_text_clip(line_info: dict, next_line_info: dict
  = None, advance: float=0.5, duration_padding: float=0.5):
2
3     duracion_liña = line_info["end"] - line_info["start"]
4     duracion_clip = duracion_liña + advance + duration_padding
5     offset_visualizacion = advance # Retraso interno do clip
6
7     def facer_frame(t):
8         # t é o tempo transcorrido no clip
9         t_efectivo = max(t - offset_visualizacion, 0)
10
11         # Renderizado da liña actual con progreso silábico
12         frameActual = render_line_image(line_info, t_efectivo)
13
14         if MOSTRAR_LIÑA_SEGUINTE and next_line_info:
15             frame_seguinte = render_next_line_image(next_line_info)
16
17             # Composición manual de arrays NumPy
18             altura1 = frameActual.shape[0]
19             altura2 = frame_seguinte.shape[0]
20             ancho = max(frameActual.shape[1],
21 frame_seguinte.shape[1])
22
23             altura_combinada = altura1 + ESPACIADO_LIÑAS + altura2
24             frame_combinado = np.zeros((altura_combinada, ancho,
25 3), dtype=np.uint8)
26
27             # Cálculos de posicionamento píxel-perfect
28             offset_x1 = (ancho - frameActual.shape[1]) // 2
29             offset_x2 = (ancho - frame_seguinte.shape[1]) // 2
30
31             # Asignación manual de píxeles
32             frame_combinado[:altura1,
33 offset_x1:offset_x1+frameActual.shape[1]] = frameActual
34
35             y_seguinte_liña = altura1 + ESPACIADO_LIÑAS
36             frame_combinado[y_seguinte_liña:y_seguinte_liña+altura2,
37 offset_x2:offset_x2+frame_seguinte.shape[1]] = frame_seguinte
38             return frame_combinado
39             return frameActual
40
41     clip_texto = VideoClip(facer_frame, duration=duracion_clip)
42     return clip_texto

```

Código A.4: Función xeradora de clips de texto con sincronización silábica

```

1 info_todas_silabas = [] # Lista de [silaba , es_final_de_palabra]
2
3 for palabra in texto_completo.split():
4     silabas_palabra = dic_pyphen.inserted(palabra).split('-')
5
6     if len(silabas_palabra) <= 1: # Palabra non divisible
7         info_todas_silabas.append((palabra , True))
8     else:
9         # Palabra divisible en silabas
10        for i, sil in enumerate(silabas_palabra):
11            e_ultima_silaba = (i == len(silabas_palabra) - 1)
12            info_todas_silabas.append((sil , e_ultima_silaba))
13
14 # Cálculo de sílabas a resaltar
15 total_silabas = len(info_todas_silabas)
16 silabas_para_resaltar = min(total_silabas ,
17                             round(progreso_combinado *
18                                 total_silabas))

```

Código A.5: Estrutura de datos para silabificación

```

1 1
2 00:00:12,450 --> 00:00:13,210
3 [SPEAKER_00] Esta é a primeira liña
4
5 2
6 00:00:13,210 --> 00:00:15,800
7 [SPEAKER_01] Segunda liña con outro falante

```

Código A.6: Formato SRT con speaker diarization

## A.6 Formato SRT con *speaker diarization*

Na sección 4.4.2 comentábase sobre o [SRT](#) que o sistema creaba, que despois iba ser usado polo renderizado para coñecer os tempos exactos de cada sílaba. O código [A.6](#) mostra cómo e o formato do arquivo [SRT](#) xerado.

## A.7 Configuración GPU para contedor principal

O listado [A.7](#) mostra a configuración [GPU](#) deseñada para o contedor principal tal e como se menciona na sección [5.1.2](#)

## A.8 Coordinación de procesos en `celery_tasks.py`

Celery é o encargado coordinación de procesos no sistema. A sección [5.5.2](#) fala do procesamento asíncrono, e o código [A.8](#) mostra parte da implementación de dita coordinación.



```
1 deploy:
2     resources:
3         reservations:
4             devices:
5                 - driver: nvidia
6                   count: all
7                   capabilities: [gpu]
8 environment:
9     - NVIDIA_VISIBLE_DEVICES=all
10    - NVIDIA_DRIVER_CAPABILITIES=compute, utility
```

Código A.7: Configuración GPU para contenedor principal

```
1 def create_with_cancellation_check(video_path ,
2     enable_diarization=False ,
3                                     hf_token=None ,
4     source_type="upload" ,
5                                     source_url=None, save_to_db=True):
6     check_if_cancelled() # Verificación antes de cada etapa crítica
7
8     # Cada etapa puede ser cancelada independientemente
9     return create(video_path, enable_diarization, hf_token,
10                   source_type, source_url, save_to_db)
```

Código A.8: Coordinación de procesos en *celery\_tasks.py*

## Táboas adicionais

### B.1 Casos de uso

A continuación vanse presentar neste apéndice todos os casos de uso que dos que se falou na sección 3.3.

Táboa B.1: CU-01: Xeración de karaoke con transcripción automática

| Campo                  | Descrición                                                                                                                                                                                                                                                                                          |
|------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>Identificador</i>   | CU-01                                                                                                                                                                                                                                                                                               |
| <i>Nome</i>            | Xeración de karaoke con transcripción automática                                                                                                                                                                                                                                                    |
| <i>Actores</i>         | Usuario                                                                                                                                                                                                                                                                                             |
| <i>Descrición</i>      | O usuario introduce unha URL de YouTube e o sistema xera automaticamente un vídeo de karaoke coa voz separada e subtítulos sincronizados                                                                                                                                                            |
| <i>Precondicións</i>   | URL de YouTube válida e accesible                                                                                                                                                                                                                                                                   |
| <i>Fluxo Principal</i> | <ol style="list-style-type: none"> <li>1. O usuario introduce a URL</li> <li>2. O sistema descarga o vídeo</li> <li>3. Separación de <i>stems</i> con Demucs</li> <li>4. Transcripción automática con WhisperX</li> <li>5. Renderizado de subtítulos</li> <li>6. Xeración do vídeo final</li> </ol> |
| <i>Postcondicións</i>  | Vídeo de karaoke xerado con subtítulos sincronizados                                                                                                                                                                                                                                                |
| <i>Excepcións</i>      | URL non válida, erro na descarga, fallo na transcripción                                                                                                                                                                                                                                            |

Táboa B.2: CU-02: Xeración de karaoke con letra manual

| Campo                  | Descrición                                                                                                                                                                                                                                                                                                   |
|------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>Identificador</i>   | CU-02                                                                                                                                                                                                                                                                                                        |
| <i>Nome</i>            | Xeración de karaoke con letra manual                                                                                                                                                                                                                                                                         |
| <i>Actores</i>         | Usuario                                                                                                                                                                                                                                                                                                      |
| <i>Descrición</i>      | O usuario proporciona a letra da canción manualmente xunto coa URL, permitindo maior precisión na sincronización, especialmente para idiomas minoritarios como o galego                                                                                                                                      |
| <i>Precondicións</i>   | URL de YouTube válida e letra da canción en formato texto                                                                                                                                                                                                                                                    |
| <i>Fluxo Principal</i> | <ol style="list-style-type: none"> <li>1. O usuario introduce a URL e a letra</li> <li>2. O sistema descarga o vídeo</li> <li>3. Separación de <i>stems</i> con Demucs</li> <li>4. <i>Forced alignment</i> con WhisperX</li> <li>5. Renderizado de subtítulos</li> <li>6. Xeración do vídeo final</li> </ol> |
| <i>Postcondicións</i>  | Vídeo de karaoke xerado con subtítulos precisos                                                                                                                                                                                                                                                              |
| <i>Excepcións</i>      | Letra non corresponde co audio, erro no aliñamento                                                                                                                                                                                                                                                           |

Táboa B.3: CU-03: Procesamento con arquivo MP4 local

| Campo                  | Descrición                                                                                                                                                                                                                                                   |
|------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>Identificador</i>   | CU-03                                                                                                                                                                                                                                                        |
| <i>Nome</i>            | Procesamento con arquivo MP4 local                                                                                                                                                                                                                           |
| <i>Actores</i>         | Usuario                                                                                                                                                                                                                                                      |
| <i>Descrición</i>      | O usuario carga un arquivo MP4 local para xerar o karaoke, útil para probas rápidas                                                                                                                                                                          |
| <i>Precondicións</i>   | Arquivo MP4 válido no sistema local                                                                                                                                                                                                                          |
| <i>Fluxo Principal</i> | <ol style="list-style-type: none"> <li>1. O usuario selecciona o arquivo MP4</li> <li>2. O sistema procesa</li> <li>3. Separación de <i>stems</i></li> <li>4. Transcrición/aliñamento</li> <li>5. Renderizado</li> <li>6. Xeración do vídeo final</li> </ol> |
| <i>Postcondicións</i>  | Vídeo de karaoke xerado a partir do arquivo local                                                                                                                                                                                                            |

..... (continúa na páxina seguinte) .....

Táboa B.3 – (vén da páxina anterior)

| Campo             | Descrición                                |
|-------------------|-------------------------------------------|
| <i>Excepcións</i> | Arquivo corrompido, formato non soportado |

Táboa B.4: CU-04: Xeración de pista instrumental

| Campo                  | Descrición                                                                                                                                                                                                                                                                                      |
|------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>Identificador</i>   | CU-04                                                                                                                                                                                                                                                                                           |
| <i>Nome</i>            | Xeración de pista instrumental                                                                                                                                                                                                                                                                  |
| <i>Actores</i>         | Usuario                                                                                                                                                                                                                                                                                         |
| <i>Descrición</i>      | O usuario solicita únicamente a separación de <i>stems</i> para obter a pista instrumental sen voz                                                                                                                                                                                              |
| <i>Precondicións</i>   | URL de YouTube válida ou arquivo MP4 local                                                                                                                                                                                                                                                      |
| <i>Fluxo Principal</i> | <ol style="list-style-type: none"> <li>1. O usuario introduce a fonte de audio ou vídeo</li> <li>2. O sistema procesa o arquivo</li> <li>3. Separación de <i>stems</i> con Demucs</li> <li>4. Extracción da pista instrumental</li> <li>5. Xeración do arquivo de audio instrumental</li> </ol> |
| <i>Postcondicións</i>  | Arquivo de audio instrumental sen voz                                                                                                                                                                                                                                                           |
| <i>Excepcións</i>      | Erro na separación de <i>stems</i> , calidade insuficiente                                                                                                                                                                                                                                      |

Táboa B.5: CU-05: Xestión de múltiples voces

| Campo                | Descrición                                                                                                           |
|----------------------|----------------------------------------------------------------------------------------------------------------------|
| <i>Identificador</i> | CU-05                                                                                                                |
| <i>Nome</i>          | Xestión de múltiples voces                                                                                           |
| <i>Actores</i>       | Usuario                                                                                                              |
| <i>Descrición</i>    | O sistema detecta e diferencia automaticamente múltiples voces na canción, asignando cores diferentes nos subtítulos |
| <i>Precondicións</i> | Canción con múltiples voces ou coros                                                                                 |

..... (continúa na páxina seguinte) .....

Táboa B.5 – (vén da páxina anterior)

| Campo                  | Descrición                                                                                                                                                                                                                                                        |
|------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>Fluxo Principal</i> | <ol style="list-style-type: none"> <li>1. O usuario marca a opción de múltiples <i>speakers</i></li> <li>2. O sistema realiza <i>speaker diarization</i></li> <li>3. Asigna cores diferentes a cada voz</li> <li>4. Renderiza subtítulos diferenciados</li> </ol> |
| <i>Postcondicións</i>  | Subtítulos con cores diferenciadas por voz                                                                                                                                                                                                                        |
| <i>Excepcións</i>      | Dificultade na separación de voces, solapamento excesivo                                                                                                                                                                                                          |

Táboa B.6: CU-06: Reprodución de vídeo de karaoke

| Campo                  | Descrición                                                                                                                                                                                                                                                                                      |
|------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>Identificador</i>   | CU-06                                                                                                                                                                                                                                                                                           |
| <i>Nome</i>            | Reprodución de vídeo de karaoke                                                                                                                                                                                                                                                                 |
| <i>Actores</i>         | Usuario                                                                                                                                                                                                                                                                                         |
| <i>Descrición</i>      | O usuario reproduce o vídeo de karaoke xerado co control independente de volume vocal e instrumental                                                                                                                                                                                            |
| <i>Precondicións</i>   | Vídeo de karaoke xerado exitosamente                                                                                                                                                                                                                                                            |
| <i>Fluxo Principal</i> | <ol style="list-style-type: none"> <li>1. O usuario accede ao reproductor</li> <li>2. Controla o volume vocal mediante un deslizador</li> <li>3. Controla o volume instrumental independentemente</li> <li>4. Reproduce/pausa o vídeo</li> <li>5. Visualiza subtítulos sincronizados</li> </ol> |
| <i>Postcondicións</i>  | Experiencia de karaoke personalizada                                                                                                                                                                                                                                                            |
| <i>Excepcións</i>      | Erro na carga do vídeo, problemas de sincronización                                                                                                                                                                                                                                             |

Táboa B.7: CU-07: Xestión da biblioteca de cancións

| Campo                | Descrición                        |
|----------------------|-----------------------------------|
| <i>Identificador</i> | CU-07                             |
| <i>Nome</i>          | Xestión da biblioteca de cancións |
| <i>Actores</i>       | Usuario                           |

..... (continúa na páxina seguinte) .....

Táboa B.7 – (vén da páxina anterior)

| Campo                  | Descrición                                                                                                                                                                                                                                                                     |
|------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>Descrición</i>      | O usuario visualiza, organiza e xestiona todas as cancións procesadas na biblioteca do sistema                                                                                                                                                                                 |
| <i>Precondicións</i>   | Polo menos unha canción procesada                                                                                                                                                                                                                                              |
| <i>Fluxo Principal</i> | <ol style="list-style-type: none"> <li>1. O usuario accede á biblioteca</li> <li>2. Visualiza lista de cancións procesadas</li> <li>3. Pode reproducir cancións directamente</li> <li>4. Elimina cancións non desexadas</li> <li>5. Accede a metadatos das cancións</li> </ol> |
| <i>Postcondicións</i>  | Biblioteca actualizada segundo preferencias                                                                                                                                                                                                                                    |
| <i>Excepcións</i>      | Erro ao eliminar ficheiros, biblioteca baleira                                                                                                                                                                                                                                 |

Táboa B.8: CU-08: Descarga de ficheiros xerados

| Campo                  | Descrición                                                                                                                                                                                                                                                                  |
|------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>Identificador</i>   | CU-08                                                                                                                                                                                                                                                                       |
| <i>Nome</i>            | Descarga de ficheiros xerados                                                                                                                                                                                                                                               |
| <i>Actores</i>         | Usuario                                                                                                                                                                                                                                                                     |
| <i>Descrición</i>      | O usuario pode descargar os diferentes ficheiros xerados durante o procesamento                                                                                                                                                                                             |
| <i>Precondicións</i>   | Procesamento completado exitosamente                                                                                                                                                                                                                                        |
| <i>Fluxo Principal</i> | <ol style="list-style-type: none"> <li>1. O usuario accede á biblioteca</li> <li>2. Selecciona o tipo de ficheiro (vídeo completo, instrumental, vocal, transcripción)</li> <li>3. Descarga o ficheiro seleccionado</li> <li>4. O sistema proporciona o ficheiro</li> </ol> |
| <i>Postcondicións</i>  | Ficheiro descargado no sistema local                                                                                                                                                                                                                                        |
| <i>Excepcións</i>      | Ficheiro non encontrado, erro na descarga                                                                                                                                                                                                                                   |

Táboa B.9: CU-9: Cancelación de procesamento

| Campo                | Descrición |
|----------------------|------------|
| <i>Identificador</i> | CU-9       |

..... (continúa na páxina seguinte) .....

Táboa B.9 – (vén da páxina anterior)

| Campo                  | Descrición                                                                                                                                                                                                                                                   |
|------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>Nome</i>            | Cancelación de procesamento                                                                                                                                                                                                                                  |
| <i>Actores</i>         | Usuario                                                                                                                                                                                                                                                      |
| <i>Descrición</i>      | O usuario pode cancelar un procesamento en curso                                                                                                                                                                                                             |
| <i>Precondicións</i>   | Procesamento en curso                                                                                                                                                                                                                                        |
| <i>Fluxo Principal</i> | <ol style="list-style-type: none"> <li>1. O usuario preme o botón de cancelar</li> <li>2. O sistema confirma a cancelación</li> <li>3. Detén todos os procesos asociados</li> <li>4. Limpa ficheiros temporais</li> <li>5. Notifica a cancelación</li> </ol> |
| <i>Postcondicións</i>  | Procesamento detido e recursos liberados                                                                                                                                                                                                                     |
| <i>Excepcións</i>      | Procesamento xa completado, erro na cancelación                                                                                                                                                                                                              |

Táboa B.10: CU-10: Configuración de modelo Whisper

| Campo                  | Descrición                                                                                                                                                                                                                  |
|------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>Identificador</i>   | CU-10                                                                                                                                                                                                                       |
| <i>Nome</i>            | Configuración de modelo Whisper                                                                                                                                                                                             |
| <i>Actores</i>         | Usuario                                                                                                                                                                                                                     |
| <i>Descrición</i>      | O usuario pode seleccionar diferentes modelos de WhisperX ( <i>tiny</i> , <i>base</i> , <i>small</i> , <i>medium</i> , <i>large</i> ...) para optimizar a relación entre precisión e velocidade de procesamento             |
| <i>Precondicións</i>   | Sistema iniciado                                                                                                                                                                                                            |
| <i>Fluxo Principal</i> | <ol style="list-style-type: none"> <li>1. O usuario accede ás opcións de configuración</li> <li>2. Selecciona o modelo Whisper desexado</li> <li>3. O sistema aplica a configuración para próximos procesamentos</li> </ol> |
| <i>Postcondicións</i>  | Modelo configurado para uso en transcripción ou aliñamento                                                                                                                                                                  |
| <i>Excepcións</i>      | -                                                                                                                                                                                                                           |

## B.2 Resultados do WER adicionais

Como se comentou na sección 6.2.2, pódense ver os resultados do WER das cancións individuais elixidas do dataset na táboa B.11.

| Canción                                   | Small WER (%) | Medium WER (%) | Large-v2 WER (%) |
|-------------------------------------------|---------------|----------------|------------------|
| La favorita (galego)                      | 28.4          | 22.1           | 18.7             |
| A fonte (galego)                          | 31.2          | 25.6           | 21.3             |
| Pirimpimpim (galego)                      | 34.8          | 28.9           | 24.1             |
| Tempestades de sal (galego)               | 26.7          | 20.8           | 17.9             |
| Ocre (español)                            | 8.2           | 4.1            | 2.8              |
| La vereda de la puerta de atrás (español) | 12.1          | 6.3            | 4.2              |
| Entre dos tierras (español)               | 9.7           | 5.2            | 3.1              |
| Slalom (inglés)                           | 6.8           | 3.4            | 2.1              |
| Don't Stop Me Now (inglés)                | 7.3           | 3.9            | 2.6              |
| Someone Like You (inglés)                 | 5.2           | 2.8            | 1.8              |

Táboa B.11: Resultados do WER das cancións individuais do dataset utilizadas



## Imaxes adicionais

---

### C.1 Funcionamento do **encoder-decoder** de **Demucs**

Na sección 2.6.1 explicábase que esta ferramenta utiliza un **encoder** para extraer características e un **decoder** para reconstruír cada fonte. Pois o esquema xeral deste funcionamento pode observarse claramente na figura C.1

### C.2 Análise visual da separación de fontes

As seguintes figuras C.2, C.3, C.4, C.5, C.6 mostran os espectrogramas resultantes da separación producida por **Demucs**. Tal como se explica na sección 5.2.3, estes espectrogramas amosan cómo o algoritmo de separación descompón a mezcla orixinal nos compoñentes de batería, baixos, voces e outros instrumentos para poder facer unha análise visual.

### C.3 Diagrama de fluxo do modo de obtención da instrumental

Nesta sección preséntase o diagrama de secuencia para a opción de obtención da instrumental que se menciona na sección 4.5.3. Este diagrama C.7 mostra as interaccións entre os compoñentes do sistema durante o proceso sen a necesidade de transcripción nin aliñamento temporal.

### C.4 Diagrama da metodoloxía seguida

A metodoloxía iterativa seguida durante o proxecto represéntase mediante un diagrama completo na figura C.8, como se comenta na sección 3.1.

### C.5 Diagramas de clases do sistema

Como se mencionou na sección 4.3, este diagrama de clases ensina unha visión da arquitectura *software* da aplicación. Debido ao gran número de clases e relacións, o diagrama divídese en dúas partes mediante as figuras C.9 e C.10 para mellorar a lexibilidade.

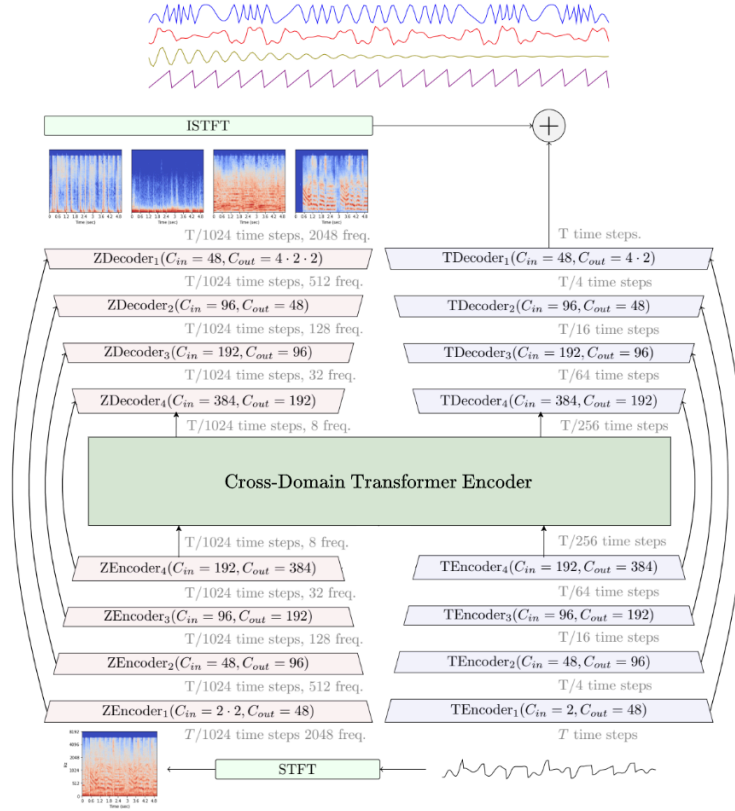
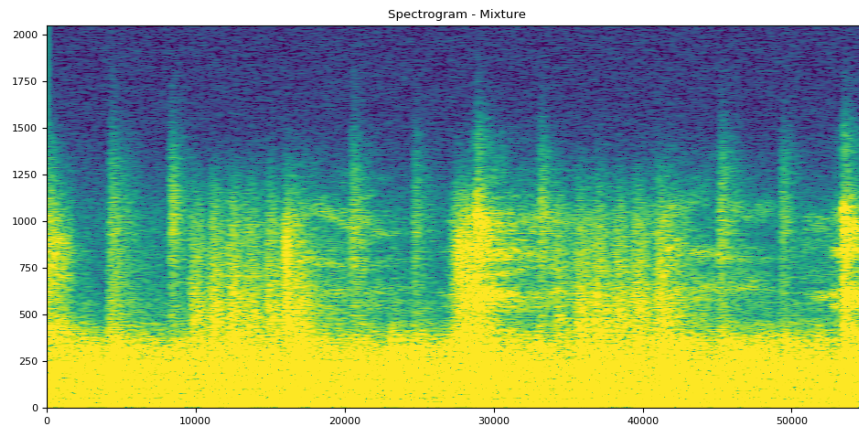
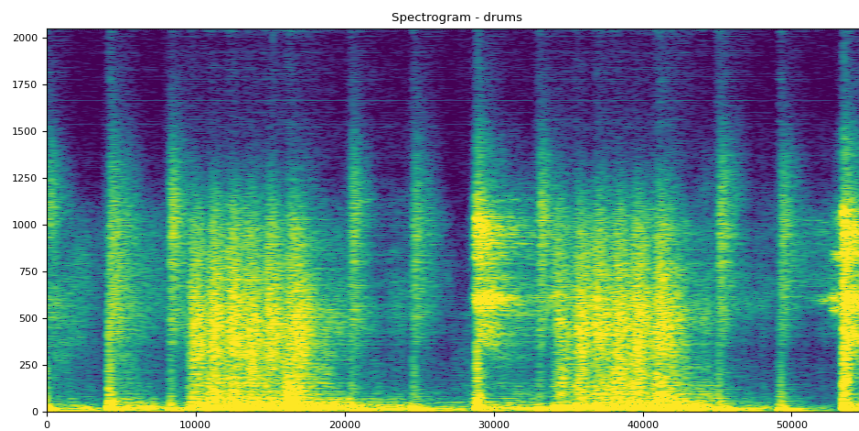
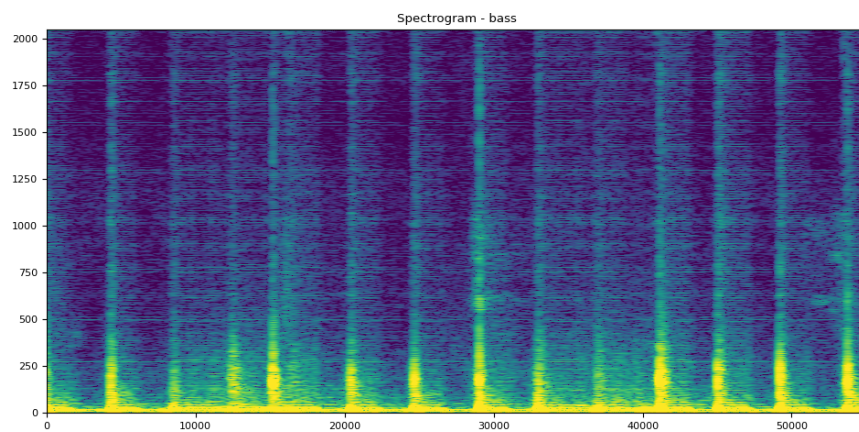
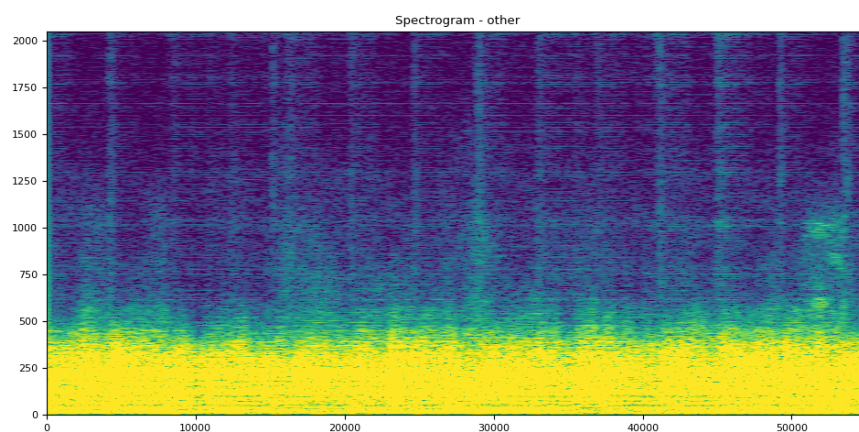


Figura C.1: Funcionamento do encoder-decoder de Demucs

Figura C.2: Espectrograma da mezcla orixinal (*mixture*).

Figura C.3: Espectrograma da pista de bateria (*drums*).Figura C.4: Espectrograma da pista de baixos (*bass*).Figura C.5: Espectrograma da pista de outros instrumentos (*other*).

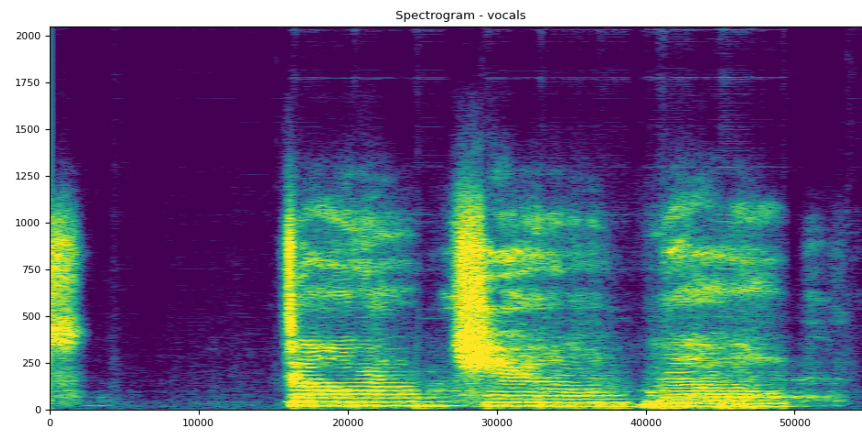
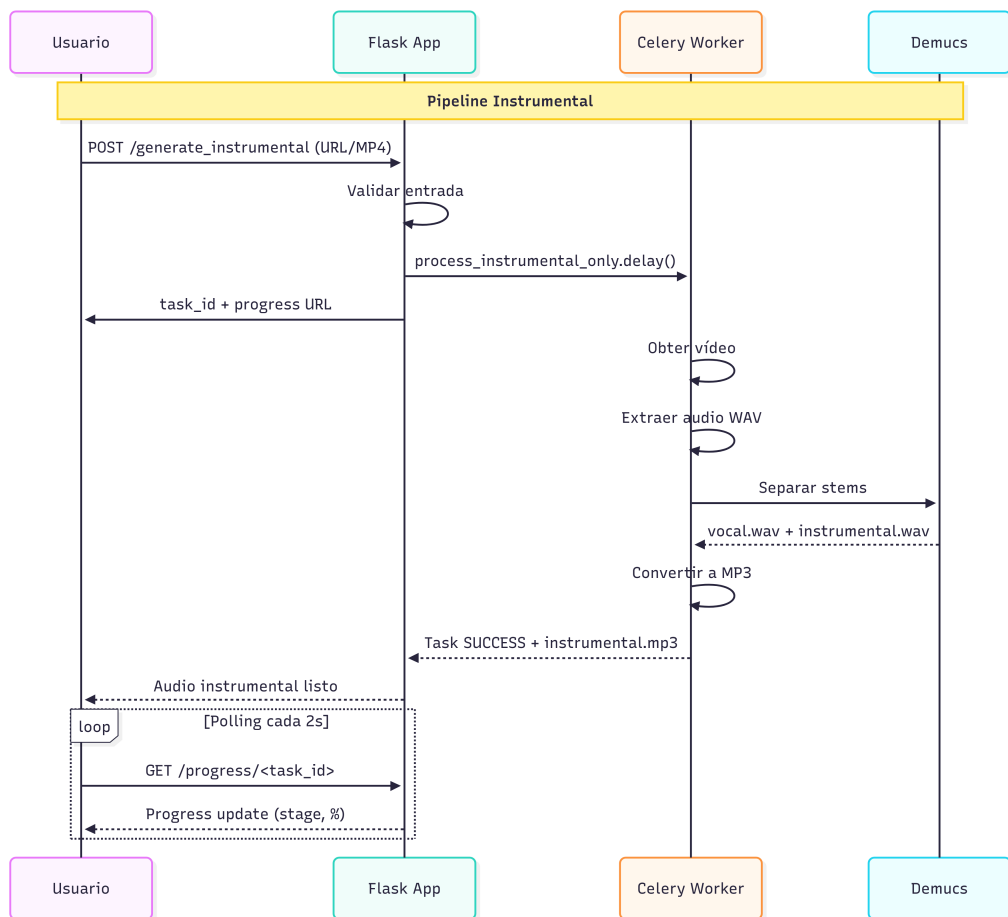
Figura C.6: Espectrograma da pista vocal (*vocals*).

Figura C.7: Diagrama de secuencia para a opción de obter a instrumental

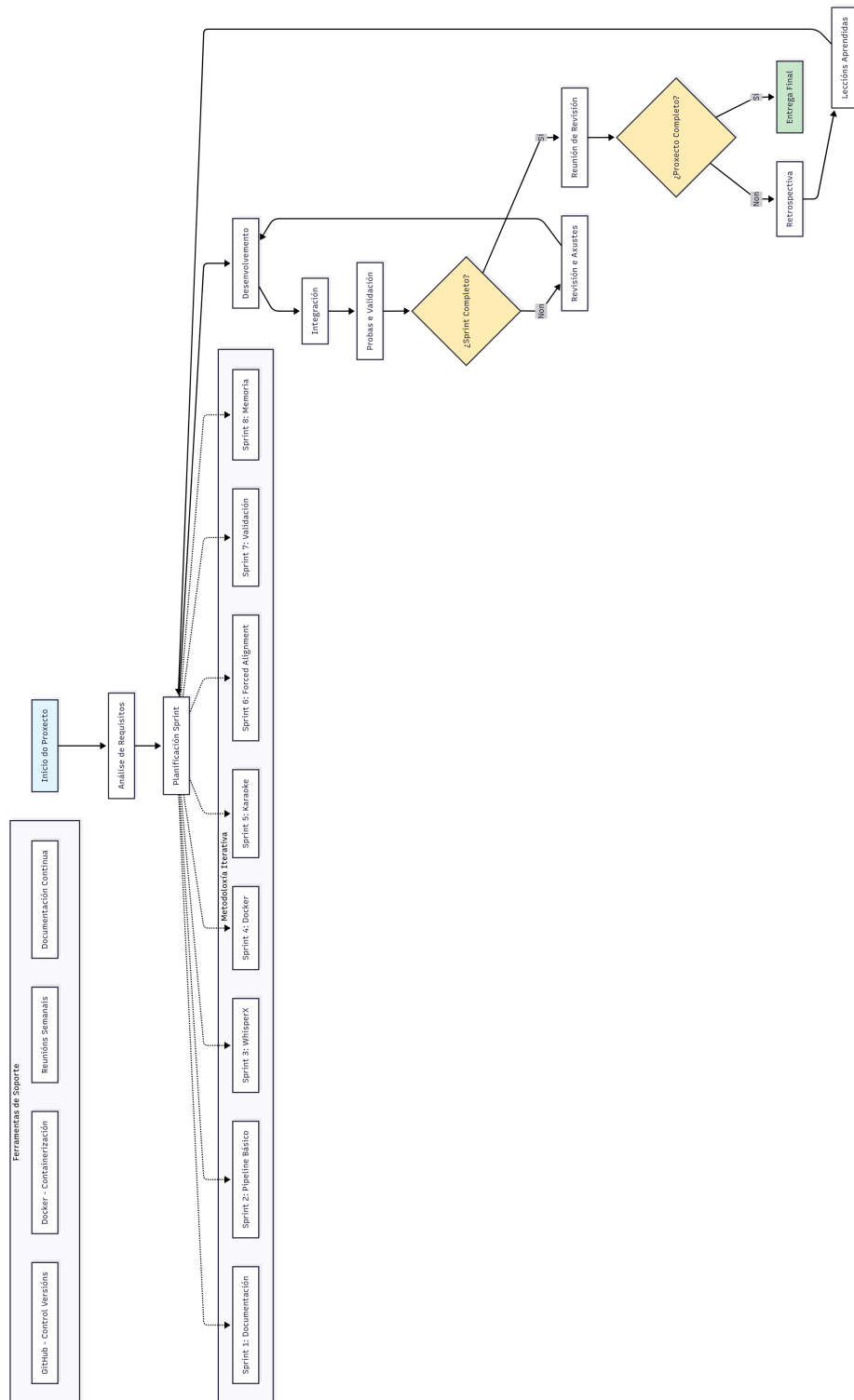


Figura C.8: Esquema da metodoloxía iterativa seguida

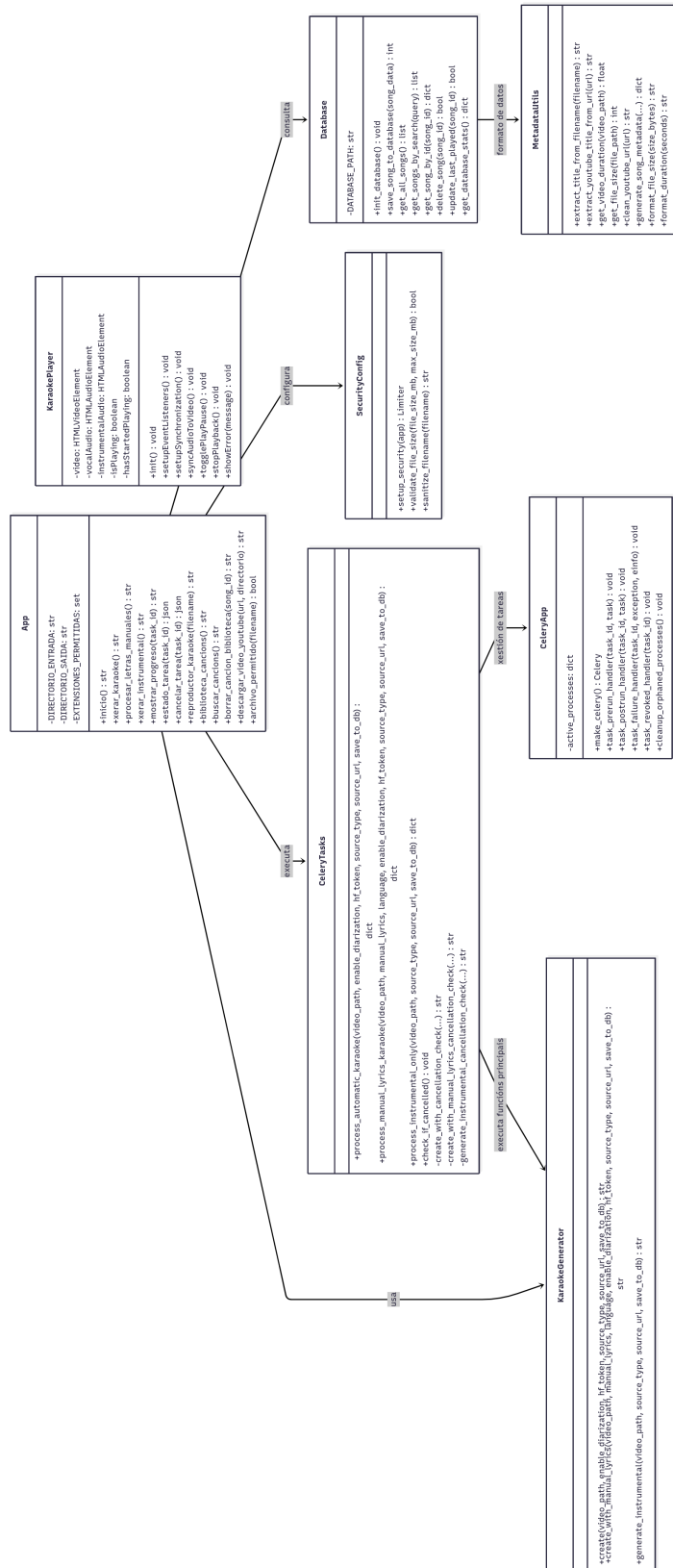
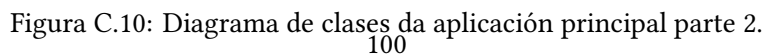


Figura C.9: Diagrama de clases da aplicación principal parte 1.





## C.6 Imaxes da interface web adicionais

Como se comenta nas seccións 5.5.1 e 5.5.3, nas figuras C.11, C.12 e C.13 móstrase, a modo de exemplo, a estética da interface da cabeceira da páxina, do formulario de xeración de instrumental e do sistema de biblioteca, respectivamente. Amósanse a modo de capturas de pantalla recortadas da interface.

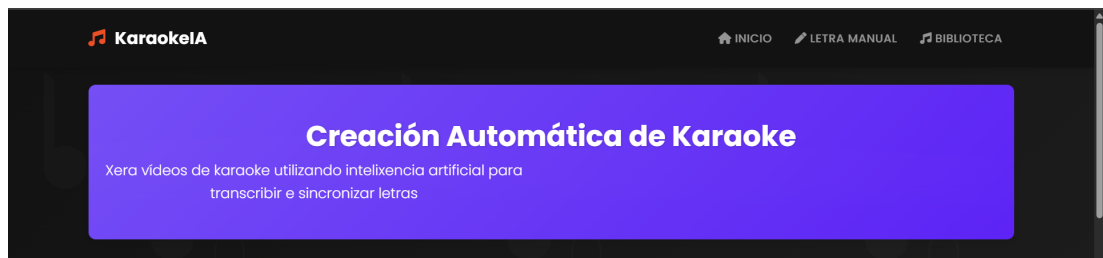


Figura C.11: Cabeceira da páxina principal

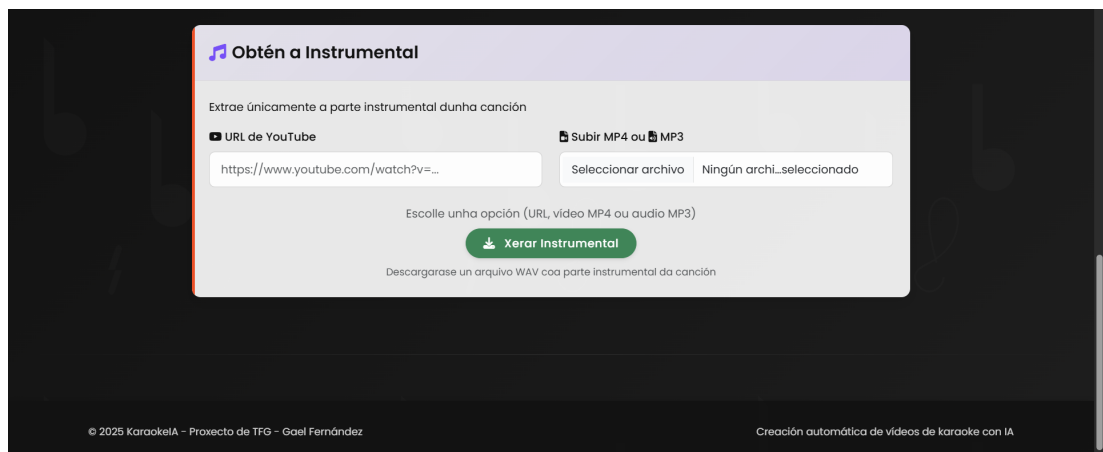


Figura C.12: Formulario para o modo de só instrumental da páxina principal



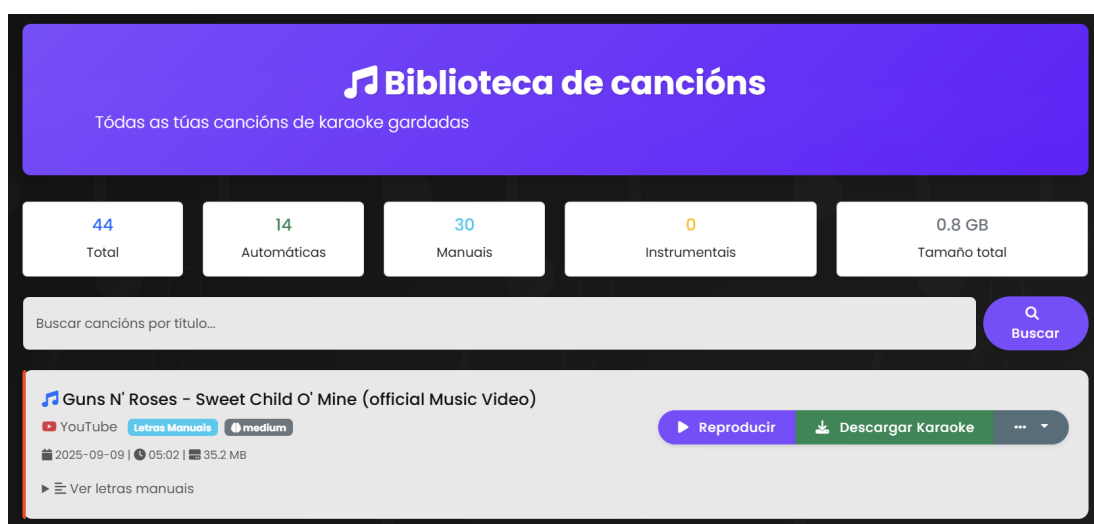


Figura C.13: Interface da biblioteca do sistema

# Relación de Acrónimos

---

- AAC** Advanced Audio Coding. 21, 44
- API** Application Programming Interface. 18, 21, 24, 39, 51
- ASR** Automatic Speech Recognition. 9, 10
- CDG** CD+Graphics. 14
- CNN** Convolutional neural networks. 6
- CPU** Central Processing Unit. 31, 35, 36, 39, 51, 53, 65, 67, 68, 78
- CSS** Cascading Style Sheets. 20
- CTC** Connectionist Temporal Classification. 10, 12
- CU** Use Case. viii, ix, 87–92
- CUDA** Compute Unified Device Architecture. 21, 22, 30, 34, 35
- Demucs** Deep Extractor for Music Sources. iv, vi, viii, 1, 9, 17, 21–23, 28, 29, 31, 35, 39, 45, 47, 50, 51, 53, 67, 68, 77, 78, 87–89, 94, 95
- DNN-HMM** Hybrid Deep Neural Network–Hidden Markov Model. 10, 12
- DTW** Dynamic Time Warping. 10, 18, 54
- FFT** Fast Fourier Transform. 5
- FLAC** Free Lossless Audio Codec. 23
- FPS** Frames Per Second. 59
- GMM** Gaussian Mixture Model. 10
- GPU** Graphics Processing Unit. 1, 14, 21–23, 30, 31, 35, 36, 39, 51, 53, 61, 65, 67, 68, 73, 78, 81, 85
- HMM** Hidden Markov Models. 10–12
- HTML** HyperText Markup Language. 38, 56, 62
- HTTP** HyperText Transfer Protocol. 35, 36, 42, 44, 45, 105
- HTTPS** Hypertext Transfer Protocol Secure. 38, 52
- IA** Artificial Intelligence. 15, 22, 24, 26, 36, 44, 76, 77, 79

- IP** Internet Protocol. 64
- JSON** JavaScript Object Notation. 35, 39, 42, 44, 45
- KAR** Karaoke MIDI file. 14
- LSTM** Long Short-Term Memory. 10, 17
- MCFF** Mel Frequency Cepstral Coefficients. 10, 19
- MFA** Montreal Forced Alignment. 12
- MIDI** Musical Instrument Digital Interface. 14
- MP3** MPEG-1 Audio Layer 3. 44, 48, 53
- MP3+G** MPEG-1 Audio Layer II + CDG. 14
- MP4** MPEG-4 Part 14. viii, 1, 2, 16, 25, 38, 44, 47, 53, 74, 88, 89
- NMF** non negative matrix factorization. 8
- PIL** Python Imaging Library. 19
- RBM** Restricted Boltzmann Machines. 9
- REST** Representational State Transfer. 35, 36, 39, 42, 44, 51
- RIFE** Real-Time Intermediate Flow Estimation. 15
- RNN** Recurrent neural networks. 6, 10
- SAR** Signal-to-Artifacts Ratio. 65, 67
- SDR** Signal-to-Distortion Ratio. 17, 65, 67
- SIR** Signal-to-Interference Ratio. 65, 67
- SQL** Structured Query Language. 20
- SRT** SubRip Subtitle. 35, 40, 43, 45, 85
- STFT** Short-time Fourier transform. 5, 6, 8, 19
- TCP** Transmission Control Protocol. 36
- URL** Uniform Resource Locator. 2, 3, 20–22, 38, 42, 45, 47, 60, 74, 87–89
- UTC** Universal Time Coordinated. 39
- VAD** Voice Activity Detection. 12, 17, 18, 54
- VBR** Variable Bit Rate. 44
- WAV** Waveform Audio Format. 23, 35, 44, 45, 47
- WER** Word Error Rate. ix, 65, 69, 75, 93

# Glosario

---

- alpha blending** Técnica de composición gráfica que combina dúas imaxes ou capas usando un canal alfa para controlar a transparencia, permitindo transicións suaves. [13](#)
- clustering** Grupo de computadores conectados que traballan xuntos como un único sistema para aumentar capacidade e fiabilidade. [57](#)
- dataset** Conxunto estruturado de datos utilizados para adestramento, validación ou probas de modelos de intelixencia artificial. [viii](#), [ix](#), [7](#), [9](#), [17](#), [18](#), [66–70](#), [72](#), [77–79](#), [93](#)
- decoder** Compoñente que reconstrúe os datos orixinais ou xera unha saída desexada a partir da representación codificada polo encoder. [iv](#), [vi](#), [17](#), [94](#), [95](#)
- encoder** Compoñente dun modelo de intelixencia artificial que transforma datos de entrada nun espazo latente de características, comprimindo a información relevante para o procesamento posterior. [iv](#), [vi](#), [17](#), [94](#), [95](#)
- endpoint** Punto de comunicación nun servizo web onde as aplicacións poden acceder aos recursos mediante peticións [HTTP](#). [39](#), [64](#)
- fine-tuning** Proceso de axustar un modelo preentrenado para unha tarefa específica mediante adestramento adicional. [79](#)
- pitch shifting** Procesamento de sinal de audio que modifica a frecuencia fundamental (ton) dunha gravación sen alterar a súa duración temporal, utilizado para cambiar a altura tonal de voces ou instrumentos. [19](#)
- polling** Técnica de consulta periódica ao servidor para verificar o estado das tarefas de procesamento, implementada na interface web para mostrar progreso en tempo real. [38](#), [61](#)
- redes neuronais** Arquitectura de aprendizaxe automática formada por capas de nodos interconectados que transforman datos de entrada mediante funcións de activación e pesos axustables durante o adestramento. [5](#), [6](#), [8–10](#), [12](#)
- scraping** Proceso automatizado de extracción e recollida de datos dende fontes dixitais como páxinas web, APIs ou bases de datos. [79](#)

# Bibliografía

---

- [1] H. Jeon, Y. Jung, S. Lee, and Y. Jung, “Area-efficient short-time fourier transform processor for time–frequency analysis of non-stationary signals,” *Applied Sciences*, vol. 10, no. 20, p. 7208, 2020. [En línea]. Disponible en: <https://www.mdpi.com/2076-3417/10/20/7208>
- [2] M. AI, “wav2vec 2.0: Learning the structure of speech from raw audio,” 2020, acceso: 18 de setembre de 2025. [En línea]. Disponible en: <https://ai.meta.com/blog/wav2vec-20-learning-the-structure-of-speech-from-raw-audio/>
- [3] G. DeepMind, “Wavenet: A generative model for raw audio,” 2016, acceso: 18 de setembre de 2025. [En línea]. Disponible en: <https://deepmind.google/research/projects/wavenet/>
- [4] A. Défossez and F. Research, “Demucs: Deep extractor for music sources,” 2023, acceso: 18 de setembre de 2025. [En línea]. Disponible en: <https://github.com/facebookresearch/demucs>
- [5] M. Bain, J. Huh, T. Han, and A. Zisserman, “Whisperx: Time-accurate speech transcription of long-form audio,” *INTERSPEECH 2023*, 2023.
- [6] M. J. F. Gales, “Automatic speech recognition,” 2023, acceso: 18 de setembre de 2025. [En línea]. Disponible en: [https://mi.eng.cam.ac.uk/~mjfg/mjfg\\_NOW.pdf](https://mi.eng.cam.ac.uk/~mjfg/mjfg_NOW.pdf)
- [7] K. Doshi, “Audio deep learning made simple: Automatic speech recognition (asr) - how it works,” 2021, acceso: 18 de setembre de 2025. [En línea]. Disponible en: <https://medium.com/data-science/audio-deep-learning-made-simple-automatic-speech-recognition-asr-how-it-works-716cfce4c706>
- [8] OpenAI, “Whisper: Robust speech recognition via large-scale weak supervision,” 2022, acceso: 18 de setembre de 2025. [En línea]. Disponible en: <https://github.com/openai/whisper>
- [9] J. Koetsier, “Forced alignment: How to match audio with a transcript via machine learning,” 2021, acceso: 18 de setembre de 2025. [En línea]. Disponible en: <https://techfirst.medium.com/forced-alignment-how-to-match-audio-with-a-transcript-via-machine-learning-dd19da8c0f04>
- [10] M. McAuliffe, M. Socolof, S. Mihuc, M. Wagner, and M. Sonderegger, “Montreal Forced Aligner: Trainable Text-Speech Alignment Using Kaldi,” in *Proc. Interspeech 2017*, 2017, pp. 498–502.
- [11] Wikipedia, “Algoritmo de viterbi,” 2024, acceso: 18 de setembre de 2025. [En línea]. Disponible en: [https://es.wikipedia.org/wiki/Algoritmo\\_de\\_Viterbi](https://es.wikipedia.org/wiki/Algoritmo_de_Viterbi)

- [12] P. Team, “Forced alignment with wav2vec2,” 2023, acceso: 18 de setembro de 2025. [En línea]. Disponible en: [https://docs.pytorch.org/audio/stable/tutorials/forced\\_alignment\\_tutorial.html](https://docs.pytorch.org/audio/stable/tutorials/forced_alignment_tutorial.html)
- [13] J. D. Foley, A. Van Dam, S. K. Feiner, and J. F. Hughes, *Computer Graphics: Principles and Practice*, 2nd ed. Addison-Wesley, 1995, capítulo sobre composición y alpha blending.
- [14] KaraPlay, “Diferencias entre midi y kar,” 2023, acceso: 18 de setembro de 2025. [En línea]. Disponible en: <https://blog.karaoplay.com/diferencias-entre-midi-y-kar/>
- [15] F. Team, “Ffmpeg: A complete, cross-platform solution to record, convert and stream audio and video,” 2023, acceso: 18 de setembro de 2025. [En línea]. Disponible en: <https://ffmpeg.org/documentation.html>
- [16] G. Bradski, “The opencv library,” in *Dr. Dobb’s Journal of Software Tools*, 2000.
- [17] Zulko, “Moviepy: Video editing with python,” 2017, acceso: 18 de setembro de 2025. [En línea]. Disponible en: <https://moviepy.readthedocs.io/>
- [18] Z. Huang, T. Zhang, W. Heng, B. Shi, and S. Zhou, “Real-time intermediate flow estimation for video frame interpolation,” in *Proceedings of the European Conference on Computer Vision (ECCV)*, 2022.
- [19] S. Inc., “Smule: Social singing karaoke app,” 2023, acceso: 18 de setembro de 2025. [En línea]. Disponible en: <https://www.smule.com/>
- [20] KaraFun, “Karafun: Karaoke online y app,” 2023, acceso: 18 de setembro de 2025. [En línea]. Disponible en: <https://www.karafun.es/>
- [21] Singa, “Singa: Karaoke platform,” 2023, acceso: 18 de setembro de 2025. [En línea]. Disponible en: <https://singa.com/>
- [22] K. M. Team, “Karaoke mugen: Community-driven karaoke software,” 2023, acceso: 18 de setembro de 2025. [En línea]. Disponible en: <https://mugen.karaokes.moe/>
- [23] U. D. Team, “Ultrastar deluxe: Free and open source karaoke game,” 2023, acceso: 18 de setembro de 2025. [En línea]. Disponible en: <https://usdx.eu/>
- [24] S. Rouard, F. Massa, and A. Défossez, “Hybrid transformers for music source separation,” in *ICASSP 23*, 2023.
- [25] Aicrowd, S. G. Corporation, Moises.AI, and M. E. R. Laboratories, “Sound demixing challenge 2023,” 2023, acceso: 18 de setembro de 2025. [En línea]. Disponible en: <https://www.aicrowd.com/challenges/sound-demixing-challenge-2023>
- [26] L. D. Team, “Librosa: Audio and music signal analysis in python,” 2023, acceso: 18 de setembro de 2025. [En línea]. Disponible en: <https://librosa.org/doc/latest/index.html>
- [27] N. Developers, “Numpy: The fundamental package for scientific computing with python,” 2023, acceso: 18 de setembro de 2025. [En línea]. Disponible en: <https://numpy.org/>
- [28] P. D. Team, “Pyphen: Pure python module to hyphenate text,” 2023, acceso: 18 de setembro de 2025. [En línea]. Disponible en: <https://pyphen.org/>
- [29] —, “Pillow: Python imaging library,” 2023, acceso: 18 de setembro de 2025. [En línea]. Disponible en: <https://pillow.readthedocs.io/en/stable/>

- [30] S. Consortium, “Sqlite documentation,” 2023, acceso: 18 de setembro de 2025. [En línea]. Disponible en: <https://www.sqlite.org/docs.html>
- [31] P. Team, “Flask: A lightweight wsgi web application framework,” 2023, acceso: 18 de setembro de 2025. [En línea]. Disponible en: <https://flask.palletsprojects.com/en/stable/>
- [32] B. Team, “Bootstrap: The most popular html, css, and js library in the world,” 2023, acceso: 18 de setembro de 2025. [En línea]. Disponible en: <https://getbootstrap.com/>
- [33] Wikipedia, “Javascript,” 2024, acceso: 18 de setembro de 2025. [En línea]. Disponible en: <https://es.wikipedia.org/wiki/JavaScript>
- [34] D. Inc., “Docker: Accelerated container application development,” 2023, acceso: 18 de setembro de 2025. [En línea]. Disponible en: <https://www.docker.com/>
- [35] yt-dlp contributors, “yt-dlp: A youtube-dl fork with additional features and fixes,” 2023, acceso: 18 de setembro de 2025. [En línea]. Disponible en: <https://github.com/yt-dlp/yt-dlp>
- [36] ngrok Inc., “ngrok: Getting started documentation,” 2023, acceso: 18 de setembro de 2025. [En línea]. Disponible en: <https://ngrok.com/docs/getting-started/>
- [37] Wikipedia, “Latex,” 2024, acceso: 18 de setembro de 2025. [En línea]. Disponible en: <https://es.wikipedia.org/wiki/LaTeX>
- [38] Microsoft, “Visual studio code: Code editing. redefined,” 2023, acceso: 18 de setembro de 2025. [En línea]. Disponible en: <https://code.visualstudio.com/>
- [39] Overleaf, “Overleaf: Editor latex colaborativo en línea,” 2023, acceso: 18 de setembro de 2025. [En línea]. Disponible en: <https://es.overleaf.com/project>
- [40] Wemake, “Mermaid.js: Diagramas como código,” 2021, acceso: 18 de setembro de 2025. [En línea]. Disponible en: <https://blog.wemake.pe/posts/2021-11-07-mermaid-js/>
- [41] G. Fernandez, “KaraokeProject,” <https://github.com/gaelfernandez1/KaraokeProject/>, 2025, acceso: 18 de setembro de 2025.
- [42] C. Project, “Backends and brokers — celery 5.5.3 documentation,” 2025, acceso: 18 de setembro de 2025. [En línea]. Disponible en: <https://docs.celeryq.dev/en/stable/getting-started/backends-and-brokers/index.html>
- [43] R. User, “Iterative model used in software development,” 2023, acceso: 18 de setembro de 2025. [En línea]. Disponible en: [https://www.researchgate.net/publication/371902841\\_Iterative\\_Model\\_Used\\_in\\_Software\\_Development](https://www.researchgate.net/publication/371902841_Iterative_Model_Used_in_Software_Development)
- [44] Wrike, “What is a sprint in agile?” Project Management Guide, consultado en 18 de setembro de 2025. [En línea]. Disponible en: <https://www.wrike.com/project-management-guide/faq/what-is-a-sprint-in-agile/>
- [45] S. Logic, “Microservices architecture with docker containers,” 2023, acceso: 18 de setembro de 2025. [En línea]. Disponible en: <https://www.sumologic.com/blog/microservices-architecture-docker-containers>
- [46] M. Richards, “Software architecture patterns,” 2015, acceso: 18 de setembro de 2025. [En línea]. Disponible en: <https://www.oreilly.com/library/view/software-architecture-patterns/9781491971437/ch01.html>

- [47] C. Luo, “Understanding web functionality: Server-side, client-side, state, and sessions,” 2025, acceso: 18 de setembro de 2025. [En línea]. Disponible en: [https://dev.to/carrie\\_luo1/understanding-web-functionality-server-side-client-side-state-and-sessions-1e61](https://dev.to/carrie_luo1/understanding-web-functionality-server-side-client-side-state-and-sessions-1e61)
- [48] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, “Design patterns: Elements of reusable object-oriented software,” 1995, acceso: 18 de setembro de 2025. [En línea]. Disponible en: <https://www.javier8a.com/itc/bd1/articulo.pdf>
- [49] R. Guru, “Command design pattern,” 2025, acceso: 18 de setembro de 2025. [En línea]. Disponible en: <https://refactoring.guru/design-patterns/command>
- [50] E. Freeman, E. Freeman, B. Bates, and K. Sierra, *Head First Design Patterns*. O’ Reilly & Associates, Inc., 2004.
- [51] SourceMaking, “Template method design pattern,” 2025, acceso: 18 de setembro de 2025. [En línea]. Disponible en: [https://sourcemaking.com/design\\_patterns/template\\_method](https://sourcemaking.com/design_patterns/template_method)
- [52] S. Kim, “Music source separation with hybrid demucs — torchaudio tutorial,” 2025, acceso: 18 de setembro de 2025. [En línea]. Disponible en: [https://docs.pytorch.org/audio/main/tutorials/hybrid\\_demucs\\_tutorial.html](https://docs.pytorch.org/audio/main/tutorials/hybrid_demucs_tutorial.html)
- [53] B. T. R. On, “Demucs: A deep dive into the ultimate audio source separation model,” 2025, acceso: 18 de setembro de 2025. [En línea]. Disponible en: <https://beatstorapon.com/blog/demucs-a-deep-dive-into-the-ultimate-audio-source-separation-model/>
- [54] Wikipedia, “Word error rate,” 2024, acceso: 18 de setembro de 2025. [En línea]. Disponible en: [https://es.wikipedia.org/wiki/Word\\_Error\\_Rate](https://es.wikipedia.org/wiki/Word_Error_Rate)